

Compute Systems Engine: 代码 实践卷

从 reference 到可恢复分布式计算引擎

CPU Performance Study

本卷是《从 C++ 到计算系统：第二册》的配套代码实践卷，专门承接完整 C++20 实现、测试、benchmark、故障注入和实验报告。

前言

第二册原理卷负责回答“为什么”：为什么热循环会被分支、依赖和内存等待限制，为什么 false sharing 会让正确程序扩展失败，为什么条件变量必须围绕等待谓词，为什么异步 I/O 的 submit、completion 和 commit 不是同一件事，为什么分布式系统必须区分逻辑任务和物理 attempt。本卷负责回答“怎样落地”：怎样把这些原则写成一个可以构建、测试、测量、故障注入和复盘的 C++20 工程。

本卷的贯穿项目统一称为 **Compute Systems Engine**（计算系统引擎）。它从一个日志统计任务开始：输入是一批记录，输出是按 key 聚合的稳定结果。第一阶段只要求顺序 reference、输入合同和诊断；后续逐步加入 split、batch、冷热字段、benchmark、并行内核、线程池、有界队列、原子协议、任务图、可靠写、manifest、checkpoint、MessageBus、attempt、epoch、分片和故障注入。每一步都必须有测试和报告，不接受只写代码、不解释边界的实现。

本卷不是代码片段合集，而是一部工程教材。每章都遵循同一条路线：先说明本章要兑现原理卷中的哪个机制；再给出接口和目录组织；随后实现最小版本、测试失败输入、补充报告字段；最后说明这个版本还不能处理什么，下一章为什么必须继续扩展。读者应当把代码、测试和报告看成一个整体：代码实现机制，测试固定语义，报告保存证据。

核心思想

代码实践卷的目标不是写一个庞大的教学框架，而是让每一项复杂度都有购买理由：它修复了哪个失败，保留了哪个正确性边界，带来了什么运行成本，怎样用测试和报告证明。

常见缩写和术语先导

代码实践卷会把第二册的概念落到工程对象。第一次看到下面这些词时，先把它们理解成“代码里必须有边界的对象”，而不是抽象口号；每个对象都要能在代码、测试、probe 输出或报告里找到证据。英文全称不是背单词用的，它用来防止把相近概念混成一个词。

CPU: Central Processing Unit 中央处理器。它执行指令、访问寄存器和内存，并通过缓存、分支预测、流水线和乱序执行影响程序速度。本卷的实验默认先落在本地 CPU 上，再逐步讨论 SIMD、线程、I/O 和分布式扩展。

HPC: High Performance Computing 高性能计算。它关注吞吐、延迟、并行度、内存带宽和资源利用率。本卷提到 HPC 时，不是泛泛追求“更快”，而是要求用 benchmark、PMU 证据、输入规模和正确性校验说明快在哪里。

PMU: Performance Monitoring Unit 性能监控单元。现代 CPU 用 PMU 统计 retired instructions、cache miss、branch miss 等硬件事件。实践卷的 [model_ch05b_pmu_simd_evidence](#) 用它提醒读者：只看耗时不够，必须知道程序到底执行了哪些指令、触发了哪些硬件事件。

SIMD: Single Instruction, Multiple Data 单指令多数据。一条向量指令同时处理多个 lane。第 7 章模型会统计 vector iterations、scalar tail 和 lane utilization，说明“用了 SIMD”不等于“每个 lane 都被有效利用”。

ILP: Instruction-Level Parallelism 指令级并行。它指 CPU 在不改变语义的前提下，让多条独立指令重叠执行的能力。本卷用它解释为什么数据依赖、分支和内存延迟会限制热循环吞吐。

TLB: Translation Lookaside Buffer 地址转换后备缓冲。CPU 访问虚拟地址时需要查页表，TLB 缓存最近的虚拟页到物理页映射。第 3 章模型把 cache line、page 和 TLB miss 分开统计，避免把所有内存慢都叫“缓存问题”。

NUMA: Non-Uniform Memory Access 非一致内存访问。多路或多节点机器上，本地内存和远端内存访问成本不同。第 4 章模型把 local access、remote access 和 coherence transfer 分开记录。

SMP: Symmetric Multiprocessing 对称多处理。多个 CPU 核心共享同一操作系统实例和统一内存视图。它是理解线程调度、锁竞争和共享队列的背景，但 NUMA 机器上的“共享内存”仍可能有远近差异。

LLC: Last-Level Cache 最后一级缓存，通常指 L3 或芯片上离内存最近的共享缓存。它影响跨核心共享数据、工作集大小和内存带宽压力。

L1/L2/L3 Cache: Level 1/2/3 Cache 一级、二级、三级缓存。L1 最靠近核心、容量小、延迟低；L2 居中；L3/LLC 容量更大、常被多个核心共享。读缓存模型时要区分 cache line 命中、容量 miss 和 TLB miss。

IPC: Instructions Per Cycle 每周期退休指令数。它粗略描述 CPU 每个时钟周期完成多少条指令。IPC 低可能来自分支失败、缓存 miss、依赖链、同步等待或 I/O 阻塞，不能单独当成最终答案。

GFLOPS: Giga Floating-Point Operations Per Second 每秒十亿次浮点运算。Roofline 模型用它表示计算上限和可达性能；但没有 bytes、bandwidth 和 checksum，GFLOPS 数字没有上下文。

I/O: Input/Output 输入输出。文件、网络、管道和设备读写都属于 I/O。本卷强调 submit、completion、commit 不是同一件事：提交请求、收到完成事件、业务确认持久化必须分开建模。

EOF: End Of File 文件结束。EOF 不是错误，也不等于短读。读取循环要区分“正常结束”“本次只读到部分数据”和“系统调用失败”。

CLI: Command-Line Interface 命令行接口。CLI 用参数、标准输入输出和退出码表达工具行为。本卷 CLI 只拼接输入和输出，不复制 parser、reference 或 split 逻辑。

- CI: Continuous Integration** 持续集成。每次提交后自动构建、测试和检查，防止本地偶然通过的代码进入主线。本卷的最小 CI 思路就是 CMake 构建、CTest、PDF/EPUB 构建和导出校验。
- JSON: JavaScript Object Notation** 结构化文本格式。适合保存配置、manifest 和机器可读报告。它比随手拼字符串更适合后续由脚本解析。
- CSV: Comma-Separated Values** 逗号分隔文本表格。适合保存 benchmark 结果、trace 摘要和 probe 输出，但字段顺序必须稳定。
- MPMC: Multiple Producer, Multiple Consumer** 多生产者多消费者。多个线程可以同时入队和出队。实践卷的 bounded queue 使用 MPMC 语义，因此要测试容量、等待、关闭和唤醒。
- MPSC: Multiple Producer, Single Consumer** 多生产者单消费者。多个生产者写入，一个消费者读取，常见于日志队列和事件汇聚。它比 MPMC 简单，但消费者成为固定汇聚点。
- SPSC: Single Producer, Single Consumer** 单生产者单消费者。一个生产者、一个消费者，常用于 ring buffer。第 12 章模型用 SPSC ring 固定 push、pop、full 和 empty 边界。
- CAS: Compare-And-Swap** 比较并交换。它读取内存位置，若旧值等于期望值则写入新值，是很多无锁结构的基础。CAS 失败不一定是错误，通常表示其他线程已经抢先修改。
- ABA Problem** ABA 问题。线程看到值从 A 变成 B 又变回 A，单看当前值会误以为没有变化。无锁栈、节点复用和内存回收都要考虑 ABA。
- RCU: Read-Copy-Update** 读-复制-更新。读者不加重锁读取旧版本，写者复制并发布新版本，等待旧读者离开后再回收旧对象。本卷用它帮助理解 epoch reclamation 的动机。
- RAII: Resource Acquisition Is Initialization** 资源获取即初始化。C++ 用对象生命周期管理锁、文件、内存和线程 join 等资源。实践代码应让析构自动释放资源，减少异常路径泄漏。
- RPC: Remote Procedure Call** 远程过程调用。调用者像调用本地函数一样发起远端请求，但工程上必须处理超时、重试、幂等和版本。第 16 到 18 章的 attempt、quorum、checkpoint 都是 RPC/分布式系统的基础问题。
- TCP: Transmission Control Protocol** 传输控制协议。它提供有序、可靠、字节流传输，但不提供业务消息边界、幂等和事务提交。上层仍要定义 framing、retry 和 manifest。
- TSan: ThreadSanitizer** 线程检查器，用于发现数据竞争和部分同步错误。它不能证明并发程序完全正确，但能尽早暴露未受保护的共享访问。
- GTest: GoogleTest** C++ 单元测试框架。本卷用测试固定正常输入、坏行、split 边界、队列关闭和运行时行为。
- GMock: GoogleMock** GoogleTest 配套的 mock 框架。它用于构造可控依赖和交互断言；本卷把它作为工程能力边界保留，即使某些小实验暂时不需要复杂 mock。
- Google Benchmark** C++ benchmark 库，用于重复测量函数成本。本卷 benchmark 只测目标热路径，不把随机输入生成、文件读取和报告打印混入同一次测量。
- reference** 参考实现。它可以慢，但必须清楚、确定、可复查，用来定义正确结果。优化版只有在输出满足 reference 合同后，才有资格谈性能。
- oracle** 判定器。它比较 reference 和待测实现，判断输出是否满足同一语义和误差合同。
- contract** 合同。这里不是法律文件，而是代码边界的明确约定：输入是什么，输出是什么，错误怎样表达，哪些行为不承诺。
- manifest** 发布清单。它记录输入版本、输出位置、校验和、参数和提交事实，让一次运行可以被脚本复查。
- checkpoint** 检查点。保存可恢复状态，让失败后可以从已提交边界继续，而不是从头重跑。
- attempt** 一次物理尝试。同一个逻辑任务可能因为超时或失败重试而产生多个 attempt；最终报告要区分逻辑任务和物理尝试。
- epoch** 版本或轮次编号。它用来区分不同时刻的状态和提交边界，也是很多内存回收和分布式提交协议的基本字段。
- watermark** 水位线。表示某个进度之前的数据已经被观察、处理或提交。流处理和恢复系统用它决定哪些旧状态可以清理。

quorum 法定多数或提交多数。分布式副本里，只要达到规定数量的确认就能做出提交判断。第 17 章模型用 replica count 和 acknowledgements 推导 quorum size。

futex: Fast Userspace Mutex Linux 的快速用户态互斥原语。无竞争时留在用户态，有竞争时进入内核等待或唤醒。实践卷用 futex probe 说明 wait/wake 不是普通自旋。

eventfd Linux 事件计数文件描述符。常用于把线程或内核事件接入 **epoll**。它不是消息队列，只是一个可读的计数通知通道。

epoll Linux I/O 事件复用接口。它让程序等待多个文件描述符的 readiness 事件。实践卷里要区分 readiness、实际 drain 和业务 completion。

UTF-8: Unicode Transformation Format 8-bit Unicode 的常见变长字节编码。实践卷保存 offset 时默认按字节定位；如果 UI 要按字符显示，不能把字节下标直接当字符下标。

全书结构

本卷按 Compute Systems Engine 的工程演化组织，但不再把一个小动作拆成一个目录项。正文每章控制在三到四个大节：先给具体问题和输入样例，再推导类型与接口，接着落到实现、测试和探针，最后给阶段验收。细节仍然要讲透，只是放在大节内部展开。

本卷现在有两条可运行 lab 主线。第一条是 `reference_pipeline`，从 `event, value` 输入开始，建立输入身份、reference、结构化诊断、report、manifest、split、partial result 和 merge。第二条是 `compute_systems`，承接第二册原理卷，为 CPU、缓存、TLB、NUMA、SIMD、线程调度、同步、原子、任务运行时、异步 I/O、分布式 quorum 和 checkpoint/recovery 提供逐章模型、探针和测试。

每个阶段都必须产出四类东西。第一，稳定接口，例如 `InputSplit`、`RecordId`、`HotBatch`、`QueueContract`、`ManifestEntry` 或 `RunReport`。第二，可运行实现，至少包含 reference 和一个带机制的版本。第三，测试，包括正常输入、边界输入和能击穿朴素方案的失败输入。第四，报告，记录正确性、性能、资源、故障和未验证边界。

本卷与第二册原理卷并行阅读。原理卷先解释机制为什么存在，本卷再把机制落成代码。若读者只读本卷而跳过原理卷，容易把接口当成固定答案；若只读原理卷而不做本卷，容易把工程边界停留在口头推导。两卷合起来，才形成从零到精通的完整训练。

目录

常见缩写和术语先导	iii
第一部分 从 reference 到可测试工程	
第 1 章 输入合同、Reference 与第一组测试	2
第二部分 Reference Pipeline 实践	
第 2 章 实践卷：两条代码主线怎样落地	6
完整源码清单	10

第零部分

从 reference 到可测试工程

第 1 章

输入合同、Reference 与第一组测试

1.1 为什么先写 reference

本章实现 Compute Systems Engine 的第一个可运行切片：输入合同、顺序 reference、诊断结果、稳定输出和测试。它对应原理卷第一章。此时不做并行、不做 SIMD、不做 manifest，也不追求吞吐；目标只有一个：让项目拥有一把可信尺子。后续所有优化都必须先通过这把尺子。

reference（参考实现）是可信但不追求快的版本。它的价值不是“代码短”，而是定义正确结果。后面无论换成 SIMD、线程池、异步 I/O 还是分布式 attempt，都必须回答同一个问题：新版本是否仍然和 reference 表达同一份语义。若 reference 自身含糊，后面的性能数字只是在测一个没有定义清楚的程序。

contract（合同）不是法律文件，而是函数边界的承诺：输入记录长什么样，缺字段算什么错误，输出 key 是否排序，计数溢出是否允许。合同必须落到字段、返回值、错误类型和测试断言里。只写“输入是日志”没有工程价值；写出 **event, value**、缺字段为 **missing_field**、多字段为 **extra_field**，才算进入代码。

diagnostic（诊断记录）是结构化错误事实，不是随手打印的日志。它要说明哪份输入、哪一代、哪一行、哪个字节位置、为什么失败。诊断质量会直接决定后续并行 split、恢复 replay 和分布式重试能不能复现事故。第一版若只输出“parse error”，后面所有复杂机制都会失去定位基础。

核心思想

第一章只买四种能力：输入身份、顺序 reference、结构化诊断和稳定报告。任何不能服务这四件事的复杂度，都应留到后续章节。

1.2 第一版接口怎样定

第一版接口要小，但不能含糊。建议先定义四个概念：**RecordId**、**ParseDiagnostic**、**ReferenceResult** 和 **InputCase**。完整字段可以随项目扩展，但第一版必须能回答三件事：哪条记录被接受，哪条记录被拒绝，最终计数是否稳定。

Listing 1.1 第一版核心结构

```
1 struct RecordId {
2     std::string input_name;
3     std::uint64_t input_generation = 0;
4     std::uint64_t byte_offset = 0;
5     std::uint64_t line_number = 0;
6 };
7
8 struct ParseDiagnostic {
9     RecordId record;
10    std::string error_kind;
11 };
12
13 struct ReferenceResult {
14     std::uint64_t accepted = 0;
15     std::vector<ParseDiagnostic> diagnostics;
16     std::map<std::string, std::uint64_t> counts;
17 };
```

RecordId 是记录身份。它把一条输入记录定位到 input name、generation、byte offset 和 line number。generation 是“同名输入的第几代”，不是装饰字段。后续 retry、checkpoint 或恢复时，旧 generation 的结果不能覆盖新 generation。

ParseDiagnostic 只保存可复查事实，不保存临时调试字符串。错误类型应稳定，例如 **missing_field**、**extra_field**、**empty_event**、**bad_value**。这样测试可以断言错误类别，报告可以聚合错误分布，用户也能根据错误类型定位合同违反点。

ReferenceResult 使用 **std::map** 保存 counts，是为了让输出 key 顺序稳定。稳定顺序不是小事：报告、checksum、manifest 和回归测试都依赖它。等后续进入热路径优化，可以用哈希表或分片结构提高吞吐，但发布报告前仍要回到稳定顺序。

1.3 测试先固定语义

第一组测试不追求覆盖所有未来功能，只固定最核心语义。正常五行输入应得到 **view=2**；缺字段输入应进入 diagnostics；字段过多、空事件名和 value 非整数都应有明确错误类型；输出 key 顺序必须稳定。若这些测试不稳定，后续任何性能实验都应暂停。

手算样例如下：

```
1 view,1
2 click,1
3 view,2
```

这三行的 **records_total** 是 3，**accepted** 是 3，**counts["view"]** 是 2，**counts["click"]** 是 1。这个例子很小，但它固定了两条语义：同一个 event 的多条记录要累加；value 字段存在并可解析时，记录才算 accepted。

坏行样例如下：

```
1 missing
2 click,1,extra
3 ,2
4 login,abc
```

它们分别对应 **missing_field**、**extra_field**、**empty_event** 和 **bad_value**。测试不只检查错误个数，还检查 line number、error kind 和 byte offset。这样后续换 parser 时，不能把诊断精度降级。

习题与作业

本章实现任务：建立 CMake target、顺序 reference 和测试。测试至少包含 normal five lines、missing field、extra field、empty event name、bad value 五类输入。每个测试同时检查 accepted、diagnostics、counts 和稳定 checksum。

1.4 报告是第一份证据

第一版报告可以是简单 key-value 文本，不需要复杂 UI。最低字段包括输入名、输入版本、总行数、accepted、rejected、输出 checksum 和错误类型计数。报告不是给人截图看的，而是后续脚本和读者复查用的证据。

Listing 1.2 第一版报告示例

```
input_name=normal_5_lines
input_generation=1
records_total=5
accepted=5
rejected=0
```

```
checksum=...  
count.click=1  
count.login=1  
count.logout=1  
count.view=2
```

checksum（校验和）把结果压成可比较摘要。它不能替代完整输出，但能让回归测试快速发现“看起来差不多”的变化。后续并行归约、split merge、manifest 发布都要继续保留 checksum，因为性能优化不能只证明自己更快，还要证明自己没有改变语义。

本章结束时，项目还很小，但它已经具备工程地基：语义明确、测试可跑、报告可查。下一章会把它扩展成两个独立 lab：**reference_pipeline** 固定输入和报告合同，**compute_systems** 为原理卷每章提供可运行模型和系统并发探针。

第零部分

Reference Pipeline 实践

第 2 章

实践卷：两条代码主线怎样落地

上一章先把 reference、contract、diagnostic 和 report 讲成代码边界。本章把边界落成两个可以独立构建、独立测试、互相衔接的 lab：

```
1 books/compute-systems-engine-code/labs/reference_pipeline
2 books/compute-systems-engine-code/labs/compute_systems
```

reference_pipeline 从一份文本输入开始，训练“先定义正确结果，再固定诊断和发布格式”。compute_systems 承接第二册原理卷，把每章的 CPU、内存、并发、运行时、异步 I/O 和分布式概念变成可运行小模型。二者不是两套无关示例：前者负责把输入、结果和验收合同讲清楚；后者负责把系统机制一章一章接到可执行代码上。

2.1 一、两个 lab 如何组成代码卷

代码实践卷的主线不是“贴一堆源码”，而是让每段源码回答一个清楚问题：这个类型承诺什么，这个函数把哪条机制跑起来，这个测试证明什么失败不会漏掉。每个阶段都要留下四类证据。

证据	在工程里的位置	为什么必须保留
接口合同	include/.../*.hpp	类型名、字段名和函数签名先固定语义边界；读者不用猜“输入是谁、输出是什么、失败怎样表达”
可运行实现	src/*.cpp	机制必须跑起来，而不是停在模板；实现要能处理正常输入、边界输入和错误输入
测试与探针	tests/*.cpp 和 *_probe.cpp	测试固定行为，probe 固定章节模型的可观察输出，避免书里结论和代码脱节
报告与清单	CLI 输出、manifest、benchmark 输出	结果要能被人复查，也要能被脚本比较；没有报告，就无法说明一次运行到底通过了什么

先看构建入口。顶层 CMakeLists.txt 只做三件事：要求 C++20，打开测试，把两个 lab 加进来。这样读者可以一次构建整本代码卷，也可以进入单个 lab 看局部目标。常用命令如下：

Listing 2.1 代码实践卷完整构建与测试

```
1 cmake -S books/compute-systems-engine-code \
2       -B books/compute-systems-engine-code/build/reference-debug \
3       -DCMAKE_BUILD_TYPE=Debug
4 cmake --build books/compute-systems-engine-code/build/reference-debug
5 ctest --test-dir books/compute-systems-engine-code/build/reference-debug \
6       --output-on-failure
```

这三条命令是进入正文结论的最低门槛。若本地构建不过，书中不能声称代码可运行；若测试不过，书中不能声称语义已经固定；若只跑 demo 不跑测试，很多边界失败会被漂亮输出盖住。

2.2 二、Reference Pipeline：从输入合同推到 split 和 merge

reference_pipeline 的起点是一行 event,value 文本。先不要急着想性能，先问四个问题：这行来自哪份输入，位于哪个位置，怎样算合法，非法时怎样报告。代码用 RecordId、

`InputCase`、`ParseDiagnostic` 和 `ReferenceResult` 把这四件事变成类型。

Listing 2.2 输入身份和 reference 结果的核心字段

```

1 struct RecordId {
2     std::string input_name;
3     std::uint64_t input_generation = 0;
4     std::uint64_t byte_offset = 0;
5     std::uint64_t line_number = 0;
6 };
7
8 struct ReferenceResult {
9     std::string input_name;
10    std::uint64_t input_generation = 0;
11    std::uint64_t records_total = 0;
12    std::uint64_t accepted = 0;
13    std::vector<ParseDiagnostic> diagnostics;
14    std::map<std::string, std::uint64_t> counts;
15    std::uint64_t checksum = 0;
16 };

```

为什么一开始就要这些字段？拿三行输入手算：

```

1 view,1
2 click,1
3 view,2

```

三行都合法，所以 `records_total=3`，`accepted=3`，`counts["view"]=2`，`counts["click"]=1`。这只是正常路径。真正能暴露设计质量的是坏行：

```

1 missing
2 click,1,extra
3 ,2
4 login,abc

```

第一行缺字段，第二行多字段，第三行事件名为空，第四行 `value` 不是整数。朴素实现可能只返回“解析失败 4 次”，但这样的错误无法重放，也无法定位。实践卷要求每条坏行进入 `ParseDiagnostic`，并保留 `line_number`、`byte_offset`、`error_kind` 和可读说明。这样后续换 parser、加并行 split 或把结果写入 manifest，都不能偷偷降低诊断精度。

接着看 reference。reference 是参考实现，不是“慢版本”的随意别名。它的责任是给出清楚、确定、可复查的正确结果：合法记录才进入 `accepted` 和 `counts`，坏行只进入 `diagnostics`，输出用 `std::map` 保持 key 顺序稳定，`checksum` 用于对照优化实现。选择 `std::map` 不是因为它最快，而是因为报告和回归测试需要稳定顺序。等进入热路径优化，可以把内部统计换成哈希表或分片结构，但发布报告前仍要回到稳定顺序。

最后推进到 split 和 merge。大输入不能永远整块处理，必须切成 `InputSplit`。难点不是“按字节切几段”，而是切点落在行中间时不能丢行、重行或错报行号。例如目标大小刚好落在 `view,2` 中间，前一个 split 不能处理半行，后一个 split 也不能从半行继续误解析。当前工程提供 `split_input_case`、`run_reference_split`、`merge_reference_results` 和 `run_reference_with_splits`，测试用整文件 reference 对照 split 后 merge 的结果，固定记录数、accepted、诊断和 checksum 都一致。

Reference Pipeline 的阶段验收：先运行 `cse_reference_tests`，再运行 `cse_reference_bench--benchmark_min_time=0.01s` 做 benchmark smoke。benchmark 只证明入口可运行；正式性能结论必须用 Release 构建，并记录 CPU、编译器、输入规模、重复次数和结果校验。

2.3 三、Compute Systems：把每章机制变成可运行模型

`compute_systems` 是第二册原理卷的代码化主线。它不把“缓存、TLB、NUMA、SIMD、线程池、futex、epoll、checkpoint”当成词表，而是给每章一个小模型：输入是什么，输出哪些计数字段，哪个探针能跑出结果，哪个测试能挡住错误实现。核心头文件是 `include/compsys/chapter_models.hpp`，实现是 `src/chapter_models.cpp`，章节探针是 `src/chapter_models_probe.cpp`。

章节	模型函数	这章固定的工程事实
1	<code>model_ch01_input_pipeline</code>	输入记录分 accepted/rejected, checksum 证明结果稳定
2	<code>model_ch02_branch_predictor</code>	分支历史、实际 taken 次数和 misprediction 数量分开统计
3	<code>model_ch03_cache_tlb</code>	cache line、page 和 TLB miss 不是同一个指标
4	<code>model_ch04_numa_coherence</code>	local/remote 访问与 coherence transfer 分开解释
5, 5b	<code>model_ch05_roofline</code> 、 <code>model_ch05b_pmu_simd_evidence</code>	roofline 需要 FLOPs、字节量、带宽和 PMU/SIMD 证据，不只看耗时
6, 7, 8	layout、SIMD、map/filter/reduce 模型	数据布局、lane 利用率和算子语义都能手算对照
9, 9b, 10	调度、Linux wait/wake、backpressure 模型	线程负载、虚假唤醒、容量压力要有计数，不靠感觉判断
11, 12, 12b	atomic、SPSC ring、epoch reclamation 模型	发布顺序、队列边界和回收安全必须可观察
13, 14, 15	reduce/scan/partition、work stealing、async I/O 模型	并行算法、偷取调度和 I/O 队列深度进入同一验收链
16, 17, 18	attempts、quorum、checkpoint/recovery 模型	分布式重试、副本提交和恢复范围用字段固定，不能只讲概念

这些模型故意很小，因为它们用于推导规律。以第 3 章为例，输入是一组字节 offset，模型分别计算访问次数、唯一 cache line、唯一 page 和 TLB miss。若两个 offset 落在同一 cache line，它们不应该算两条 cache line；若 page 超过 TLB 容量，就会产生额外 miss。这样读者先能手算，再跑 probe，再回到原理卷理解为什么局部性会影响吞吐。

并发和运行时部分也同样处理。队列不是只展示 `push/pop`，而是记录 `push_wait_count`、`pop_wait_count`、`close_count` 和等待耗时；wait channel 不是只说“等待被唤醒”，而是分别跑 semaphore、barrier、future/promise 和 atomic wait；异步 I/O 模型不把 submit、completion 和 commit 混成一个动作，而是统计 submitted、completed、backpressure 和 max in flight。读这些代码时，先看报告字段，再看测试如何让字段变成承诺。

Listing 2.3 Compute Systems 探针入口

```
1 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/↵
  ↵ cse_compsys_demo
2 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/↵
  ↵ cse_compsys_chapter_models_probe
```



```
3 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/↵
  ↪ cse_compsys_wait_channels_probe
4 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/↵
  ↪ cse_compsys_task_runtime_probe
5 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/↵
  ↪ cse_compsys_futex_lab_probe
```

2.4 四、递进大作业和验收方式

代码卷的大作业分两条线推进。第一条线从 `reference_pipeline` 出发，目标是做出一个可以解释输入身份、坏行诊断、reference 结果、report、manifest、split 和 merge 的数据处理骨架。第二条线从 `compute_systems` 出发，目标是把第二册每章机制做成可运行模型，并用测试和 probe 固定输出。两个项目最后合在一起：系统机制服务一条有合同、有恢复边界、有报告证据的工程流水线。

阶段	交付物	通过标准
0	输入身份与 reference 合同	每条记录有 input name、generation、offset、line；坏行进入结构化 diagnostics
1	report、manifest、checksum	人读报告和机器清单字段顺序稳定；同一输入重复运行 checksum 一致
2	split、partial result、merge	split 落在行中间时，与整文件 reference 的 records、accepted、diagnostics 和 checksum 一致
3	逐章 compute systems 模型	章节 probe 输出 21 个模型结果，字段能回到原理卷逐项解释
4	并发、wait、futex、runtime 探针	队列关闭、等待唤醒、future/promise、atomic wait、futex 和任务运行时都有测试证据
5	端到端工程报告	报告写清构建命令、测试结果、probe 输出、benchmark 边界和未验证假设

提交前按下面顺序验收。第一步构建整本代码卷，第二步跑全部 CTest，第三步单独跑 compute systems 章节 probe，第四步把 PDF/EPUB 重新生成并导出。任何一步失败，都说明书里的代码、解释或构建环境还没有对齐。

Listing 2.4 提交前最小验收命令

```
1 cmake -S books/compute-systems-engine-code \
2     -B books/compute-systems-engine-code/build/reference-debug \
3     -DCMAKE_BUILD_TYPE=Debug
4 cmake --build books/compute-systems-engine-code/build/reference-debug
5 ctest --test-dir books/compute-systems-engine-code/build/reference-debug \
6     --output-on-failure
7 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/↵
  ↪ cse_compsys_chapter_models_probe
```

章节结构约定：正文每章只保留三到四个大节。更细的推导、例子、反例和验收点放进大节内部，用表格、代码和段落承载；不要把每个小动作都拆成独立 section。这样目录不会碎，正文仍然能从具体案例推到规律。

完整源码清单

本附录不是节选代码，而是从 [books/compute-systems-engine-code/labs](#) 直接读入的完整工程源码。读者在 PDF 或 EPUB 中看到的清单，应当和本地可构建工程保持同源；如果源码修改，重建书籍后清单会随之更新。

源码阅读顺序不要从最长的实现文件开始。先读 CMake 和 README，确认工程边界；再读公共头文件，确认类型和函数合同；然后读实现和 probe，观察机制如何跑起来；最后读测试和 benchmark，检查哪些行为已经被固定，哪些只是性能 smoke。

一、构建入口与 Reference Pipeline

这一组文件给出代码实践卷的顶层构建入口，以及第一条主线 [reference_pipeline](#) 的完整源码。它从输入身份、解析诊断、reference 结果、report、manifest、split 和 merge 开始，训练“先定义正确结果，再谈优化”的工程习惯。

Listing 2.5 代码实践卷顶层 CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.31)
2 project(ComputeSystemsEngineCode LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 option(COMPUTE_SYSTEMS_ENGINE_CODE_BUILD_LABS "Build Compute Systems Engine ↵
    ↵ code labs" ON)
10
11 if(COMPUTE_SYSTEMS_ENGINE_CODE_BUILD_LABS)
12     enable_testing()
13     add_subdirectory(labs/reference_pipeline)
14     add_subdirectory(labs/compute_systems)
15 endif()
```

Listing 2.6 reference pipeline README.md

```
1 # Reference Pipeline
2
3 This lab is the first executable slice of the Compute Systems Engine code ↵
    ↵ practice volume.
4
5 It fixes the smallest contract that later optimized versions must preserve:
6
7 - Every input line has a stable `RecordId`.
8 - Accepted records update deterministic event counts.
9 - Rejected records produce structured diagnostics.
10 - Reports and manifest rows are stable enough for regression tests.
11
12 Build and test:
```

```

13
14 ```bash
15 cmake -S books/compute-systems-engine-code -B books/compute-systems-engine-code↵
    ↪ /build/reference-debug -DCMAKE_BUILD_TYPE=Debug
16 cmake --build books/compute-systems-engine-code/build/reference-debug
17 ctest --test-dir books/compute-systems-engine-code/build/reference-debug --↵
    ↪ output-on-failure
18 ```
19
20 Run the demo:
21
22 ```bash
23 books/compute-systems-engine-code/build/reference-debug/labs/reference_pipeline↵
    ↪ /cse_reference_demo
24 ```

```

Listing 2.7 reference pipeline CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.31)
2 project(CseReferencePipeline LANGUAGES CXX)
3
4 list(APPEND CMAKE_MODULE_PATH "${CMAKE_CURRENT_LIST_DIR}/../../../../../cmake")
5 include(BookLabDependencies)
6 book_labs_enable_gtest()
7 book_labs_enable_benchmark()
8
9 add_library(cse_reference_pipeline STATIC
10     src/reference_pipeline.cpp
11 )
12
13 target_include_directories(cse_reference_pipeline
14     PUBLIC
15     ${CMAKE_CURRENT_SOURCE_DIR}/include
16 )
17
18 target_compile_features(cse_reference_pipeline PUBLIC cxx_std_20)
19
20 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
21     target_compile_options(cse_reference_pipeline PRIVATE -Wall -Wextra ↵
        ↪ -Wpedantic)
22 endif()
23
24 add_executable(cse_reference_demo
25     src/main.cpp
26 )
27
28 target_link_libraries(cse_reference_demo PRIVATE cse_reference_pipeline)
29
30 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
31     target_compile_options(cse_reference_demo PRIVATE -Wall -Wextra -Wpedantic)
32 endif()
33

```

```

34 add_executable(cse_reference_tests
35     tests/reference_pipeline_tests.cpp
36 )
37
38 target_link_libraries(cse_reference_tests
39     PRIVATE
40         cse_reference_pipeline
41         GTest::gmock_main
42 )
43
44 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
45     target_compile_options(cse_reference_tests PRIVATE -Wall -Wextra -Wpedantic↵
46         ↵ )
47 endif()
48
49 add_test(NAME cse_reference_tests COMMAND ${TARGET_FILE:cse_reference_tests})
50
51 add_executable(cse_reference_bench
52     bench/reference_pipeline_bench.cpp
53 )
54
55 target_link_libraries(cse_reference_bench
56     PRIVATE
57         cse_reference_pipeline
58         benchmark::benchmark
59 )
60
61 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
62     target_compile_options(cse_reference_bench PRIVATE -Wall -Wextra -Wpedantic↵
63         ↵ )
64 endif()

```

Listing 2.8 include/cse/reference_pipeline.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <cstdint>
5 #include <map>
6 #include <string>
7 #include <string_view>
8 #include <vector>
9
10 namespace cse {
11
12 struct RecordId {
13     std::string input_name;
14     std::uint64_t input_generation = 0;
15     std::uint64_t byte_offset = 0;
16     std::uint64_t line_number = 0;
17 };
18

```

```

19 struct ParsedRecord {
20     RecordId record;
21     std::string event_name;
22     std::int64_t value = 0;
23 };
24
25 struct ParseDiagnostic {
26     RecordId record;
27     std::string error_kind;
28     std::string detail;
29 };
30
31 struct InputCase {
32     std::string input_name;
33     std::uint64_t input_generation = 0;
34     std::string text;
35 };
36
37 struct InputSplit {
38     std::string input_name;
39     std::uint64_t input_generation = 0;
40     std::uint64_t byte_begin = 0;
41     std::uint64_t first_line_number = 1;
42     std::string text;
43 };
44
45 struct ReferenceResult {
46     std::string input_name;
47     std::uint64_t input_generation = 0;
48     std::uint64_t records_total = 0;
49     std::uint64_t accepted = 0;
50     std::vector<ParseDiagnostic> diagnostics;
51     std::map<std::string, std::uint64_t> counts;
52     std::uint64_t checksum = 0;
53 };
54
55 struct ManifestRow {
56     std::string input_name;
57     std::uint64_t input_generation = 0;
58     std::uint64_t records_total = 0;
59     std::uint64_t accepted = 0;
60     std::uint64_t rejected = 0;
61     std::uint64_t checksum = 0;
62 };
63
64 [[nodiscard]] ReferenceResult run_reference(const InputCase& input);
65 [[nodiscard]] std::vector<InputSplit> split_input_case(const InputCase& input,
66                                                         std::size_t target_bytes↵
        ↵ );
67 [[nodiscard]] ReferenceResult run_reference_split(const InputSplit& split);
68 [[nodiscard]] ReferenceResult merge_reference_results(const std::vector<↵
        ↵ ReferenceResult>& partials);

```

```

69 [[nodiscard]] ReferenceResult run_reference_with_splits(const InputCase& input,
70                                                         std::size_t ←
    ↪ target_bytes);
71 [[nodiscard]] ManifestRow make_manifest_row(const ReferenceResult& result);
72 [[nodiscard]] std::string format_report(const ReferenceResult& result);
73 [[nodiscard]] std::string format_manifest_row(const ManifestRow& row);
74
75 } // namespace cse

```

Listing 2.9 src/reference_pipeline.cpp

```

1  #include <cse/reference_pipeline.hpp>
2
3  #include <charconv>
4  #include <optional>
5  #include <sstream>
6  #include <stdexcept>
7  #include <utility>
8
9  namespace cse {
10 namespace {
11
12 constexpr std::uint64_t CSE_FIRST_LINE_NUMBER = 1;
13 constexpr std::uint64_t CSE_FNV_OFFSET_BASIS = 14695981039346656037ULL;
14 constexpr std::uint64_t CSE_FNV_PRIME = 1099511628211ULL;
15 constexpr std::size_t CSE_MIN_SPLIT_SIZE = 1;
16 constexpr std::string_view CSE_ERROR_MISSING_FIELD = "missing_field";
17 constexpr std::string_view CSE_ERROR_EXTRA_FIELD = "extra_field";
18 constexpr std::string_view CSE_ERROR_EMPTY_EVENT = "empty_event";
19 constexpr std::string_view CSE_ERROR_BAD_VALUE = "bad_value";
20
21 [[nodiscard]] std::string trim_copy(std::string_view text)
22 {
23     std::size_t begin = 0;
24     while (begin < text.size() &&
25           (text[begin] == ' ' || text[begin] == '\t' || text[begin] == '\r')) ←
    ↪ {
26         ++begin;
27     }
28
29     std::size_t end = text.size();
30     while (end > begin &&
31           (text[end - 1] == ' ' || text[end - 1] == '\t' || text[end - 1] == '\r')) ←
    ↪ {
32         --end;
33     }
34
35     return std::string(text.substr(begin, end - begin));
36 }
37
38 [[nodiscard]] std::vector<std::string_view> split_csv_line(std::string_view ←
    ↪ line)

```

```

39 {
40     std::vector<std::string_view> fields;
41     std::size_t begin = 0;
42     while (begin <= line.size()) {
43         const std::size_t comma = line.find(',', begin);
44         const std::size_t end = comma == std::string_view::npos ? line.size() : ←
↪ comma;
45         fields.push_back(line.substr(begin, end - begin));
46         if (comma == std::string_view::npos) {
47             break;
48         }
49         begin = comma + 1;
50     }
51     return fields;
52 }
53
54 [[nodiscard]] std::optional<std::int64_t> parse_int64(std::string_view text)
55 {
56     std::int64_t value = 0;
57     const char* first = text.data();
58     const char* last = text.data() + text.size();
59     const auto [ptr, error] = std::from_chars(first, last, value);
60     if (error != std::errc{} || ptr != last) {
61         return std::nullopt;
62     }
63     return value;
64 }
65
66 void append_diagnostic(ReferenceResult& result,
67                        const RecordId& record,
68                        std::string_view error_kind,
69                        std::string detail)
70 {
71     result.diagnostics.push_back(ParseDiagnostic{
72         record,
73         std::string(error_kind),
74         std::move(detail),
75     });
76 }
77
78 void update_checksum_byte(std::uint64_t& checksum, unsigned char byte)
79 {
80     checksum ^= byte;
81     checksum *= CSE_FNV_PRIME;
82 }
83
84 void update_checksum_text(std::uint64_t& checksum, std::string_view text)
85 {
86     for (const char ch : text) {
87         update_checksum_byte(checksum, static_cast<unsigned char>(ch));
88     }
89     update_checksum_byte(checksum, 0);

```

```

90 }
91
92 void update_checksum_u64(std::uint64_t& checksum, std::uint64_t value)
93 {
94     constexpr std::int32_t CSE_U64_BYTE_COUNT = 8;
95     constexpr std::int32_t CSE_BITS_PER_BYTE = 8;
96     for (std::int32_t index = 0; index < CSE_U64_BYTE_COUNT; ++index) {
97         const auto byte = static_cast<unsigned char>(
98             (value >> (index * CSE_BITS_PER_BYTE)) & 0xffU);
99         update_checksum_byte(checksum, byte);
100     }
101 }
102
103 [[nodiscard]] std::uint64_t compute_checksum(const ReferenceResult& result)
104 {
105     std::uint64_t checksum = CSE_FNV_OFFSET_BASIS;
106     update_checksum_text(checksum, result.input_name);
107     update_checksum_u64(checksum, result.input_generation);
108     update_checksum_u64(checksum, result.records_total);
109     update_checksum_u64(checksum, result.accepted);
110
111     for (const auto& [event_name, count] : result.counts) {
112         update_checksum_text(checksum, event_name);
113         update_checksum_u64(checksum, count);
114     }
115
116     for (const ParseDiagnostic& diagnostic : result.diagnostics) {
117         update_checksum_u64(checksum, diagnostic.record.line_number);
118         update_checksum_u64(checksum, diagnostic.record.byte_offset);
119         update_checksum_text(checksum, diagnostic.error_kind);
120     }
121
122     return checksum;
123 }
124
125 void process_line(std::string_view line,
126                  const RecordId& record,
127                  ReferenceResult& result)
128 {
129     ++result.records_total;
130     const std::vector<std::string_view> fields = split_csv_line(line);
131     if (fields.size() < 2) {
132         append_diagnostic(result, record, CSE_ERROR_MISSING_FIELD, "expected ↵
↵ event,value");
133         return;
134     }
135     if (fields.size() > 2) {
136         append_diagnostic(result, record, CSE_ERROR_EXTRA_FIELD, "expected ↵
↵ exactly two fields");
137         return;
138     }
139

```



```

140     const std::string event_name = trim_copy(fields[0]);
141     if (event_name.empty()) {
142         append_diagnostic(result, record, CSE_ERROR_EMPTY_EVENT, "event name is ↵
↵ empty");
143         return;
144     }
145
146     const std::string value_text = trim_copy(fields[1]);
147     const std::optional<std::int64_t> value = parse_int64(value_text);
148     if (!value.has_value()) {
149         append_diagnostic(result, record, CSE_ERROR_BAD_VALUE, "value is not a ↵
↵ full integer");
150         return;
151     }
152
153     ++result.accepted;
154     ++result.counts[event_name];
155 }
156
157 [[nodiscard]] ReferenceResult make_empty_result(std::string input_name,
158                                                 std::uint64_t input_generation)
159 {
160     ReferenceResult result;
161     result.input_name = std::move(input_name);
162     result.input_generation = input_generation;
163     result.checksum = compute_checksum(result);
164     return result;
165 }
166
167 void process_text_lines(std::string_view text,
168                        std::string_view input_name,
169                        std::uint64_t input_generation,
170                        std::uint64_t byte_begin,
171                        std::uint64_t first_line_number,
172                        ReferenceResult& result)
173 {
174     std::size_t begin = 0;
175     std::uint64_t line_number = first_line_number;
176     while (begin < text.size()) {
177         const std::size_t newline = text.find('\n', begin);
178         const std::size_t end = newline == std::string_view::npos ? text.size() ↵
↵ : newline;
179         const RecordId record{
180             std::string(input_name),
181             input_generation,
182             byte_begin + static_cast<std::uint64_t>(begin),
183             line_number,
184         };
185         process_line(text.substr(begin, end - begin), record, result);
186         if (newline == std::string_view::npos) {
187             break;
188         }

```

```

189         begin = newline + 1;
190         ++line_number;
191     }
192 }
193
194 } // namespace
195
196 ReferenceResult run_reference(const InputCase& input)
197 {
198     ReferenceResult result;
199     result.input_name = input.input_name;
200     result.input_generation = input.input_generation;
201     process_text_lines(input.text,
202                        input.input_name,
203                        input.input_generation,
204                        0,
205                        CSE_FIRST_LINE_NUMBER,
206                        result);
207
208     result.checksum = compute_checksum(result);
209     return result;
210 }
211
212 std::vector<InputSplit> split_input_case(const InputCase& input, std::size_t ←
    ↪ target_bytes)
213 {
214     if (target_bytes < CSE_MIN_SPLIT_SIZE) {
215         throw std::invalid_argument("target_bytes must be positive");
216     }
217
218     std::vector<InputSplit> splits;
219     if (input.text.empty()) {
220         return splits;
221     }
222
223     std::size_t split_begin = 0;
224     std::size_t split_end = 0;
225     std::uint64_t split_first_line = CSE_FIRST_LINE_NUMBER;
226     std::uint64_t line_number = CSE_FIRST_LINE_NUMBER;
227     std::size_t line_begin = 0;
228
229     while (line_begin < input.text.size()) {
230         const std::size_t newline = input.text.find('\n', line_begin);
231         const std::size_t line_end =
232             newline == std::string::npos ? input.text.size() : newline + 1;
233         const bool split_has_line = split_end > split_begin;
234         const bool would_exceed_target =
235             split_has_line && (line_end - split_begin > target_bytes);
236
237         if (would_exceed_target) {
238             splits.push_back(InputSplit{
239                 input.input_name,

```

```

240         input.input_generation,
241         static_cast<std::uint64_t>(split_begin),
242         split_first_line,
243         input.text.substr(split_begin, split_end - split_begin),
244     });
245     split_begin = line_begin;
246     split_end = line_begin;
247     split_first_line = line_number;
248 }
249
250     split_end = line_end;
251     line_begin = line_end;
252     ++line_number;
253 }
254
255     if (split_end > split_begin) {
256         splits.push_back(InputSplit{
257             input.input_name,
258             input.input_generation,
259             static_cast<std::uint64_t>(split_begin),
260             split_first_line,
261             input.text.substr(split_begin, split_end - split_begin),
262         });
263     }
264
265     return splits;
266 }
267
268 ReferenceResult run_reference_split(const InputSplit& split)
269 {
270     ReferenceResult result;
271     result.input_name = split.input_name;
272     result.input_generation = split.input_generation;
273     process_text_lines(split.text,
274         split.input_name,
275         split.input_generation,
276         split.byte_begin,
277         split.first_line_number,
278         result);
279     result.checksum = compute_checksum(result);
280     return result;
281 }
282
283 ReferenceResult merge_reference_results(const std::vector<ReferenceResult>& ↵
    ↵ partials)
284 {
285     if (partials.empty()) {
286         return make_empty_result("", 0);
287     }
288
289     ReferenceResult merged;
290     merged.input_name = partials.front().input_name;

```

```

291 merged.input_generation = partials.front().input_generation;
292
293 for (const ReferenceResult& partial : partials) {
294     if (partial.input_name != merged.input_name ||
295         partial.input_generation != merged.input_generation) {
296         throw std::invalid_argument("cannot merge results from different
↪ inputs");
297     }
298     merged.records_total += partial.records_total;
299     merged.accepted += partial.accepted;
300     for (const auto& [event_name, count] : partial.counts) {
301         merged.counts[event_name] += count;
302     }
303     merged.diagnostics.insert(
304         merged.diagnostics.end(), partial.diagnostics.begin(), partial.
↪ diagnostics.end());
305 }
306
307 merged.checksum = compute_checksum(merged);
308 return merged;
309 }
310
311 ReferenceResult run_reference_with_splits(const InputCase& input, std::size_t
↪ target_bytes)
312 {
313     const std::vector<InputSplit> splits = split_input_case(input, target_bytes
↪ );
314     if (splits.empty()) {
315         return make_empty_result(input.input_name, input.input_generation);
316     }
317
318     std::vector<ReferenceResult> partials;
319     partials.reserve(splits.size());
320     for (const InputSplit& split : splits) {
321         partials.push_back(run_reference_split(split));
322     }
323     return merge_reference_results(partial);
324 }
325
326 ManifestRow make_manifest_row(const ReferenceResult& result)
327 {
328     return ManifestRow{
329         result.input_name,
330         result.input_generation,
331         result.records_total,
332         result.accepted,
333         static_cast<std::uint64_t>(result.diagnostics.size()),
334         result.checksum,
335     };
336 }
337
338 std::string format_report(const ReferenceResult& result)

```

```

339 {
340     std::map<std::string, std::uint64_t> error_counts;
341     for (const ParseDiagnostic& diagnostic : result.diagnostics) {
342         ++error_counts[diagnostic.error_kind];
343     }
344
345     std::ostringstream out;
346     out << "input_name=" << result.input_name << '\n';
347     out << "input_generation=" << result.input_generation << '\n';
348     out << "records_total=" << result.records_total << '\n';
349     out << "accepted=" << result.accepted << '\n';
350     out << "rejected=" << result.diagnostics.size() << '\n';
351     out << "checksum=" << result.checksum << '\n';
352
353     for (const auto& [event_name, count] : result.counts) {
354         out << "count." << event_name << '=' << count << '\n';
355     }
356     for (const auto& [error_kind, count] : error_counts) {
357         out << "error." << error_kind << '=' << count << '\n';
358     }
359     return out.str();
360 }
361
362 std::string format_manifest_row(const ManifestRow& row)
363 {
364     std::ostringstream out;
365     out << row.input_name << ','
366         << row.input_generation << ','
367         << row.records_total << ','
368         << row.accepted << ','
369         << row.rejected << ','
370         << row.checksum;
371     return out.str();
372 }
373
374 } // namespace cse

```

Listing 2.10 reference pipeline src/main.cpp

```

1 #include <cse/reference_pipeline.hpp>
2
3 #include <iostream>
4
5 int main()
6 {
7     const cse::InputCase input{
8         "normal_5_lines",
9         1,
10        "view,1\nclick,1\nview,2\nlogin,1\nlogout,1\n",
11    };
12
13    const cse::ReferenceResult result = cse::run_reference(input);

```

```

14     std::cout << cse::format_report(result);
15     std::cout << "manifest=" << cse::format_manifest_row(cse::make_manifest_row(
    ↪ (result)) << '\n';
16     return 0;
17 }

```

二、Compute Systems 合同层

这一组文件给出第二条主线 `compute_systems` 的构建入口、阅读地图和公共头文件。公共头文件是合同层：每个 `struct` 都把一个章节机制压成可观察字段，每个函数签名都说明输入规模、边界和返回结果。

Listing 2.11 compute systems README.md

```

1 # Compute Systems Models And Runtime Labs
2
3 This lab is the standalone code-practice version of the CPU volume 2 compute
4 systems experiments. It gives each chapter a runnable model, then adds the
5 concurrency, wait-channel, task-runtime, and futex probes that later stages use
    ↪ .
6
7 ## What This Lab Covers
8
9 - Per-chapter runnable models in `chapter_models.hpp/.cpp`.
10 - Sequential and parallel reduction.
11 - Mutex and atomic counters.
12 - Closable bounded MPMC queue with wait and close reports.
13 - Semaphore, barrier, future/promise, and atomic wait contracts.
14 - `eventfd`/`epoll` readiness and drain contracts.
15 - Futex wait/wake probes.
16 - Minimal task runtime, same-pool future wait probe, continuation probe, and
    ↪ CSV report.
17
18 ## Per-Chapter Model Map
19
20 | Chapter | Function | What the model fixes |
21 | --- | --- | --- |
22 | 1 | `model_ch01_input_pipeline` | Accepted/rejected records and checksum |
23 | 2 | `model_ch02_branch_predictor` | Branch history and misprediction count |
24 | 3 | `model_ch03_cache_tlb` | Cache line, page, and TLB miss accounting |
25 | 4 | `model_ch04_numa_coherence` | Local/remote access and coherence transfers
    ↪ |
26 | 5 | `model_ch05_roofline` | FLOPs, bytes, bandwidth, and attainable GFLOPS |
27 | 5b | `model_ch05b_pmu_simd_evidence` | PMU/SIMD evidence completeness |
28 | 6 | `model_ch06_layout_locality` | AoS/SoA bytes touched |
29 | 7 | `model_ch07_simd_ilp` | Vector iterations, scalar tail, and lane
    ↪ utilization |
30 | 8 | `model_ch08_map_filter_reduce` | Filtered records and reduced sum |
31 | 9 | `model_ch09_thread_scheduling` | Worker load and imbalance |
32 | 9b | `model_ch09b_linux_wait_wake` | Spurious wakeups versus real completion
    ↪ |
33 | 10 | `model_ch10_sync_backpressure` | Capacity, production, consumption, and
    ↪

```

```

    ↪ backpressure |
34 | 11 | `model_ch11_atomic_publication` | Stale reads and monotonic version ↪
    ↪ observations |
35 | 12 | `model_ch12_spsc_ring` | Push/pop/full/empty ring-buffer states |
36 | 12b | `model_ch12b_epoch_reclamation` | Reclaimed versus deferred retired ↪
    ↪ nodes |
37 | 13 | `model_ch13_reduce_scan_partition` | Reduce, scan, and partition outputs ↪
    ↪ |
38 | 14 | `model_ch14_work_stealing` | Stolen and completed tasks |
39 | 15 | `model_ch15_async_io_backpressure` | Submitted/completed I/O and in- ↪
    ↪ flight pressure |
40 | 16 | `model_ch16_distributed_attempts` | Logical tasks, duplicates, and stale ↪
    ↪ generations |
41 | 17 | `model_ch17_quorum_commit` | Quorum size and commit decision |
42 | 18 | `model_ch18_checkpoint_recovery` | Checkpoint count and replay range |
43
44 ## Build And Run
45
46 ```bash
47 cmake -S books/compute-systems-engine-code -B books/compute-systems-engine-code ↪
    ↪ /build/reference-debug -DCMAKE_BUILD_TYPE=Debug
48 cmake --build books/compute-systems-engine-code/build/reference-debug
49 ctest --test-dir books/compute-systems-engine-code/build/reference-debug -- ↪
    ↪ output-on-failure
50 ```
51
52 Useful executables:
53
54 ```bash
55 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/ ↪
    ↪ cse_compsys_demo
56 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/ ↪
    ↪ cse_compsys_chapter_models_probe
57 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/ ↪
    ↪ cse_compsys_wait_channels_probe
58 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/ ↪
    ↪ cse_compsys_task_runtime_probe
59 books/compute-systems-engine-code/build/reference-debug/labs/compute_systems/ ↪
    ↪ cse_compsys_futex_lab_probe
60 ```

```

Listing 2.12 compute systems CMakeLists.txt

```

1 add_library(cse_compsys STATIC
2     src/chapter_models.cpp
3     src/parallel_reduce.cpp
4     src/futex_lab.cpp
5     src/sync_primitives.cpp
6     src/task_runtime.cpp
7     src/wait_channels.cpp
8 )
9

```

```
10 target_include_directories(cse_compsys
11     PUBLIC
12     ${CMAKE_CURRENT_SOURCE_DIR}/include
13 )
14
15 target_compile_features(cse_compsys PUBLIC cxx_std_20)
16
17 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
18     target_compile_options(cse_compsys PRIVATE -Wall -Wextra -Wpedantic)
19 endif()
20
21 function(cse_add_compsys_executable target_name source_file)
22     add_executable(${target_name} ${source_file})
23     target_link_libraries(${target_name} PRIVATE cse_compsys)
24     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
25         target_compile_options(${target_name} PRIVATE -Wall -Wextra -Wpedantic)
26     endif()
27 endfunction()
28
29 function(cse_add_compsys_probe test_name source_file pass_regex)
30     cse_add_compsys_executable(${test_name} ${source_file})
31     add_test(NAME ${test_name} COMMAND ${<TARGET_FILE:${test_name}>})
32     set_tests_properties(${test_name} PROPERTIES
33         PASS_REGULAR_EXPRESSION "${pass_regex}"
34     )
35 endfunction()
36
37 cse_add_compsys_executable(cse_compsys_demo src/main.cpp)
38 cse_add_compsys_executable(cse_compsys_tests tests/compsys_tests.cpp)
39 add_test(NAME cse_compsys_tests COMMAND ${<TARGET_FILE:cse_compsys_tests>})
40
41 cse_add_compsys_probe(
42     cse_compsys_chapter_models_probe
43     src/chapter_models_probe.cpp
44     "chapter_models,21"
45 )
46 cse_add_compsys_probe(
47     cse_compsys_wait_channels_probe
48     src/wait_channels_probe.cpp
49     "contract,2,1,1,2,2,3,3,1,1"
50 )
51 cse_add_compsys_probe(
52     cse_compsys_task_runtime_probe
53     src/task_runtime_probe.cpp
54     "task_runtime,3,3,3,0,1,5,5,15,"
55 )
56 cse_add_compsys_probe(
57     cse_compsys_futex_lab_probe
58     src/futex_lab_probe.cpp
59     "futex_mutex,"
60 )
```


Listing 2.13 include/compsys/chapter_models.hpp

```

1  #pragma once
2
3  #include <cstdint>
4  #include <span>
5
6  namespace compsys {
7
8  struct Chapter01PipelineReport {
9      std::int32_t records_total = 0;
10     std::int32_t accepted = 0;
11     std::int32_t rejected = 0;
12     std::int64_t checksum = 0;
13 };
14
15 [[nodiscard]] Chapter01PipelineReport model_ch01_input_pipeline(
16     std::span<const std::int32_t> records);
17
18 struct Chapter02BranchReport {
19     std::int32_t iterations = 0;
20     std::int32_t predicted_taken = 0;
21     std::int32_t actual_taken = 0;
22     std::int32_t mispredictions = 0;
23 };
24
25 [[nodiscard]] Chapter02BranchReport model_ch02_branch_predictor(
26     std::span<const std::int32_t> values,
27     std::int32_t taken_threshold);
28
29 struct Chapter03MemoryAccessReport {
30     std::int32_t access_count = 0;
31     std::int32_t unique_cache_lines = 0;
32     std::int32_t unique_pages = 0;
33     std::int32_t tlb_misses = 0;
34 };
35
36 [[nodiscard]] Chapter03MemoryAccessReport model_ch03_cache_tlb(
37     std::span<const std::uint64_t> byte_offsets,
38     std::int32_t cache_line_bytes,
39     std::int32_t page_bytes,
40     std::int32_t tlb_entries);
41
42 struct Chapter04NumaReport {
43     std::int32_t access_count = 0;
44     std::int32_t local_accesses = 0;
45     std::int32_t remote_accesses = 0;
46     std::int32_t coherence_transfers = 0;
47 };
48
49 [[nodiscard]] Chapter04NumaReport model_ch04_numa_coherence(
50     std::span<const std::int32_t> worker_nodes,
51     std::span<const std::int32_t> memory_nodes);

```

```

52
53 struct Chapter05RooflineReport {
54     double arithmetic_intensity = 0.0;
55     double compute_bound_gflops = 0.0;
56     double memory_bound_gflops = 0.0;
57     double attainable_gflops = 0.0;
58     bool memory_bound = false;
59 };
60
61 [[nodiscard]] Chapter05RooflineReport model_ch05_roofline(
62     double flops,
63     double bytes_read_written,
64     double peak_gflops,
65     double bandwidth_gbytes_per_second);
66
67 struct Chapter05bPmuSimdReport {
68     std::int64_t scalar_instructions = 0;
69     std::int64_t vector_instructions = 0;
70     std::int64_t retired_instructions = 0;
71     double vector_fraction = 0.0;
72     bool counter_available = false;
73     bool evidence_complete = false;
74 };
75
76 [[nodiscard]] Chapter05bPmuSimdReport model_ch05b_pmu_simd_evidence(
77     std::int64_t scalar_instructions,
78     std::int64_t vector_instructions,
79     bool counter_available);
80
81 struct Chapter06LayoutReport {
82     std::int64_t records = 0;
83     std::int64_t aos_bytes_touched = 0;
84     std::int64_t soa_bytes_touched = 0;
85     std::int64_t saved_bytes = 0;
86 };
87
88 [[nodiscard]] Chapter06LayoutReport model_ch06_layout_locality(
89     std::int64_t records,
90     std::int32_t field_count,
91     std::int32_t hot_field_count,
92     std::int32_t field_bytes);
93
94 struct Chapter07SimdReport {
95     std::int32_t elements = 0;
96     std::int32_t lane_width = 0;
97     std::int32_t vector_iterations = 0;
98     std::int32_t scalar_tail = 0;
99     double lane_utilization = 0.0;
100 };
101
102 [[nodiscard]] Chapter07SimdReport model_ch07_simd_ilp(
103     std::int32_t elements,

```

```

104     std::int32_t lane_width);
105
106 struct Chapter08KernelReport {
107     std::int32_t input_count = 0;
108     std::int32_t filtered_count = 0;
109     std::int64_t mapped_sum = 0;
110     std::int64_t reduced_sum = 0;
111 };
112
113 [[nodiscard]] Chapter08KernelReport model_ch08_map_filter_reduce(
114     std::span<const std::int32_t> values);
115
116 struct Chapter09SchedulingReport {
117     std::int32_t worker_count = 0;
118     std::int32_t task_count = 0;
119     std::int64_t min_load = 0;
120     std::int64_t max_load = 0;
121     std::int64_t imbalance = 0;
122 };
123
124 [[nodiscard]] Chapter09SchedulingReport model_ch09_thread_scheduling(
125     std::span<const std::int32_t> task_costs,
126     std::int32_t worker_count);
127
128 struct Chapter09bLinuxWaitReport {
129     std::int32_t poll_count = 0;
130     std::int32_t wakeups = 0;
131     std::int32_t spurious_wakeups = 0;
132     bool completed = false;
133 };
134
135 [[nodiscard]] Chapter09bLinuxWaitReport model_ch09b_linux_wait_wake(
136     std::span<const bool> ready_observations);
137
138 struct Chapter10BackpressureReport {
139     std::int32_t ticks = 0;
140     std::int32_t capacity = 0;
141     std::int32_t produced = 0;
142     std::int32_t consumed = 0;
143     std::int32_t backpressure_events = 0;
144     std::int32_t max_depth = 0;
145 };
146
147 [[nodiscard]] Chapter10BackpressureReport model_ch10_sync_backpressure(
148     std::int32_t ticks,
149     std::int32_t produce_per_tick,
150     std::int32_t consume_per_tick,
151     std::int32_t capacity);
152
153 struct Chapter11AtomicPublicationReport {
154     std::int32_t published_versions = 0;
155     std::int32_t observed_versions = 0;

```

```

156     std::int32_t stale_reads = 0;
157     std::int32_t final_version = 0;
158     bool monotonic_observations = true;
159 };
160
161 [[nodiscard]] Chapter11AtomicPublicationReport model_ch11_atomic_publication(
162     std::span<const std::int32_t> observed_versions,
163     std::int32_t final_version);
164
165 struct Chapter12LockFreeQueueReport {
166     std::int32_t capacity = 0;
167     std::int32_t pushed = 0;
168     std::int32_t popped = 0;
169     std::int32_t rejected_full = 0;
170     std::int32_t empty_pops = 0;
171 };
172
173 [[nodiscard]] Chapter12LockFreeQueueReport model_ch12_spsc_ring(
174     std::span<const std::int32_t> operations,
175     std::int32_t capacity);
176
177 struct Chapter12bReclamationReport {
178     std::int32_t retired_nodes = 0;
179     std::int32_t protected_nodes = 0;
180     std::int32_t reclaimed_nodes = 0;
181     std::int32_t deferred_nodes = 0;
182 };
183
184 [[nodiscard]] Chapter12bReclamationReport model_ch12b_epoch_reclamation(
185     std::span<const std::int32_t> retired_node_ids,
186     std::span<const std::int32_t> protected_node_ids);
187
188 struct Chapter13ReduceScanPartitionReport {
189     std::int32_t input_count = 0;
190     std::int64_t reduce_sum = 0;
191     std::int64_t last_prefix = 0;
192     std::int32_t partition_true_count = 0;
193     std::int32_t partition_false_count = 0;
194 };
195
196 [[nodiscard]] Chapter13ReduceScanPartitionReport ↵
197     ↵ model_ch13_reduce_scan_partition(
198     std::span<const std::int32_t> values,
199     std::int32_t partition_threshold);
200
201 struct Chapter14WorkStealingReport {
202     std::int32_t worker_count = 0;
203     std::int32_t initial_tasks = 0;
204     std::int32_t stolen_tasks = 0;
205     std::int32_t completed_tasks = 0;
206     std::int32_t max_queue_depth = 0;
207 };

```

```

207
208 [[nodiscard]] Chapter14WorkStealingReport model_ch14_work_stealing(
209     std::span<const std::int32_t> initial_worker_queues);
210
211 struct Chapter15AsyncIoReport {
212     std::int32_t requested = 0;
213     std::int32_t submitted = 0;
214     std::int32_t completed = 0;
215     std::int32_t backpressure_events = 0;
216     std::int32_t max_in_flight = 0;
217 };
218
219 [[nodiscard]] Chapter15AsyncIoReport model_ch15_async_io_backpressure(
220     std::int32_t request_count,
221     std::int32_t queue_depth,
222     std::int32_t completions_per_tick);
223
224 struct Chapter16DistributedModelReport {
225     std::int32_t logical_tasks = 0;
226     std::int32_t attempts = 0;
227     std::int32_t duplicate_attempts = 0;
228     std::int32_t stale_generation_attempts = 0;
229 };
230
231 [[nodiscard]] Chapter16DistributedModelReport model_ch16_distributed_attempts(
232     std::span<const std::int32_t> task_ids,
233     std::span<const std::int32_t> generations,
234     std::int32_t current_generation);
235
236 struct Chapter17ReplicationReport {
237     std::int32_t replica_count = 0;
238     std::int32_t ack_count = 0;
239     std::int32_t quorum_size = 0;
240     bool committed = false;
241 };
242
243 [[nodiscard]] Chapter17ReplicationReport model_ch17_quorum_commit(
244     std::int32_t replica_count,
245     std::span<const bool> acknowledgements);
246
247 struct Chapter18RecoveryReport {
248     std::int32_t input_records = 0;
249     std::int32_t checkpoint_interval = 0;
250     std::int32_t checkpoints = 0;
251     std::int32_t replay_from_record = 0;
252     std::int32_t replay_records = 0;
253 };
254
255 [[nodiscard]] Chapter18RecoveryReport model_ch18_checkpoint_recovery(
256     std::int32_t input_records,
257     std::int32_t checkpoint_interval,
258     std::int32_t failed_after_record);

```

```

259
260 } // namespace compsys

```

Listing 2.14 include/compsys/parallel_reduce.hpp

```

1 #pragma once
2
3 #include <atomic>
4 #include <condition_variable>
5 #include <cstdint>
6 #include <mutex>
7 #include <optional>
8 #include <span>
9 #include <vector>
10
11 namespace compsys {
12
13 std::int64_t sequential_sum(std::span<const std::int32_t> values);
14
15 std::int64_t parallel_sum(std::span<const std::int32_t> values,
16                          std::int32_t worker_count);
17
18 class MutexCounter {
19 public:
20     void add(std::int64_t delta);
21     [[nodiscard]] std::int64_t value() const;
22
23 private:
24     mutable std::mutex mutex_;
25     std::int64_t value_ = 0;
26 };
27
28 class AtomicCounter {
29 public:
30     void add(std::int64_t delta);
31     [[nodiscard]] std::int64_t value() const;
32
33 private:
34     std::atomic<std::int64_t> value_{0};
35 };
36
37 class BoundedMpmcQueue {
38 public:
39     explicit BoundedMpmcQueue(std::int32_t capacity);
40
41     [[nodiscard]] bool push(std::int32_t value);
42     [[nodiscard]] std::optional<std::int32_t> pop();
43     [[nodiscard]] std::optional<std::int32_t> try_pop();
44     void close();
45     [[nodiscard]] bool closed() const;
46     [[nodiscard]] std::int32_t size() const;
47

```

```

48     struct Report {
49         std::int64_t push_success = 0;
50         std::int64_t push_closed = 0;
51         std::int64_t push_wait_count = 0;
52         std::int64_t push_wait_ns = 0;
53         std::int64_t pop_success = 0;
54         std::int64_t pop_closed = 0;
55         std::int64_t pop_wait_count = 0;
56         std::int64_t pop_wait_ns = 0;
57         std::int64_t close_count = 0;
58         std::int32_t max_size = 0;
59     };
60
61     [[nodiscard]] Report report() const;
62
63 private:
64     std::vector<std::int32_t> buffer_;
65     std::int32_t head_ = 0;
66     std::int32_t tail_ = 0;
67     std::int32_t size_ = 0;
68     bool closed_ = false;
69     Report report_;
70     mutable std::mutex mutex_;
71     std::condition_variable not_full_;
72     std::condition_variable not_empty_;
73 };
74
75 } // namespace compsys

```

Listing 2.15 include/compsys/sync_primitives.hpp

```

1  #pragma once
2
3  #include <cstdint>
4  #include <string>
5
6  namespace compsys {
7
8  struct PermitLimiterReport {
9      std::int32_t task_count = 0;
10     std::int32_t permit_count = 0;
11     std::int32_t entered_count = 0;
12     std::int32_t max_in_flight = 0;
13 };
14
15 [[nodiscard]] PermitLimiterReport run_permit_limiter(std::int32_t task_count,
16                                                     std::int32_t permit_count)↵
17     ↵ ;
18
19 struct PhaseBarrierReport {
20     std::int32_t worker_count = 0;
21     std::int32_t phase_count = 0;

```

```

21     std::int32_t completed_phases = 0;
22     std::int32_t arrival_count = 0;
23 };
24
25 [[nodiscard]] PhaseBarrierReport run_phase_barrier(std::int32_t worker_count,
26                                                    std::int32_t phase_count);
27
28 struct FutureResultReport {
29     bool value_ready = false;
30     bool exception_ready = false;
31     bool broken_promise_seen = false;
32     std::int32_t value = 0;
33     std::string exception_message;
34 };
35
36 [[nodiscard]] FutureResultReport run_future_result_paths();
37
38 struct AtomicWaitReport {
39     std::int32_t final_version = 0;
40     std::int32_t wake_count = 0;
41     std::int32_t observed_sum = 0;
42 };
43
44 [[nodiscard]] AtomicWaitReport run_atomic_wait_version_counter(
45     std::int32_t publish_count);
46
47 } // namespace compsys

```

Listing 2.16 include/compsys/wait_channels.hpp

```

1 #pragma once
2
3 #include <cstdint>
4
5 namespace compsys {
6
7 struct EventFdEpollReport {
8     std::int32_t wake_write_count = 0;
9     std::uint64_t first_counter_read = 0;
10    std::uint64_t second_counter_read = 0;
11    std::int32_t epoll_ready_observations = 0;
12    std::int32_t epoll_empty_observations = 0;
13    std::int32_t queue_drain_count = 0;
14    std::int32_t semaphore_read_count = 0;
15    bool queue_fifo_ok = false;
16    bool semaphore_empty_after_reads = false;
17 };
18
19 [[nodiscard]] EventFdEpollReport run_eventfd_epoll_contract();
20
21 } // namespace compsys

```


Listing 2.17 include/compsys/task_runtime.hpp

```

1  #pragma once
2
3  #include <condition_variable>
4  #include <cstdint>
5  #include <deque>
6  #include <functional>
7  #include <mutex>
8  #include <ostream>
9  #include <thread>
10 #include <vector>
11
12 namespace compsys {
13
14 class TaskRuntime {
15 public:
16     struct Report {
17         std::int32_t worker_count = 0;
18         std::int32_t active_workers = 0;
19         std::int32_t queued_tasks = 0;
20         std::int32_t max_queue_depth = 0;
21         std::int64_t unfinished_count = 0;
22         std::int64_t submitted_count = 0;
23         std::int64_t completed_count = 0;
24         std::int64_t failed_count = 0;
25         std::int64_t rejected_count = 0;
26         bool accepting = false;
27         bool stopping = false;
28     };
29
30     explicit TaskRuntime(std::int32_t worker_count);
31     ~TaskRuntime();
32
33     TaskRuntime(const TaskRuntime&) = delete;
34     TaskRuntime& operator=(const TaskRuntime&) = delete;
35     TaskRuntime(TaskRuntime&&) = delete;
36     TaskRuntime& operator=(TaskRuntime&&) = delete;
37
38     [[nodiscard]] bool submit(std::function<void()> task);
39     void wait_idle();
40     void shutdown();
41
42     [[nodiscard]] Report report() const;
43     [[nodiscard]] std::int32_t queued_task_count() const;
44
45 private:
46     void worker_loop();
47
48     std::deque<std::function<void()>> tasks_;
49     std::vector<std::thread> workers_;
50     mutable std::mutex mutex_;
51     std::condition_variable ready_;

```

```

52     std::condition_variable idle_;
53     bool accepting_ = true;
54     bool stopping_ = false;
55     std::int32_t active_workers_ = 0;
56     Report report_;
57 };
58
59 struct SamePoolFutureWaitReport {
60     std::int32_t worker_count = 0;
61     std::int32_t parent_count = 0;
62     std::int32_t blocked_parent_count = 0;
63     std::int32_t child_submitted_count_at_stall = 0;
64     std::int32_t queued_child_count_at_stall = 0;
65     std::int32_t child_started_before_release = 0;
66     std::int32_t child_completed_count = 0;
67     bool reached_stall = false;
68     bool starved_without_extra_worker = false;
69 };
70
71 [[nodiscard]] SamePoolFutureWaitReport run_same_pool_future_wait_probe(
72     std::int32_t worker_count);
73
74 struct ContinuationRuntimeReport {
75     std::int32_t worker_count = 0;
76     std::int32_t root_task_count = 0;
77     std::int32_t completed_roots = 0;
78     std::int32_t completed_continuations = 0;
79     std::int32_t rejected_continuations = 0;
80     std::int32_t max_queue_depth = 0;
81     std::int64_t final_sum = 0;
82     bool wait_idle_completed = false;
83 };
84
85 [[nodiscard]] ContinuationRuntimeReport run_continuation_runtime_probe(
86     std::int32_t worker_count,
87     std::int32_t root_task_count);
88
89 void write_task_runtime_report_csv(std::ostream& output,
90     const TaskRuntime::Report& report);
91
92 } // namespace compsys

```

Listing 2.18 include/compsys/futex_lab.hpp

```

1 #pragma once
2
3 #include <cstdint>
4
5 namespace compsys {
6
7 struct FutexMutexProbeReport {
8     std::int32_t worker_count = 0;

```

```

9     std::int32_t increments_per_worker = 0;
10    std::int32_t final_value = 0;
11    std::int32_t contended_count = 0;
12    std::int32_t wait_count = 0;
13    std::int32_t wake_count = 0;
14 };
15
16 [[nodiscard]] FutexMutexProbeReport run_futex_mutex_probe(
17     std::int32_t worker_count,
18     std::int32_t increments_per_worker);
19
20 struct FutexContractProbeReport {
21     std::int32_t waiter_count = 0;
22     std::int32_t wake_count = 0;
23     std::int32_t eagain_count = 0;
24     std::int32_t timeout_count = 0;
25     std::int32_t eintr_count = 0;
26     std::int32_t observed_ready_count = 0;
27     bool wait_released_on_ready = false;
28 };
29
30 [[nodiscard]] FutexContractProbeReport run_futex_contract_probe();
31
32 } // namespace compsys

```

三、Compute Systems 实现和探针

这一组文件是第二条主线的完整实现与命令行探针。实现文件负责让章节模型、并行 reduce、同步原语、wait channel、任务运行时和 futex 实验真正运行；probe 文件负责打印稳定输出，让书中结论、命令行输出和测试断言可以互相对照。

Listing 2.19 src/chapter_models.cpp

```

1  #include <compsys/chapter_models.hpp>
2
3  #include <algorithm>
4  #include <cstdint>
5  #include <map>
6  #include <set>
7  #include <stdexcept>
8  #include <vector>
9
10 namespace compsys {
11 namespace {
12
13 constexpr std::int32_t COMPSYS_MODEL_RECORD_ACCEPT_THRESHOLD = 0;
14 constexpr std::int32_t COMPSYS_MODEL_HISTORY_INITIAL_PREDICTION = 0;
15 constexpr std::int32_t COMPSYS_MODEL_PUSH_OPERATION = 1;
16 constexpr std::int32_t COMPSYS_MODEL_POP_OPERATION = -1;
17
18 void require_positive(std::int64_t value, const char* message) {
19     if (value <= 0) {

```

```

20     throw std::runtime_error(message);
21 }
22 }
23
24 [[nodiscard]] std::int32_t ceil_div(std::int32_t numerator,
25                                     std::int32_t denominator) {
26     require_positive(denominator, "denominator must be positive");
27     if (numerator <= 0) {
28         return 0;
29     }
30     return (numerator + denominator - 1) / denominator;
31 }
32
33 [[nodiscard]] bool contains_value(std::span<const std::int32_t> values,
34                                   std::int32_t needle) {
35     return std::find(values.begin(), values.end(), needle) != values.end();
36 }
37
38 } // namespace
39
40 Chapter01PipelineReport model_ch01_input_pipeline(
41     std::span<const std::int32_t> records) {
42     Chapter01PipelineReport report;
43     report.records_total = static_cast<std::int32_t>(records.size());
44     for (const std::int32_t record : records) {
45         if (record >= COMPSYS_MODEL_RECORD_ACCEPT_THRESHOLD) {
46             ++report.accepted;
47             report.checksum = (report.checksum * 131) + record + 17;
48         } else {
49             ++report.rejected;
50             report.checksum = (report.checksum * 131) - record + 19;
51         }
52     }
53     return report;
54 }
55
56 Chapter02BranchReport model_ch02_branch_predictor(
57     std::span<const std::int32_t> values,
58     std::int32_t taken_threshold) {
59     Chapter02BranchReport report;
60     report.iterations = static_cast<std::int32_t>(values.size());
61     bool predicted_taken = false;
62     for (const std::int32_t value : values) {
63         const bool actual_taken = value >= taken_threshold;
64         if (predicted_taken) {
65             ++report.predicted_taken;
66         }
67         if (actual_taken) {
68             ++report.actual_taken;
69         }
70         if (predicted_taken != actual_taken) {
71             ++report.mispredictions;

```

```

72     }
73     predicted_taken = actual_taken;
74 }
75 return report;
76 }
77
78 Chapter03MemoryAccessReport model_ch03_cache_tlb(
79     std::span<const std::uint64_t> byte_offsets,
80     std::int32_t cache_line_bytes,
81     std::int32_t page_bytes,
82     std::int32_t tlb_entries) {
83     require_positive(cache_line_bytes, "cache line bytes must be positive");
84     require_positive(page_bytes, "page bytes must be positive");
85     require_positive(tlb_entries, "TLB entries must be positive");
86
87     std::set<std::uint64_t> cache_lines;
88     std::vector<std::uint64_t> tlb_lru;
89     std::set<std::uint64_t> unique_pages;
90     std::int32_t tlb_misses = 0;
91
92     for (const std::uint64_t offset : byte_offsets) {
93         cache_lines.insert(offset / static_cast<std::uint64_t>(cache_line_bytes↵
94         ↵ ));
95         const std::uint64_t page = offset / static_cast<std::uint64_t>(↵
96         ↵ page_bytes);
97         unique_pages.insert(page);
98
99         const auto it = std::find(tlb_lru.begin(), tlb_lru.end(), page);
100         if (it == tlb_lru.end()) {
101             ++tlb_misses;
102             if (static_cast<std::int32_t>(tlb_lru.size()) == tlb_entries) {
103                 tlb_lru.erase(tlb_lru.begin());
104             }
105         } else {
106             tlb_lru.erase(it);
107         }
108         tlb_lru.push_back(page);
109     }
110
111     Chapter03MemoryAccessReport report;
112     report.access_count = static_cast<std::int32_t>(byte_offsets.size());
113     report.unique_cache_lines = static_cast<std::int32_t>(cache_lines.size());
114     report.unique_pages = static_cast<std::int32_t>(unique_pages.size());
115     report.tlb_misses = tlb_misses;
116     return report;
117 }
118
119 Chapter04NumaReport model_ch04_numa_coherence(
120     std::span<const std::int32_t> worker_nodes,
121     std::span<const std::int32_t> memory_nodes) {
122     if (worker_nodes.size() != memory_nodes.size()) {
123         throw std::runtime_error("worker and memory node spans must have equal ↵

```

```

122     ↪ size");
123 }
124
125 Chapter04NumaReport report;
126 report.access_count = static_cast<std::int32_t>(worker_nodes.size());
127 std::int32_t previous_writer_node = -1;
128 for (std::size_t i = 0; i < worker_nodes.size(); ++i) {
129     const std::int32_t worker_node = worker_nodes[i];
130     const std::int32_t memory_node = memory_nodes[i];
131     if (worker_node < 0 || memory_node < 0) {
132         throw std::runtime_error("NUMA node ids must not be negative");
133     }
134     if (worker_node == memory_node) {
135         ++report.local_accesses;
136     } else {
137         ++report.remote_accesses;
138     }
139     if (previous_writer_node != -1 && previous_writer_node != worker_node) ↪
140     ↪ {
141         ++report.coherence_transfers;
142     }
143     previous_writer_node = worker_node;
144 }
145 return report;
146 }
147
148 Chapter05RooflineReport model_ch05_roofline(
149     double flops,
150     double bytes_read_written,
151     double peak_gflops,
152     double bandwidth_gbytes_per_second) {
153     if (flops <= 0.0 || bytes_read_written <= 0.0 || peak_gflops <= 0.0 ||
154         bandwidth_gbytes_per_second <= 0.0) {
155         throw std::runtime_error("roofline inputs must be positive");
156     }
157
158     Chapter05RooflineReport report;
159     report.arithmetic_intensity = flops / bytes_read_written;
160     report.compute_bound_gflops = peak_gflops;
161     report.memory_bound_gflops =
162         report.arithmetic_intensity * bandwidth_gbytes_per_second;
163     report.attainable_gflops =
164         std::min(report.compute_bound_gflops, report.memory_bound_gflops);
165     report.memory_bound = report.memory_bound_gflops < report.↪
166     ↪ compute_bound_gflops;
167     return report;
168 }
169
170 Chapter05bPmuSimdReport model_ch05b_pmu_simd_evidence(
171     std::int64_t scalar_instructions,
172     std::int64_t vector_instructions,
173     bool counter_available) {

```

```

171     if (scalar_instructions < 0 || vector_instructions < 0) {
172         throw std::runtime_error("instruction counts must not be negative");
173     }
174     Chapter05bPmuSimdReport report;
175     report.scalar_instructions = scalar_instructions;
176     report.vector_instructions = vector_instructions;
177     report.retired_instructions = scalar_instructions + vector_instructions;
178     report.counter_available = counter_available;
179     if (report.retired_instructions > 0) {
180         report.vector_fraction =
181             static_cast<double>(vector_instructions) /
182             static_cast<double>(report.retired_instructions);
183     }
184     report.evidence_complete = counter_available && report.retired_instructions > 0;
185     return report;
186 }
187
188 Chapter06LayoutReport model_ch06_layout_locality(
189     std::int64_t records,
190     std::int32_t field_count,
191     std::int32_t hot_field_count,
192     std::int32_t field_bytes) {
193     require_positive(records, "records must be positive");
194     require_positive(field_count, "field count must be positive");
195     require_positive(hot_field_count, "hot field count must be positive");
196     require_positive(field_bytes, "field bytes must be positive");
197     if (hot_field_count > field_count) {
198         throw std::runtime_error("hot field count must not exceed field count");
199     }
200
201     Chapter06LayoutReport report;
202     report.records = records;
203     report.aos_bytes_touched =
204         records * static_cast<std::int64_t>(field_count) * field_bytes;
205     report.soa_bytes_touched =
206         records * static_cast<std::int64_t>(hot_field_count) * field_bytes;
207     report.saved_bytes = report.aos_bytes_touched - report.soa_bytes_touched;
208     return report;
209 }
210
211 Chapter07SimdReport model_ch07_simd_ilp(std::int32_t elements,
212                                         std::int32_t lane_width) {
213     require_positive(elements, "elements must be positive");
214     require_positive(lane_width, "lane width must be positive");
215     Chapter07SimdReport report;
216     report.elements = elements;
217     report.lane_width = lane_width;
218     report.vector_iterations = elements / lane_width;
219     report.scalar_tail = elements % lane_width;
220     const std::int32_t vector_slots =

```

```

221         ceil_div(elements, lane_width) * lane_width;
222     report.lane_utilization =
223         static_cast<double>(elements) / static_cast<double>(vector_slots);
224     return report;
225 }
226
227 Chapter08KernelReport model_ch08_map_filter_reduce(
228     std::span<const std::int32_t> values) {
229     Chapter08KernelReport report;
230     report.input_count = static_cast<std::int32_t>(values.size());
231     for (const std::int32_t value : values) {
232         if ((value % 2) == 0) {
233             ++report.filtered_count;
234             const std::int32_t mapped = value * value;
235             report.mapped_sum += mapped;
236             report.reduced_sum += mapped;
237         }
238     }
239     return report;
240 }
241
242 Chapter09SchedulingReport model_ch09_thread_scheduling(
243     std::span<const std::int32_t> task_costs,
244     std::int32_t worker_count) {
245     require_positive(worker_count, "worker count must be positive");
246     std::vector<std::int64_t> loads(static_cast<std::size_t>(worker_count), 0);
247     for (const std::int32_t cost : task_costs) {
248         if (cost < 0) {
249             throw std::runtime_error("task cost must not be negative");
250         }
251         auto it = std::min_element(loads.begin(), loads.end());
252         *it += cost;
253     }
254
255     Chapter09SchedulingReport report;
256     report.worker_count = worker_count;
257     report.task_count = static_cast<std::int32_t>(task_costs.size());
258     report.min_load = *std::min_element(loads.begin(), loads.end());
259     report.max_load = *std::max_element(loads.begin(), loads.end());
260     report.imbalance = report.max_load - report.min_load;
261     return report;
262 }
263
264 Chapter09bLinuxWaitReport model_ch09b_linux_wait_wake(
265     std::span<const bool> ready_observations) {
266     Chapter09bLinuxWaitReport report;
267     report.poll_count = static_cast<std::int32_t>(ready_observations.size());
268     for (const bool ready : ready_observations) {
269         ++report.wakeups;
270         if (ready) {
271             report.completed = true;
272             return report;

```



```

273     }
274     ++report.spurious_wakeups;
275 }
276 return report;
277 }
278
279 Chapter10BackpressureReport model_ch10_sync_backpressure(
280     std::int32_t ticks,
281     std::int32_t produce_per_tick,
282     std::int32_t consume_per_tick,
283     std::int32_t capacity) {
284     require_positive(ticks, "ticks must be positive");
285     require_positive(produce_per_tick, "produce rate must be positive");
286     require_positive(consume_per_tick, "consume rate must be positive");
287     require_positive(capacity, "capacity must be positive");
288
289     Chapter10BackpressureReport report;
290     report.ticks = ticks;
291     report.capacity = capacity;
292     std::int32_t depth = 0;
293     for (std::int32_t tick = 0; tick < ticks; ++tick) {
294         const std::int32_t available_space = capacity - depth;
295         const std::int32_t accepted = std::min(produce_per_tick, ←
        ↪ available_space);
296         const std::int32_t rejected = produce_per_tick - accepted;
297         report.produced += accepted;
298         if (rejected > 0) {
299             ++report.backpressure_events;
300         }
301         depth += accepted;
302         const std::int32_t consumed = std::min(depth, consume_per_tick);
303         depth -= consumed;
304         report.consumed += consumed;
305         report.max_depth = std::max(report.max_depth, depth);
306     }
307     return report;
308 }
309
310 Chapter11AtomicPublicationReport model_ch11_atomic_publication(
311     std::span<const std::int32_t> observed_versions,
312     std::int32_t final_version) {
313     require_positive(final_version, "final version must be positive");
314     Chapter11AtomicPublicationReport report;
315     report.published_versions = final_version;
316     report.observed_versions = static_cast<std::int32_t>(observed_versions.size() ←
        ↪ ());
317     report.final_version = final_version;
318     std::int32_t previous = COMPSYS_MODEL_HISTORY_INITIAL_PREDICTION;
319     for (const std::int32_t observed : observed_versions) {
320         if (observed < 0) {
321             throw std::runtime_error("observed version must not be negative");
322         }

```

```

323     if (observed < previous) {
324         report.monotonic_observations = false;
325     }
326     if (observed < final_version) {
327         ++report.stale_reads;
328     }
329     previous = observed;
330 }
331 return report;
332 }
333
334 Chapter12LockFreeQueueReport model_ch12_spsc_ring(
335     std::span<const std::int32_t> operations,
336     std::int32_t capacity) {
337     require_positive(capacity, "capacity must be positive");
338     Chapter12LockFreeQueueReport report;
339     report.capacity = capacity;
340     std::int32_t depth = 0;
341     for (const std::int32_t operation : operations) {
342         if (operation == COMPSYS_MODEL_PUSH_OPERATION) {
343             if (depth == capacity) {
344                 ++report.rejected_full;
345             } else {
346                 ++depth;
347                 ++report.pushed;
348             }
349         } else if (operation == COMPSYS_MODEL_POP_OPERATION) {
350             if (depth == 0) {
351                 ++report.empty_pops;
352             } else {
353                 --depth;
354                 ++report.popped;
355             }
356         } else {
357             throw std::runtime_error("queue operation must be 1 for push or -1 ←
↪ for pop");
358         }
359     }
360     return report;
361 }
362
363 Chapter12bReclamationReport model_ch12b_epoch_reclamation(
364     std::span<const std::int32_t> retired_node_ids,
365     std::span<const std::int32_t> protected_node_ids) {
366     Chapter12bReclamationReport report;
367     report.retired_nodes = static_cast<std::int32_t>(retired_node_ids.size());
368     report.protected_nodes = static_cast<std::int32_t>(protected_node_ids.size() ←
↪ ());
369     for (const std::int32_t node : retired_node_ids) {
370         if (contains_value(protected_node_ids, node)) {
371             ++report.deferred_nodes;
372         } else {

```

```

373         ++report.reclaimed_nodes;
374     }
375 }
376 return report;
377 }
378
379 Chapter13ReduceScanPartitionReport model_ch13_reduce_scan_partition(
380     std::span<const std::int32_t> values,
381     std::int32_t partition_threshold) {
382     Chapter13ReduceScanPartitionReport report;
383     report.input_count = static_cast<std::int32_t>(values.size());
384     std::int64_t prefix = 0;
385     for (const std::int32_t value : values) {
386         report.reduce_sum += value;
387         prefix += value;
388         report.last_prefix = prefix;
389         if (value >= partition_threshold) {
390             ++report.partition_true_count;
391         } else {
392             ++report.partition_false_count;
393         }
394     }
395     return report;
396 }
397
398 Chapter14WorkStealingReport model_ch14_work_stealing(
399     std::span<const std::int32_t> initial_worker_queues) {
400     if (initial_worker_queues.empty()) {
401         throw std::runtime_error("at least one worker queue is required");
402     }
403     Chapter14WorkStealingReport report;
404     report.worker_count = static_cast<std::int32_t>(initial_worker_queues.size()
405     ↵ ());
406     for (const std::int32_t queue_depth : initial_worker_queues) {
407         if (queue_depth < 0) {
408             throw std::runtime_error("queue depth must not be negative");
409         }
410         report.initial_tasks += queue_depth;
411         report.max_queue_depth = std::max(report.max_queue_depth, queue_depth);
412     }
413     const std::int32_t target_load =
414         ceil_div(report.initial_tasks, report.worker_count);
415     for (const std::int32_t queue_depth : initial_worker_queues) {
416         if (queue_depth > target_load) {
417             report.stolen_tasks += queue_depth - target_load;
418         }
419     }
420     report.completed_tasks = report.initial_tasks;
421     return report;
422 }
423 Chapter15AsyncIoReport model_ch15_async_io_backpressure(

```

```

424     std::int32_t request_count,
425     std::int32_t queue_depth,
426     std::int32_t completions_per_tick) {
427     require_positive(request_count, "request count must be positive");
428     require_positive(queue_depth, "queue depth must be positive");
429     require_positive(completions_per_tick, "completions per tick must be ←
↪ positive");
430
431     Chapter15AsyncIoReport report;
432     report.requested = request_count;
433     std::int32_t remaining = request_count;
434     std::int32_t in_flight = 0;
435     while (remaining > 0 || in_flight > 0) {
436         while (remaining > 0 && in_flight < queue_depth) {
437             --remaining;
438             ++in_flight;
439             ++report.submitted;
440             report.max_in_flight = std::max(report.max_in_flight, in_flight);
441         }
442         if (remaining > 0 && in_flight == queue_depth) {
443             ++report.backpressure_events;
444         }
445         const std::int32_t completed = std::min(in_flight, completions_per_tick←
↪ );
446         in_flight -= completed;
447         report.completed += completed;
448     }
449     return report;
450 }
451
452 Chapter16DistributedModelReport model_ch16_distributed_attempts(
453     std::span<const std::int32_t> task_ids,
454     std::span<const std::int32_t> generations,
455     std::int32_t current_generation) {
456     if (task_ids.size() != generations.size()) {
457         throw std::runtime_error("task id and generation spans must have equal ←
↪ size");
458     }
459     Chapter16DistributedModelReport report;
460     report.attempts = static_cast<std::int32_t>(task_ids.size());
461     std::map<std::int32_t, std::int32_t> current_attempt_counts;
462     for (std::size_t i = 0; i < task_ids.size(); ++i) {
463         if (task_ids[i] < 0 || generations[i] < 0) {
464             throw std::runtime_error("task ids and generations must not be ←
↪ negative");
465         }
466         if (generations[i] != current_generation) {
467             ++report.stale_generation_attempts;
468             continue;
469         }
470         ++current_attempt_counts[task_ids[i]];
471     }

```

```

472     report.logical_tasks = static_cast<std::int32_t>(current_attempt_counts.↵
↵ size());
473     for (const auto& [_, count] : current_attempt_counts) {
474         if (count > 1) {
475             report.duplicate_attempts += count - 1;
476         }
477     }
478     return report;
479 }
480
481 Chapter17ReplicationReport model_ch17_quorum_commit(
482     std::int32_t replica_count,
483     std::span<const bool> acknowledgements) {
484     require_positive(replica_count, "replica count must be positive");
485     if (static_cast<std::int32_t>(acknowledgements.size()) != replica_count) {
486         throw std::runtime_error("acknowledgement count must equal replica ↵
↵ count");
487     }
488     Chapter17ReplicationReport report;
489     report.replica_count = replica_count;
490     report.quorum_size = (replica_count / 2) + 1;
491     for (const bool acknowledged : acknowledgements) {
492         if (acknowledged) {
493             ++report.ack_count;
494         }
495     }
496     report.committed = report.ack_count >= report.quorum_size;
497     return report;
498 }
499
500 Chapter18RecoveryReport model_ch18_checkpoint_recovery(
501     std::int32_t input_records,
502     std::int32_t checkpoint_interval,
503     std::int32_t failed_after_record) {
504     require_positive(input_records, "input records must be positive");
505     require_positive(checkpoint_interval, "checkpoint interval must be positive↵
↵ ");
506     if (failed_after_record < 0 || failed_after_record > input_records) {
507         throw std::runtime_error("failed record must be inside input range");
508     }
509
510     Chapter18RecoveryReport report;
511     report.input_records = input_records;
512     report.checkpoint_interval = checkpoint_interval;
513     report.checkpoints = input_records / checkpoint_interval;
514     report.replay_from_record =
515         (failed_after_record / checkpoint_interval) * checkpoint_interval;
516     report.replay_records = input_records - report.replay_from_record;
517     return report;
518 }
519
520 } // namespace compsys

```

Listing 2.20 src/parallel_reduce.cpp

```
1 #include <compsys/parallel_reduce.hpp>
2
3 #include <algorithm>
4 #include <chrono>
5 #include <stdexcept>
6 #include <thread>
7
8 namespace compsys {
9     namespace {
10
11         constexpr std::int32_t COMPSYS_MIN_WORKER_COUNT = 1;
12
13         using Clock = std::chrono::steady_clock;
14
15         std::int64_t sum_range(std::span<const std::int32_t> values,
16                               std::size_t begin,
17                               std::size_t end) {
18             std::int64_t sum = 0;
19             for (std::size_t i = begin; i < end; ++i) {
20                 sum += values[i];
21             }
22             return sum;
23         }
24
25         std::int64_t sum_partials(std::span<const std::int64_t> partials) {
26             std::int64_t sum = 0;
27             for (const std::int64_t value : partials) {
28                 sum += value;
29             }
30             return sum;
31         }
32
33         std::size_t checked_queue_capacity(std::int32_t capacity) {
34             if (capacity <= 0) {
35                 throw std::runtime_error("queue capacity must be positive");
36             }
37             return static_cast<std::size_t>(capacity);
38         }
39
40     } // namespace
41
42     std::int64_t sequential_sum(std::span<const std::int32_t> values) {
43         return sum_range(values, 0, values.size());
44     }
45
46     std::int64_t parallel_sum(std::span<const std::int32_t> values,
47                              std::int32_t worker_count) {
48         if (worker_count < COMPSYS_MIN_WORKER_COUNT) {
49             throw std::runtime_error("worker_count must be positive");
50         }
51         if (values.empty()) {
```

```

52     return 0;
53 }
54
55 const std::int32_t actual_workers = std::min<std::int32_t>(
56     worker_count,
57     static_cast<std::int32_t>(values.size()));
58 std::vector<std::thread> threads;
59 std::vector<std::int64_t> partials(static_cast<std::size_t>(actual_workers)↵
↵ , 0);
60 threads.reserve(static_cast<std::size_t>(actual_workers));
61
62 for (std::int32_t worker = 0; worker < actual_workers; ++worker) {
63     const std::size_t begin =
64         values.size() * static_cast<std::size_t>(worker) /
65         static_cast<std::size_t>(actual_workers);
66     const std::size_t end =
67         values.size() * static_cast<std::size_t>(worker + 1) /
68         static_cast<std::size_t>(actual_workers);
69     threads.emplace_back([&values, &partials, worker, begin, end]() {
70         partials[static_cast<std::size_t>(worker)] = sum_range(values, ↵
↵ begin, end);
71     });
72 }
73
74 for (std::thread& thread : threads) {
75     thread.join();
76 }
77
78 return sum_partials(partials);
79 }
80
81 void MutexCounter::add(std::int64_t delta) {
82     std::lock_guard<std::mutex> lock(this->mutex_);
83     this->value_ += delta;
84 }
85
86 std::int64_t MutexCounter::value() const {
87     std::lock_guard<std::mutex> lock(this->mutex_);
88     return this->value_;
89 }
90
91 void AtomicCounter::add(std::int64_t delta) {
92     this->value_.fetch_add(delta, std::memory_order_relaxed);
93 }
94
95 std::int64_t AtomicCounter::value() const {
96     return this->value_.load(std::memory_order_relaxed);
97 }
98
99 BoundedMpmcQueue::BoundedMpmcQueue(std::int32_t capacity)
100     : buffer_(checked_queue_capacity(capacity), 0) {}
101

```

```

102 bool BoundedMpmcQueue::push(std::int32_t value) {
103     std::unique_lock<std::mutex> lock(this->mutex_);
104     if (this->closed_) {
105         ++this->report_.push_closed;
106         return false;
107     }
108     Clock::time_point wait_begin;
109     bool waited = false;
110     if (this->size_ == static_cast<std::int32_t>(this->buffer_.size())) {
111         ++this->report_.push_wait_count;
112         wait_begin = Clock::now();
113         waited = true;
114     }
115     this->not_full_.wait(lock, [this]() {
116         return this->closed_ ||
117             this->size_ < static_cast<std::int32_t>(this->buffer_.size());
118     });
119     if (waited) {
120         this->report_.push_wait_ns +=
121             std::chrono::duration_cast<std::chrono::nanoseconds>(
122                 Clock::now() - wait_begin)
123                 .count();
124     }
125     if (this->closed_) {
126         ++this->report_.push_closed;
127         return false;
128     }
129
130     this->buffer_[static_cast<std::size_t>(this->tail_)] = value;
131     this->tail_ = (this->tail_ + 1) % static_cast<std::int32_t>(this->buffer_.↵
↵ size());
132     ++this->size_;
133     ++this->report_.push_success;
134     this->report_.max_size = std::max(this->report_.max_size, this->size_);
135     lock.unlock();
136     this->not_empty_.notify_one();
137     return true;
138 }
139
140 std::optional<std::int32_t> BoundedMpmcQueue::pop() {
141     std::unique_lock<std::mutex> lock(this->mutex_);
142     Clock::time_point wait_begin;
143     bool waited = false;
144     if (this->size_ == 0 && !this->closed_) {
145         ++this->report_.pop_wait_count;
146         wait_begin = Clock::now();
147         waited = true;
148     }
149     this->not_empty_.wait(lock, [this]() {
150         return this->closed_ || this->size_ > 0;
151     });
152     if (waited) {

```



```

153         this->report_.pop_wait_ns +=
154             std::chrono::duration_cast<std::chrono::nanoseconds>(
155                 Clock::now() - wait_begin)
156                 .count();
157     }
158     if (this->size_ == 0) {
159         ++this->report_.pop_closed;
160         return std::nullopt;
161     }
162
163     const std::int32_t value = this->buffer_[static_cast<std::size_t>(this->head_)]
164     this->head_ = (this->head_ + 1) % static_cast<std::int32_t>(this->buffer_.size());
165     --this->size_;
166     ++this->report_.pop_success;
167     lock.unlock();
168     this->not_full_.notify_one();
169     return value;
170 }
171
172 std::optional<std::int32_t> BoundedMpmcQueue::try_pop() {
173     std::lock_guard<std::mutex> lock(this->mutex_);
174     if (this->size_ == 0) {
175         return std::nullopt;
176     }
177     const std::int32_t value = this->buffer_[static_cast<std::size_t>(this->head_)]
178     this->head_ = (this->head_ + 1) % static_cast<std::int32_t>(this->buffer_.size());
179     --this->size_;
180     ++this->report_.pop_success;
181     this->not_full_.notify_one();
182     return value;
183 }
184
185 void BoundedMpmcQueue::close() {
186     std::lock_guard<std::mutex> lock(this->mutex_);
187     if (!this->closed_) {
188         this->closed_ = true;
189         ++this->report_.close_count;
190     }
191     this->not_empty_.notify_all();
192     this->not_full_.notify_all();
193 }
194
195 bool BoundedMpmcQueue::closed() const {
196     std::lock_guard<std::mutex> lock(this->mutex_);
197     return this->closed_;
198 }
199
200 std::int32_t BoundedMpmcQueue::size() const {

```

```

201     std::lock_guard<std::mutex> lock(this->mutex_);
202     return this->size_;
203 }
204
205 BoundedMpmcQueue::Report BoundedMpmcQueue::report() const {
206     std::lock_guard<std::mutex> lock(this->mutex_);
207     return this->report_;
208 }
209
210 } // namespace compsys

```

Listing 2.21 src/sync_primitives.cpp

```

1  #include <compsys/sync_primitives.hpp>
2
3  #include <algorithm>
4  #include <atomic>
5  #include <barrier>
6  #include <future>
7  #include <latch>
8  #include <limits>
9  #include <semaphore>
10 #include <stdexcept>
11 #include <string>
12 #include <thread>
13 #include <vector>
14
15 namespace compsys {
16 namespace {
17
18 constexpr std::int32_t COMPSYS_MIN_SYNC_COUNT = 1;
19 constexpr std::int32_t COMPSYS_READY_VALUE = 42;
20
21 void require_positive(std::int32_t value, const char* name) {
22     if (value < COMPSYS_MIN_SYNC_COUNT) {
23         throw std::runtime_error(std::string(name) + " must be positive");
24     }
25 }
26
27 void update_max(std::atomic<std::int32_t>& max_value, std::int32_t candidate) {
28     std::int32_t current = max_value.load(std::memory_order_relaxed);
29     while (candidate > current &&
30           !max_value.compare_exchange_weak(current,
31                                           candidate,
32                                           std::memory_order_relaxed,
33                                           std::memory_order_relaxed)) {
34     }
35 }
36
37 } // namespace
38
39 PermitLimiterReport run_permit_limiter(std::int32_t task_count,

```

```

40         std::int32_t permit_count) {
41     require_positive(task_count, "task_count");
42     require_positive(permit_count, "permit_count");
43
44     const std::int32_t expected_peak = std::min(task_count, permit_count);
45     std::counting_semaphore<std::numeric_limits<std::int32_t>::max()> permits(
46         permit_count);
47     std::atomic<std::int32_t> in_flight{0};
48     std::atomic<std::int32_t> entered_count{0};
49     std::atomic<std::int32_t> max_in_flight{0};
50     std::latch start_gate(1);
51     std::latch release_gate(1);
52     std::vector<std::thread> workers;
53     workers.reserve(static_cast<std::size_t>(task_count));
54
55     for (std::int32_t task = 0; task < task_count; ++task) {
56         workers.emplace_back([&permits,
57                               &in_flight,
58                               &entered_count,
59                               &max_in_flight,
60                               &start_gate,
61                               &release_gate]() {
62             start_gate.wait();
63             permits.acquire();
64             const std::int32_t now =
65                 in_flight.fetch_add(1, std::memory_order_acq_rel) + 1;
66             entered_count.fetch_add(1, std::memory_order_relaxed);
67             update_max(max_in_flight, now);
68             release_gate.wait();
69             in_flight.fetch_sub(1, std::memory_order_acq_rel);
70             permits.release();
71         });
72     }
73
74     start_gate.count_down();
75     while (entered_count.load(std::memory_order_acquire) < expected_peak) {
76         std::this_thread::yield();
77     }
78     release_gate.count_down();
79
80     for (std::thread& worker : workers) {
81         worker.join();
82     }
83
84     PermitLimiterReport report;
85     report.task_count = task_count;
86     report.permit_count = permit_count;
87     report.entered_count = entered_count.load(std::memory_order_relaxed);
88     report.max_in_flight = max_in_flight.load(std::memory_order_relaxed);
89     return report;
90 }
91

```

```

92 PhaseBarrierReport run_phase_barrier(std::int32_t worker_count,
93                                     std::int32_t phase_count) {
94     require_positive(worker_count, "worker_count");
95     require_positive(phase_count, "phase_count");
96
97     std::atomic<std::int32_t> completed_phases{0};
98     std::atomic<std::int32_t> arrival_count{0};
99     std::barrier phase(worker_count, [&completed_phases]() noexcept {
100         completed_phases.fetch_add(1, std::memory_order_relaxed);
101     });
102     std::vector<std::thread> workers;
103     workers.reserve(static_cast<std::size_t>(worker_count));
104
105     for (std::int32_t worker = 0; worker < worker_count; ++worker) {
106         workers.emplace_back([phase_count, &arrival_count, &phase]() {
107             for (std::int32_t phase_index = 0; phase_index < phase_count; ++phase_index) {
108                 arrival_count.fetch_add(1, std::memory_order_relaxed);
109                 phase.arrive_and_wait();
110             }
111         });
112     }
113
114     for (std::thread& worker : workers) {
115         worker.join();
116     }
117
118     PhaseBarrierReport report;
119     report.worker_count = worker_count;
120     report.phase_count = phase_count;
121     report.completed_phases = completed_phases.load(std::memory_order_relaxed);
122     report.arrival_count = arrival_count.load(std::memory_order_relaxed);
123     return report;
124 }
125
126 FutureResultReport run_future_result_paths() {
127     FutureResultReport report;
128
129     std::promise<std::int32_t> value_promise;
130     std::future<std::int32_t> value_future = value_promise.get_future();
131     value_promise.set_value(COMPSYS_READY_VALUE);
132     report.value = value_future.get();
133     report.value_ready = report.value == COMPSYS_READY_VALUE;
134
135     std::promise<std::int32_t> exception_promise;
136     std::future<std::int32_t> exception_future = exception_promise.get_future();
137     ;
138     exception_promise.set_exception(std::make_exception_ptr(
139         std::runtime_error("future path failed")));
140     try {
141         static_cast<void>(exception_future.get());
142     } catch (const std::runtime_error& error) {

```

```

142     report.exception_ready = true;
143     report.exception_message = error.what();
144 }
145
146 std::future<std::int32_t> broken_future;
147 {
148     std::promise<std::int32_t> broken_promise;
149     broken_future = broken_promise.get_future();
150 }
151 try {
152     static_cast<void>(broken_future.get());
153 } catch (const std::future_error& error) {
154     report.broken_promise_seen =
155         error.code() == std::make_error_code(std::future_errc::↵
↵ broken_promise);
156 }
157
158 return report;
159 }
160
161 AtomicWaitReport run_atomic_wait_version_counter(std::int32_t publish_count) {
162     require_positive(publish_count, "publish_count");
163
164     std::atomic<std::int32_t> version{0};
165     std::atomic<std::int32_t> observed_sum{0};
166     std::atomic<std::int32_t> wake_count{0};
167     std::latch waiter_started(1);
168
169     std::thread waiter([publish_count, &version, &observed_sum, &wake_count, &↵
↵ waiter_started]() {
170         std::int32_t observed = version.load(std::memory_order_acquire);
171         waiter_started.count_down();
172         while (observed < publish_count) {
173             version.wait(observed, std::memory_order_acquire);
174             const std::int32_t next = version.load(std::memory_order_acquire);
175             if (next != observed) {
176                 wake_count.fetch_add(1, std::memory_order_relaxed);
177                 observed_sum.fetch_add(next, std::memory_order_relaxed);
178                 observed = next;
179             }
180         }
181     });
182
183     waiter_started.wait();
184     for (std::int32_t value = 1; value <= publish_count; ++value) {
185         version.store(value, std::memory_order_release);
186         version.notify_all();
187     }
188
189     waiter.join();
190
191     AtomicWaitReport report;

```

```

192     report.final_version = version.load(std::memory_order_acquire);
193     report.wake_count = wake_count.load(std::memory_order_relaxed);
194     report.observed_sum = observed_sum.load(std::memory_order_relaxed);
195     return report;
196 }
197
198 } // namespace compsys

```

Listing 2.22 src/wait_channels.cpp

```

1  #include <compsys/wait_channels.hpp>
2
3  #include <array>
4  #include <cstdint>
5  #include <deque>
6  #include <stdexcept>
7  #include <vector>
8
9  #include <cerrno>
10 #include <sys/epoll.h>
11 #include <sys/eventfd.h>
12 #include <unistd.h>
13
14 namespace compsys {
15 namespace {
16
17 constexpr std::int32_t COMPSYS_EVENTFD_QUEUE_FIRST = 11;
18 constexpr std::int32_t COMPSYS_EVENTFD_QUEUE_SECOND = 22;
19 constexpr std::int32_t COMPSYS_EVENTFD_QUEUE_THIRD = 33;
20 constexpr std::uint64_t COMPSYS_EVENTFD_WAKE_VALUE = 1;
21 constexpr std::uint64_t COMPSYS_EVENTFD_SEMAPHORE_PULSES = 3;
22 constexpr std::size_t COMPSYS_EVENTFD_EXPECTED_DRAIN_COUNT = 3;
23
24 class UniqueFd {
25 public:
26     UniqueFd() = default;
27     explicit UniqueFd(int fd) noexcept : fd_(fd) {}
28     ~UniqueFd() { this->reset(); }
29
30     UniqueFd(const UniqueFd&) = delete;
31     UniqueFd& operator=(const UniqueFd&) = delete;
32
33     UniqueFd(UniqueFd&& other) noexcept : fd_(other.release()) {}
34
35     UniqueFd& operator=(UniqueFd&& other) noexcept {
36         if (this != &other) {
37             this->reset(other.release());
38         }
39         return *this;
40     }
41
42     [[nodiscard]] int get() const noexcept {

```

```

43     return this->fd_;
44 }
45
46 [[nodiscard]] bool valid() const noexcept {
47     return this->fd_ >= 0;
48 }
49
50 int release() noexcept {
51     const int fd = this->fd_;
52     this->fd_ = -1;
53     return fd;
54 }
55
56 void reset(int fd = -1) noexcept {
57     if (this->fd_ >= 0) {
58         ::close(this->fd_);
59     }
60     this->fd_ = fd;
61 }
62
63 private:
64     int fd_ = -1;
65 };
66
67 [[nodiscard]] UniqueFd make_eventfd(unsigned int flags) {
68     const int fd = ::eventfd(0, flags);
69     if (fd < 0) {
70         throw std::runtime_error("failed to create eventfd");
71     }
72     return UniqueFd(fd);
73 }
74
75 [[nodiscard]] UniqueFd make_epollfd() {
76     const int fd = ::epoll_create1(EPOOL_CLOEXEC);
77     if (fd < 0) {
78         throw std::runtime_error("failed to create epoll instance");
79     }
80     return UniqueFd(fd);
81 }
82
83 void write_eventfd_value(int fd, std::uint64_t value) {
84     while (true) {
85         const ssize_t written = ::write(fd, &value, sizeof(value));
86         if (written == static_cast<ssize_t>(sizeof(value))) {
87             return;
88         }
89         if (written < 0 && errno == EINTR) {
90             continue;
91         }
92         throw std::runtime_error("failed to write eventfd value");
93     }
94 }

```

```

95
96 [[nodiscard]] bool read_eventfd_value(int fd, std::uint64_t& value) {
97     while (true) {
98         const ssize_t read_bytes = ::read(fd, &value, sizeof(value));
99         if (read_bytes == static_cast<ssize_t>(sizeof(value))) {
100             return true;
101         }
102         if (read_bytes < 0 && errno == EINTR) {
103             continue;
104         }
105         if (read_bytes < 0 && errno == EAGAIN) {
106             return false;
107         }
108         throw std::runtime_error("failed to read eventfd value");
109     }
110 }
111
112 [[nodiscard]] bool poll_epoll_ready(int epoll_fd, int expected_fd) {
113     std::array<epoll_event, 1> events{};
114     const int ready = ::epoll_wait(epoll_fd, events.data(), 1, 0);
115     if (ready < 0) {
116         throw std::runtime_error("epoll_wait failed");
117     }
118     if (ready == 0) {
119         return false;
120     }
121     if (ready != 1) {
122         throw std::runtime_error("epoll_wait returned unexpected count");
123     }
124     if (events[0].data.fd != expected_fd) {
125         throw std::runtime_error("epoll returned unexpected fd");
126     }
127     if ((events[0].events & EPOLLIN) == 0) {
128         throw std::runtime_error("epoll returned unexpected event mask");
129     }
130     return true;
131 }
132
133 } // namespace
134
135 EventFdEpollReport run_eventfd_epoll_contract() {
136     EventFdEpollReport report;
137
138     UniqueFd queue_wake_fd = make_eventfd(EFD_CLOEXEC | EFD_NONBLOCK);
139     UniqueFd semaphore_fd = make_eventfd(EFD_CLOEXEC | EFD_NONBLOCK | ↵
    ↵ EFD_SEMAPHORE);
140     UniqueFd epoll_fd = make_epollfd();
141
142     epoll_event event{};
143     event.events = EPOLLIN;
144     event.data.fd = queue_wake_fd.get();
145     if (::epoll_ctl(epoll_fd.get(), EPOLL_CTL_ADD, queue_wake_fd.get(), &event)↵

```



```

146     != 0) {
147         throw std::runtime_error("failed to register eventfd with epoll");
148     }
149
150     std::deque<std::int32_t> queue;
151     std::vector<std::int32_t> drained_values;
152     drained_values.reserve(COMPSYS_EVENTFD_EXPECTED_DRAIN_COUNT);
153
154     auto enqueue_request = [&](std::int32_t value) {
155         const bool was_empty = queue.empty();
156         queue.push_back(value);
157         if (was_empty) {
158             write_eventfd_value(queue_wake_fd.get(), COMPSYS_EVENTFD_WAKE_VALUE);
159         }
160     };
161
162     auto drain_queue = [&]() {
163         while (!queue.empty()) {
164             drained_values.push_back(queue.front());
165             queue.pop_front();
166             ++report.queue_drain_count;
167         }
168     };
169
170     enqueue_request(COMPSYS_EVENTFD_QUEUE_FIRST);
171     enqueue_request(COMPSYS_EVENTFD_QUEUE_SECOND);
172     if (!poll_epoll_ready(epoll_fd.get(), queue_wake_fd.get())) {
173         throw std::runtime_error("eventfd should become readable after queue
174         wake");
175     }
176     ++report.epoll_ready_observations;
177
178     std::uint64_t value = 0;
179     if (!read_eventfd_value(queue_wake_fd.get(), value)) {
180         throw std::runtime_error("queue wake fd should be readable");
181     }
182     report.first_counter_read = value;
183
184     drain_queue();
185     if (poll_epoll_ready(epoll_fd.get(), queue_wake_fd.get())) {
186         throw std::runtime_error("eventfd should be empty after queue drain");
187     }
188     ++report.epoll_empty_observations;
189
190     enqueue_request(COMPSYS_EVENTFD_QUEUE_THIRD);
191     if (!poll_epoll_ready(epoll_fd.get(), queue_wake_fd.get())) {
192         throw std::runtime_error("eventfd should become readable after second
193         wake");
194     }
195     ++report.epoll_ready_observations;

```

```

194
195     value = 0;
196     if (!read_eventfd_value(queue_wake_fd.get(), value)) {
197         throw std::runtime_error("queue wake fd should be readable after second
↵ wake");
198     }
199     report.second_counter_read = value;
200
201     drain_queue();
202     if (poll_epoll_ready(epoll_fd.get(), queue_wake_fd.get())) {
203         throw std::runtime_error("eventfd should be empty after second drain");
204     }
205     ++report.epoll_empty_observations;
206
207     report.queue_fifo_ok =
208         drained_values.size() == COMPSYS_EVENTFD_EXPECTED_DRAIN_COUNT &&
209         drained_values[0] == COMPSYS_EVENTFD_QUEUE_FIRST &&
210         drained_values[1] == COMPSYS_EVENTFD_QUEUE_SECOND &&
211         drained_values[2] == COMPSYS_EVENTFD_QUEUE_THIRD;
212
213     if (::write(semaphore_fd.get(), &COMPSYS_EVENTFD_SEMAPHORE_PULSES,
214             sizeof(COMPSYS_EVENTFD_SEMAPHORE_PULSES)) !=
215         static_cast<ssize_t>(sizeof(COMPSYS_EVENTFD_SEMAPHORE_PULSES))) {
216         throw std::runtime_error("failed to prime eventfd semaphore mode");
217     }
218
219     for (std::int32_t i = 0; i < static_cast<std::int32_t>(
↵ COMPSYS_EVENTFD_SEMAPHORE_PULSES);
220         ++i) {
221         std::uint64_t one = 0;
222         if (!read_eventfd_value(semaphore_fd.get(), one)) {
223             throw std::runtime_error("semaphore eventfd should be readable");
224         }
225         if (one != 1U) {
226             throw std::runtime_error("semaphore eventfd should return one wake
↵ per read");
227         }
228         ++report.semaphore_read_count;
229     }
230
231     std::uint64_t empty = 0;
232     report.semaphore_empty_after_reads = !read_eventfd_value(semaphore_fd.get()
↵ , empty);
233     return report;
234 }
235
236 } // namespace compsys

```

Listing 2.23 src/task_runtime.cpp

```

1 #include <compsys/task_runtime.hpp>
2

```

```

3  #include <algorithm>
4  #include <atomic>
5  #include <chrono>
6  #include <future>
7  #include <latch>
8  #include <memory>
9  #include <ostream>
10 #include <stdexcept>
11 #include <string>
12 #include <utility>
13
14 namespace compsys {
15 namespace {
16
17 constexpr std::int32_t COMPSYS_TASK_RUNTIME_MIN_WORKERS = 1;
18 constexpr std::int32_t COMPSYS_TASK_RUNTIME_CHILD_VALUE = 1;
19 constexpr std::int32_t COMPSYS_TASK_RUNTIME_STALL_TIMEOUT_MS = 2000;
20
21 void require_positive(std::int32_t value, const char* name) {
22     if (value < COMPSYS_TASK_RUNTIME_MIN_WORKERS) {
23         throw std::runtime_error(std::string(name) + " must be positive");
24     }
25 }
26
27 } // namespace
28
29 TaskRuntime::TaskRuntime(std::int32_t worker_count) {
30     require_positive(worker_count, "worker_count");
31     this->report_.worker_count = worker_count;
32     this->workers_.reserve(static_cast<std::size_t>(worker_count));
33     for (std::int32_t worker = 0; worker < worker_count; ++worker) {
34         this->workers_.emplace_back([this]() {
35             this->worker_loop();
36         });
37     }
38 }
39
40 TaskRuntime::~TaskRuntime() {
41     this->shutdown();
42 }
43
44 bool TaskRuntime::submit(std::function<void()> task) {
45     if (!task) {
46         throw std::runtime_error("cannot submit an empty task");
47     }
48
49     {
50         std::lock_guard<std::mutex> lock(this->mutex_);
51         if (!this->accepting_ || this->stopping_) {
52             ++this->report_.rejected_count;
53             return false;
54         }

```

```

55     this->tasks_.push_back(std::move(task));
56     ++this->report_.submitted_count;
57     ++this->report_.unfinished_count;
58     this->report_.max_queue_depth = std::max(
59         this->report_.max_queue_depth,
60         static_cast<std::int32_t>(this->tasks_.size()));
61 }
62 this->ready_.notify_one();
63 return true;
64 }
65
66 void TaskRuntime::wait_idle() {
67     std::unique_lock<std::mutex> lock(this->mutex_);
68     this->idle_.wait(lock, [this]() {
69         return this->report_.unfinished_count == 0;
70     });
71 }
72
73 void TaskRuntime::shutdown() {
74     {
75         std::lock_guard<std::mutex> lock(this->mutex_);
76         this->accepting_ = false;
77         this->stopping_ = true;
78     }
79     this->ready_.notify_all();
80
81     for (std::thread& worker : this->workers_) {
82         if (worker.joinable()) {
83             worker.join();
84         }
85     }
86 }
87
88 TaskRuntime::Report TaskRuntime::report() const {
89     std::lock_guard<std::mutex> lock(this->mutex_);
90     Report snapshot = this->report_;
91     snapshot.active_workers = this->active_workers_;
92     snapshot.queued_tasks = static_cast<std::int32_t>(this->tasks_.size());
93     snapshot.accepting = this->accepting_;
94     snapshot.stopping = this->stopping_;
95     return snapshot;
96 }
97
98 std::int32_t TaskRuntime::queued_task_count() const {
99     std::lock_guard<std::mutex> lock(this->mutex_);
100     return static_cast<std::int32_t>(this->tasks_.size());
101 }
102
103 void TaskRuntime::worker_loop() {
104     while (true) {
105         std::function<void()> task;
106         {

```

```

107         std::unique_lock<std::mutex> lock(this->mutex_);
108         this->ready_.wait(lock, [this]() {
109             return this->stopping_ || !this->tasks_.empty();
110         });
111         if (this->tasks_.empty()) {
112             if (this->stopping_) {
113                 return;
114             }
115             continue;
116         }
117         task = std::move(this->tasks_.front());
118         this->tasks_.pop_front();
119         ++this->active_workers_;
120     }
121
122     bool failed = false;
123     try {
124         task();
125     } catch (...) {
126         failed = true;
127     }
128
129     {
130         std::lock_guard<std::mutex> lock(this->mutex_);
131         --this->active_workers_;
132         ++this->report_.completed_count;
133         if (failed) {
134             ++this->report_.failed_count;
135         }
136         --this->report_.unfinished_count;
137         if (this->report_.unfinished_count == 0) {
138             this->idle_.notify_all();
139         }
140     }
141 }
142 }
143
144 SamePoolFutureWaitReport run_same_pool_future_wait_probe(std::int32_t ↵
    ↵ worker_count) {
145     require_positive(worker_count, "worker_count");
146
147     TaskRuntime runtime(worker_count);
148     std::latch start_gate(1);
149     std::mutex progress_mutex;
150     std::condition_variable progress_cv;
151     std::condition_variable release_cv;
152     bool release_parents = false;
153     std::int32_t blocked_parent_count = 0;
154     std::int32_t child_submitted_count = 0;
155     std::int32_t child_started_count = 0;
156     std::int32_t child_completed_count = 0;
157

```

```

158     for (std::int32_t parent = 0; parent < worker_count; ++parent) {
159         const bool accepted = runtime.submit([&runtime,
160                                             &start_gate,
161                                             &progress_mutex,
162                                             &progress_cv,
163                                             &release_cv,
164                                             &release_parents,
165                                             &blocked_parent_count,
166                                             &child_submitted_count,
167                                             &child_started_count,
168                                             &child_completed_count]() {
169             start_gate.wait();
170             auto promise = std::make_shared<std::promise<std::int32_t>>();
171             std::future<std::int32_t> future = promise->get_future();
172             const bool child_accepted = runtime.submit([promise,
173                                                         &progress_mutex,
174                                                         &progress_cv,
175                                                         &child_started_count,
176                                                         &child_completed_count]
177             ↪ )() {
178                 {
179                     std::lock_guard<std::mutex> lock(progress_mutex);
180                     ++child_started_count;
181                     progress_cv.notify_all();
182                 }
183                 promise->set_value(COMPSYS_TASK_RUNTIME_CHILD_VALUE);
184                 {
185                     std::lock_guard<std::mutex> lock(progress_mutex);
186                     ++child_completed_count;
187                     progress_cv.notify_all();
188                 }
189             });
190             if (!child_accepted) {
191                 throw std::runtime_error("child task should be accepted");
192             }
193
194             if (future.wait_for(std::chrono::milliseconds(0)) !=
195                 std::future_status::ready) {
196                 std::unique_lock<std::mutex> lock(progress_mutex);
197                 ++child_submitted_count;
198                 ++blocked_parent_count;
199                 progress_cv.notify_all();
200                 release_cv.wait(lock, [&release_parents]() {
201                     return release_parents;
202                 });
203             });
204             if (!accepted) {
205                 throw std::runtime_error("parent task should be accepted");
206             }
207         }
208         start_gate.count_down();

```

```

209
210 bool reached_stall = false;
211 std::int32_t child_submitted_count_at_stall = 0;
212 std::int32_t queued_child_count_at_stall = 0;
213 std::int32_t child_started_before_release = 0;
214 {
215     std::unique_lock<std::mutex> lock(progress_mutex);
216     reached_stall = progress_cv.wait_for(
217         lock,
218         std::chrono::milliseconds(COMPSYS_TASK_RUNTIME_STALL_TIMEOUT_MS),
219         [&blocked_parent_count, worker_count]() {
220             return blocked_parent_count == worker_count;
221         });
222     child_submitted_count_at_stall = child_submitted_count;
223     queued_child_count_at_stall = runtime.queued_task_count();
224     child_started_before_release = child_started_count;
225     release_parents = true;
226 }
227 release_cv.notify_all();
228
229 runtime.wait_idle();
230 const std::int32_t final_child_completed = [&]() {
231     std::lock_guard<std::mutex> lock(progress_mutex);
232     return child_completed_count;
233 }();
234 runtime.shutdown();
235
236 SamePoolFutureWaitReport report;
237 report.worker_count = worker_count;
238 report.parent_count = worker_count;
239 report.blocked_parent_count = blocked_parent_count;
240 report.child_submitted_count_at_stall = child_submitted_count_at_stall;
241 report.queued_child_count_at_stall = queued_child_count_at_stall;
242 report.child_started_before_release = child_started_before_release;
243 report.child_completed_count = final_child_completed;
244 report.reached_stall = reached_stall;
245 report.starved_without_extra_worker =
246     reached_stall &&
247     blocked_parent_count == worker_count &&
248     child_submitted_count_at_stall == worker_count &&
249     child_started_before_release == 0;
250 return report;
251 }
252
253 ContinuationRuntimeReport run_continuation_runtime_probe(std::int32_t ↵
    ↵ worker_count,
254                                                         std::int32_t ↵
    ↵ root_task_count) {
255     require_positive(worker_count, "worker_count");
256     require_positive(root_task_count, "root_task_count");
257
258     TaskRuntime runtime(worker_count);

```

```

259     std::atomic<std::int32_t> completed_roots{0};
260     std::atomic<std::int32_t> completed_continuations{0};
261     std::atomic<std::int32_t> rejected_continuations{0};
262     std::atomic<std::int64_t> final_sum{0};
263
264     for (std::int32_t task_id = 0; task_id < root_task_count; ++task_id) {
265         const std::int32_t value = task_id + 1;
266         const bool accepted = runtime.submit([&runtime,
267                                             &completed_roots,
268                                             &completed_continuations,
269                                             &rejected_continuations,
270                                             &final_sum,
271                                             value]() {
272             completed_roots.fetch_add(1, std::memory_order_relaxed);
273             const bool continuation_accepted = runtime.submit([&
274 ↪ completed_continuations,
275                                             &final_sum,
276                                             value]() {
277 ↪ completed_continuations.fetch_add(1, std::memory_order_relaxed)↪
278 ↪ ;
279             final_sum.fetch_add(value, std::memory_order_relaxed);
280             });
281             if (!continuation_accepted) {
282                 rejected_continuations.fetch_add(1, std::memory_order_relaxed);
283             }
284             });
285             if (!accepted) {
286                 throw std::runtime_error("root task should be accepted");
287             }
288         }
289
290     runtime.wait_idle();
291     const TaskRuntime::Report runtime_report = runtime.report();
292     runtime.shutdown();
293
294     ContinuationRuntimeReport report;
295     report.worker_count = worker_count;
296     report.root_task_count = root_task_count;
297     report.completed_roots = completed_roots.load(std::memory_order_relaxed);
298     report.completed_continuations =
299         completed_continuations.load(std::memory_order_relaxed);
300     report.rejected_continuations =
301         rejected_continuations.load(std::memory_order_relaxed);
302     report.max_queue_depth = runtime_report.max_queue_depth;
303     report.final_sum = final_sum.load(std::memory_order_relaxed);
304     report.wait_idle_completed = runtime_report.unfinished_count == 0;
305     return report;
306 }
307
308 void write_task_runtime_report_csv(std::ostream& output,
309                                  const TaskRuntime::Report& report)
310 {

```



```

309     output << "worker_count,active_workers,queued_tasks,max_queue_depth,"
310             "unfinished_count,submitted_count,completed_count,failed_count,"
311             "rejected_count,accepting,stopping\n";
312     output << report.worker_count << ','
313             << report.active_workers << ','
314             << report.queued_tasks << ','
315             << report.max_queue_depth << ','
316             << report.unfinished_count << ','
317             << report.submitted_count << ','
318             << report.completed_count << ','
319             << report.failed_count << ','
320             << report.rejected_count << ','
321             << (report.accepting ? 1 : 0) << ','
322             << (report.stopping ? 1 : 0) << '\n';
323 }
324
325 } // namespace compsys

```

Listing 2.24 src/futex_lab.cpp

```

1  #include <compsys/futex_lab.hpp>
2
3  #include <atomic>
4  #include <cerrno>
5  #include <chrono>
6  #include <cstdint>
7  #include <ctime>
8  #include <latch>
9  #include <stdexcept>
10 #include <string>
11 #include <thread>
12 #include <vector>
13
14 #include <linux/futex.h>
15 #include <pthread.h>
16 #include <signal.h>
17 #include <sys/syscall.h>
18 #include <unistd.h>
19
20 namespace compsys {
21 namespace {
22
23 constexpr std::int32_t COMPSYS_FUTEX_MIN_WORKERS = 1;
24 constexpr std::int32_t COMPSYS_FUTEX_LOCKED = 1;
25 constexpr std::int32_t COMPSYS_FUTEX_CONTENTED = 2;
26 constexpr std::int32_t COMPSYS_FUTEX_READY_VALUE = 1;
27 constexpr std::int32_t COMPSYS_FUTEX_CONTRACT_TIMEOUT_MS = 100;
28 constexpr std::int32_t COMPSYS_FUTEX_SIGNAL_WAIT_MS = 20;
29 constexpr std::int32_t COMPSYS_FUTEX_MUTEX_SLEEP_MS = 10;
30 constexpr std::int32_t COMPSYS_FUTEX_MS_PER_SECOND = 1000;
31 constexpr long COMPSYS_FUTEX_NS_PER_MS = 1000000L;
32 constexpr std::int32_t COMPSYS_FUTEX_WAKE_SPIN_LIMIT = 100000;

```

```

33 constexpr std::int32_t COMPSYS_FUTEX_SIGNAL_RETRY_LIMIT = 100;
34 constexpr std::int32_t COMPSYS_FUTEX_SIGNAL_RETRY_SLEEP_MS = 1;
35 constexpr std::int32_t COMPSYS_FUTEX_STAGE_SIGNAL_WAITING = 1;
36 constexpr std::int32_t COMPSYS_FUTEX_STAGE_SIGNAL_INTERRUPTED = 2;
37 constexpr std::int32_t COMPSYS_FUTEX_STAGE_WAKE_WAITING = 3;
38 constexpr std::int32_t COMPSYS_FUTEX_STAGE_WAKE_DONE = 4;
39 constexpr std::int32_t COMPSYS_FUTEX_STAGE_READY_WAITING = 5;
40 constexpr int COMPSYS_FUTEX_SIGNAL = SIGUSR1;
41
42 void require_positive(std::int32_t value, const char* name) {
43     if (value < COMPSYS_FUTEX_MIN_WORKERS) {
44         throw std::runtime_error(std::string(name) + " must be positive");
45     }
46 }
47
48 void futex_signal_handler(int) {}
49
50 class FutexSignalGuard {
51 public:
52     FutexSignalGuard() {
53         struct sigaction action {};
54         action.sa_handler = futex_signal_handler;
55         sigemptyset(&action.sa_mask);
56         action.sa_flags = 0;
57         if (::sigaction(COMPSYS_FUTEX_SIGNAL, &action, &this->old_action_) !=
↵ 0) {
58             throw std::runtime_error("failed to install futex signal handler");
59         }
60     }
61
62     FutexSignalGuard(const FutexSignalGuard&) = delete;
63     FutexSignalGuard& operator=(const FutexSignalGuard&) = delete;
64
65     ~FutexSignalGuard() {
66         static_cast<void*>(&::sigaction(COMPSYS_FUTEX_SIGNAL,
67                                         &this->old_action_,
68                                         nullptr));
69     }
70
71 private:
72     struct sigaction old_action_ {};
73 };
74
75 [[nodiscard]] int futex_wait_word(std::int32_t* word, std::int32_t expected) {
76     return static_cast<int*>(&::syscall(SYS_futex,
77                                         word,
78                                         FUTEX_WAIT_PRIVATE,
79                                         expected,
80                                         nullptr,
81                                         nullptr,
82                                         0));
83 }

```

```

84
85 [[nodiscard]] int futex_wait_word_timeout(std::int32_t* word,
86                                           std::int32_t expected,
87                                           std::int32_t timeout_ms) {
88     timespec timeout {};
89     timeout.tv_sec = timeout_ms / COMPSYS_FUTEX_MS_PER_SECOND;
90     timeout.tv_nsec =
91         static_cast<long>(timeout_ms % COMPSYS_FUTEX_MS_PER_SECOND) *
92         COMPSYS_FUTEX_NS_PER_MS;
93     return static_cast<int>(::syscall(SYS_futex,
94                                     word,
95                                     FUTEX_WAIT_PRIVATE,
96                                     expected,
97                                     &timeout,
98                                     nullptr,
99                                     0));
100 }
101
102 [[nodiscard]] int futex_wake_word(std::int32_t* word, std::int32_t wake_count) ←
    ↪ {
103     return static_cast<int>(::syscall(SYS_futex,
104                                     word,
105                                     FUTEX_WAKE_PRIVATE,
106                                     wake_count,
107                                     nullptr,
108                                     nullptr,
109                                     0));
110 }
111
112 class FutexMutex {
113 public:
114     void lock() {
115         std::atomic_ref<std::int32_t> state(this->state_word_);
116         std::int32_t expected = 0;
117         if (state.compare_exchange_strong(expected,
118                                           COMPSYS_FUTEX_LOCKED,
119                                           std::memory_order_acquire,
120                                           std::memory_order_relaxed)) {
121             return;
122         }
123
124         this->contended_count_.fetch_add(1, std::memory_order_relaxed);
125         while (state.exchange(COMPSYS_FUTEX_CONTENTED,
126                               std::memory_order_acquire) != 0) {
127             this->wait_count_.fetch_add(1, std::memory_order_relaxed);
128             const int rc =
129                 futex_wait_word(&this->state_word_, COMPSYS_FUTEX_CONTENTED);
130             if (rc == 0 || errno == EAGAIN || errno == EINTR) {
131                 continue;
132             }
133             throw std::runtime_error("futex mutex wait failed");
134         }

```

```

135
136     }
137
138     void unlock() {
139         std::atomic_ref<std::int32_t> state(this->state_word_);
140         const std::int32_t previous =
141             state.exchange(0, std::memory_order_release);
142         if (previous == COMPSYS_FUTEX_CONTENTED) {
143             const int woke = futex_wake_word(&this->state_word_, 1);
144             if (woke < 0) {
145                 throw std::runtime_error("futex mutex wake failed");
146             }
147             this->wake_count_.fetch_add(woke, std::memory_order_relaxed);
148         }
149     }
150
151     [[nodiscard]] std::int32_t contended_count() const {
152         return this->contended_count_.load(std::memory_order_relaxed);
153     }
154
155     [[nodiscard]] std::int32_t wait_count() const {
156         return this->wait_count_.load(std::memory_order_relaxed);
157     }
158
159     [[nodiscard]] std::int32_t wake_count() const {
160         return this->wake_count_.load(std::memory_order_relaxed);
161     }
162
163 private:
164     alignas(std::int32_t) std::int32_t state_word_ = 0;
165     std::atomic<std::int32_t> contended_count_{0};
166     std::atomic<std::int32_t> wait_count_{0};
167     std::atomic<std::int32_t> wake_count_{0};
168 };
169
170 [[nodiscard]] bool wait_for_futex_mutex_waiter(const FutexMutex& mutex) {
171     for (std::int32_t attempt = 0;
172          attempt < COMPSYS_FUTEX_WAKE_SPIN_LIMIT;
173          ++attempt) {
174         if (mutex.wait_count() > 0) {
175             std::this_thread::sleep_for(
176                 std::chrono::milliseconds(COMPSYS_FUTEX_MUTEX_SLEEP_MS));
177             return true;
178         }
179         std::this_thread::yield();
180     }
181     return false;
182 }
183
184 } // namespace
185
186 FutexMutexProbeReport run_futex_mutex_probe(std::int32_t worker_count,

```

```

187                                     std::int32_t increments_per_worker)←
188     ↪ {
189     require_positive(worker_count, "worker_count");
190     require_positive(increments_per_worker, "increments_per_worker");
191
192     FutexMutex mutex;
193     std::int32_t shared_value = 0;
194     mutex.lock();
195
196     std::latch ready_gate(worker_count);
197     std::latch start_gate(1);
198     std::vector<std::thread> workers;
199     workers.reserve(static_cast<std::size_t>(worker_count));
200
201     for (std::int32_t worker = 0; worker < worker_count; ++worker) {
202         workers.emplace_back([worker,
203                               increments_per_worker,
204                               &mutex,
205                               &shared_value,
206                               &ready_gate,
207                               &start_gate]() {
208             ready_gate.count_down();
209             start_gate.wait();
210             for (std::int32_t step = 0; step < increments_per_worker; ++step) {
211                 mutex.lock();
212                 ++shared_value;
213                 mutex.unlock();
214             }
215         });
216     }
217
218     ready_gate.wait();
219     start_gate.count_down();
220
221     const bool saw_waiter = wait_for_futex_mutex_waiter(mutex);
222     mutex.unlock();
223
224     for (std::thread& worker : workers) {
225         worker.join();
226     }
227
228     if (!saw_waiter) {
229         throw std::runtime_error("futex mutex did not observe a waiting thread"←
230     ↪ );
231     }
232
233     FutexMutexProbeReport report;
234     report.worker_count = worker_count;
235     report.increments_per_worker = increments_per_worker;
236     report.final_value = shared_value;
237     report.contended_count = mutex.contended_count();
238     report.wait_count = mutex.wait_count();

```



```

289         break;
290     }
291     if (errno == ETIMEDOUT || errno == EAGAIN) {
292         continue;
293     }
294     throw std::runtime_error("futex signal wait failed");
295 }
296
297 stage.store(COMPSYS_FUTEX_STAGE_WAKE_WAITING,
298             std::memory_order_release);
299 const int wake_rc = futex_wait_word(&spark_word, 0);
300 if (wake_rc < 0 && errno != EAGAIN && errno != EINTR) {
301     throw std::runtime_error("futex wake-count wait failed");
302 }
303 stage.store(COMPSYS_FUTEX_STAGE_WAKE_DONE,
304             std::memory_order_release);
305
306 while (true) {
307     if (ready_ref.load(std::memory_order_acquire) != 0) {
308         observed_ready_count.fetch_add(1, std::memory_order_relaxed);
309         break;
310     }
311
312     stage.store(COMPSYS_FUTEX_STAGE_READY_WAITING,
313                 std::memory_order_release);
314     const int rc = futex_wait_word(&ready_word, 0);
315     if (rc == 0 || errno == EAGAIN) {
316         continue;
317     }
318     if (errno == EINTR) {
319         continue;
320     }
321     throw std::runtime_error("futex contract wait failed");
322 }
323 });
324
325 while (stage.load(std::memory_order_acquire) <
326        COMPSYS_FUTEX_STAGE_SIGNAL_WAITING) {
327     std::this_thread::yield();
328 }
329
330 for (std::int32_t attempt = 0;
331      attempt < COMPSYS_FUTEX_SIGNAL_RETRY_LIMIT &&
332      stage.load(std::memory_order_acquire) <
333      COMPSYS_FUTEX_STAGE_SIGNAL_INTERRUPTED;
334      ++attempt) {
335     const int signal_rc =
336         ::pthread_kill(waiter.native_handle(), COMPSYS_FUTEX_SIGNAL);
337     if (signal_rc != 0) {
338         throw std::runtime_error("failed to signal futex waiter");
339     }
340     std::this_thread::sleep_for(

```

```

341         std::chrono::milliseconds(COMPSYS_FUTEX_SIGNAL_RETRY_SLEEP_MS));
342     }
343
344     if (stage.load(std::memory_order_acquire) <
345         COMPSYS_FUTEX_STAGE_SIGNAL_INTERRUPTED) {
346         throw std::runtime_error("futex signal wait was not interrupted");
347     }
348
349     std::int32_t actual_wake_count = 0;
350     for (std::int32_t attempt = 0;
351         attempt < COMPSYS_FUTEX_WAKE_SPIN_LIMIT && actual_wake_count == 0;
352         ++attempt) {
353         if (stage.load(std::memory_order_acquire) >=
354             COMPSYS_FUTEX_STAGE_WAKE_WAITING) {
355             const int woke = futex_wake_word(&park_word, 1);
356             if (woke < 0) {
357                 throw std::runtime_error("futex wake failed");
358             }
359             actual_wake_count += woke;
360         }
361         std::this_thread::yield();
362     }
363     if (actual_wake_count == 0) {
364         throw std::runtime_error("futex wake did not observe a waiting thread");
365     }
366
367     while (stage.load(std::memory_order_acquire) <
368         COMPSYS_FUTEX_STAGE_READY_WAITING) {
369         std::this_thread::yield();
370     }
371
372     ready_ref.store(COMPSYS_FUTEX_READY_VALUE, std::memory_order_release);
373     for (std::int32_t attempt = 0;
374         attempt < COMPSYS_FUTEX_WAKE_SPIN_LIMIT &&
375         observed_ready_count.load(std::memory_order_relaxed) == 0;
376         ++attempt) {
377         const int woke = futex_wake_word(&ready_word, 1);
378         if (woke < 0) {
379             throw std::runtime_error("futex wake failed");
380         }
381         std::this_thread::yield();
382     }
383     if (observed_ready_count.load(std::memory_order_relaxed) == 0) {
384         throw std::runtime_error("futex waiter did not observe ready state");
385     }
386     wake_count.store(actual_wake_count, std::memory_order_relaxed);
387
388     waiter.join();
389
390     FutexContractProbeReport report;
391     report.waiter_count = 1;

```



```

392     report.wake_count = wake_count.load(std::memory_order_relaxed);
393     report.eagain_count = eagain_count.load(std::memory_order_relaxed);
394     report.timeout_count = timeout_count.load(std::memory_order_relaxed);
395     report.eintr_count = eintr_count.load(std::memory_order_relaxed);
396     report.observed_ready_count =
397         observed_ready_count.load(std::memory_order_relaxed);
398     report.wait_released_on_ready =
399         report.observed_ready_count == 1;
400     return report;
401 }
402
403 } // namespace compsys

```

Listing 2.25 compute systems src/main.cpp

```

1  #include <compsys/parallel_reduce.hpp>
2
3  #include <cstdint>
4  #include <cstdlib>
5  #include <iostream>
6  #include <vector>
7
8  namespace {
9
10 constexpr std::int32_t COMPSYS_DEMO_SIZE = 1024;
11 constexpr std::int32_t COMPSYS_DEMO_WORKERS = 4;
12
13 std::vector<std::int32_t> make_demo_values() {
14     std::vector<std::int32_t> values(static_cast<std::size_t>(COMPSYS_DEMO_SIZE↵
↵ ));
15     for (std::int32_t i = 0; i < COMPSYS_DEMO_SIZE; ++i) {
16         values[static_cast<std::size_t>(i)] = i % 17;
17     }
18     return values;
19 }
20
21 } // namespace
22
23 int main() {
24     const std::vector<std::int32_t> values = make_demo_values();
25     const std::int64_t sequential = compsys::sequential_sum(values);
26     const std::int64_t parallel = compsys::parallel_sum(values, ↵
↵ COMPSYS_DEMO_WORKERS);
27
28     compsys::AtomicCounter counter;
29     counter.add(sequential);
30     counter.add(parallel);
31
32     std::cout << "sequential_sum " << sequential << "\n";
33     std::cout << "parallel_sum " << parallel << "\n";
34     std::cout << "counter " << counter.value() << "\n";
35     return EXIT_SUCCESS;

```

```
36 }
```

Listing 2.26 src/chapter_models_probe.cpp

```
1 #include <compsys/chapter_models.hpp>
2
3 #include <array>
4 #include <cstdint>
5 #include <cstdlib>
6 #include <exception>
7 #include <iostream>
8
9 namespace {
10
11 constexpr std::int32_t COMPSYS_CHAPTER_MODEL_COUNT = 21;
12 constexpr std::int32_t COMPSYS_BRANCH_THRESHOLD = 5;
13 constexpr std::int32_t COMPSYS_CACHE_LINE_BYTES = 64;
14 constexpr std::int32_t COMPSYS_PAGE_BYTES = 4096;
15 constexpr std::int32_t COMPSYS_TLB_ENTRIES = 2;
16 constexpr double COMPSYS_ROOFLINE_FLOPS = 200.0;
17 constexpr double COMPSYS_ROOFLINE_BYTES = 100.0;
18 constexpr double COMPSYS_ROOFLINE_PEAK_GFLOPS = 10.0;
19 constexpr double COMPSYS_ROOFLINE_BANDWIDTH_GBPS = 20.0;
20 constexpr std::int32_t COMPSYS_LAYOUT_FIELDS = 4;
21 constexpr std::int32_t COMPSYS_LAYOUT_HOT_FIELDS = 1;
22 constexpr std::int32_t COMPSYS_FIELD_BYTES = 8;
23 constexpr std::int32_t COMPSYS_SIMD_ELEMENTS = 10;
24 constexpr std::int32_t COMPSYS_SIMD_LANES = 4;
25 constexpr std::int32_t COMPSYS_SCHEDULING_WORKERS = 2;
26 constexpr std::int32_t COMPSYS_BACKPRESSURE_TICKS = 3;
27 constexpr std::int32_t COMPSYS_BACKPRESSURE_PRODUCE_RATE = 4;
28 constexpr std::int32_t COMPSYS_BACKPRESSURE_CONSUME_RATE = 1;
29 constexpr std::int32_t COMPSYS_BACKPRESSURE_CAPACITY = 5;
30 constexpr std::int32_t COMPSYS_FINAL_VERSION = 3;
31 constexpr std::int32_t COMPSYS_RING_CAPACITY = 2;
32 constexpr std::int32_t COMPSYS_PARTITION_THRESHOLD = 2;
33 constexpr std::int32_t COMPSYS_IO_REQUESTS = 5;
34 constexpr std::int32_t COMPSYS_IO_QUEUE_DEPTH = 2;
35 constexpr std::int32_t COMPSYS_IO_COMPLETIONS_PER_TICK = 1;
36 constexpr std::int32_t COMPSYS_CURRENT_GENERATION = 2;
37 constexpr std::int32_t COMPSYS_REPLICA_COUNT = 3;
38 constexpr std::int32_t COMPSYS_INPUT_RECORDS = 100;
39 constexpr std::int32_t COMPSYS_CHECKPOINT_INTERVAL = 25;
40 constexpr std::int32_t COMPSYS_FAILED_AFTER_RECORD = 63;
41
42 } // namespace
43
44 int main() {
45     try {
46         const std::array<std::int32_t, 3> records{4, -1, 7};
47         const std::array<std::int32_t, 4> branch_values{1, 9, 2, 10};
48         const std::array<std::uint64_t, 5> offsets{0, 64, 4096, 8192, 0};
```

```

49  const std::array<std::int32_t, 3> worker_nodes{0, 1, 1};
50  const std::array<std::int32_t, 3> memory_nodes{0, 0, 1};
51  const std::array<std::int32_t, 4> kernel_values{1, 2, 3, 4};
52  const std::array<std::int32_t, 4> task_costs{5, 1, 4, 2};
53  const std::array<bool, 3> ready_observations{false, false, true};
54  const std::array<std::int32_t, 4> observed_versions{1, 2, 2, 3};
55  const std::array<std::int32_t, 6> ring_operations{1, 1, 1, -1, -1, -1};
56  const std::array<std::int32_t, 3> retired_nodes{1, 2, 3};
57  const std::array<std::int32_t, 1> protected_nodes{2};
58  const std::array<std::int32_t, 3> scan_values{1, 2, 3};
59  const std::array<std::int32_t, 3> worker_queues{6, 0, 0};
60  const std::array<std::int32_t, 4> task_ids{1, 1, 2, 3};
61  const std::array<std::int32_t, 4> generations{2, 2, 1, 2};
62  const std::array<bool, 3> acknowledgements{true, true, false};
63
64  const compsys::Chapter01PipelineReport ch01 =
65      compsys::model_ch01_input_pipeline(records);
66  const compsys::Chapter02BranchReport ch02 =
67      compsys::model_ch02_branch_predictor(branch_values,
68                                          COMPSYS_BRANCH_THRESHOLD);
69  const compsys::Chapter03MemoryAccessReport ch03 =
70      compsys::model_ch03_cache_tlb(offsets,
71                                    COMPSYS_CACHE_LINE_BYTES,
72                                    COMPSYS_PAGE_BYTES,
73                                    COMPSYS_TLB_ENTRIES);
74  const compsys::Chapter04NumaReport ch04 =
75      compsys::model_ch04_numa_coherence(worker_nodes, memory_nodes);
76  const compsys::Chapter05RooflineReport ch05 =
77      compsys::model_ch05_roofline(COMPSYS_ROOFLINE_FLOPS,
78                                   COMPSYS_ROOFLINE_BYTES,
79                                   COMPSYS_ROOFLINE_PEAK_GFLOPS,
80                                   COMPSYS_ROOFLINE_BANDWIDTH_GBPS);
81  const compsys::Chapter05bPmuSimdReport ch05b =
82      compsys::model_ch05b_pmu_simd_evidence(80, 20, true);
83  const compsys::Chapter06LayoutReport ch06 =
84      compsys::model_ch06_layout_locality(COMPSYS_SIMD_ELEMENTS,
85                                           COMPSYS_LAYOUT_FIELDS,
86                                           COMPSYS_LAYOUT_HOT_FIELDS,
87                                           COMPSYS_FIELD_BYTES);
88  const compsys::Chapter07SimdReport ch07 =
89      compsys::model_ch07_simd_ilp(COMPSYS_SIMD_ELEMENTS,
90                                   COMPSYS_SIMD_LANES);
91  const compsys::Chapter08KernelReport ch08 =
92      compsys::model_ch08_map_filter_reduce(kernel_values);
93  const compsys::Chapter09SchedulingReport ch09 =
94      compsys::model_ch09_thread_scheduling(task_costs,
95                                           COMPSYS_SCHEDULING_WORKERS);
96  const compsys::Chapter09bLinuxWaitReport ch09b =
97      compsys::model_ch09b_linux_wait_wake(ready_observations);
98  const compsys::Chapter10BackpressureReport ch10 =
99      compsys::model_ch10_sync_backpressure(
100         COMPSYS_BACKPRESSURE_TICKS,

```

```

101         COMPSYS_BACKPRESSURE_PRODUCE_RATE,
102         COMPSYS_BACKPRESSURE_CONSUME_RATE,
103         COMPSYS_BACKPRESSURE_CAPACITY);
104     const compsys::Chapter11AtomicPublicationReport ch11 =
105         compsys::model_ch11_atomic_publication(observed_versions,
106                                                 COMPSYS_FINAL_VERSION);
107     const compsys::Chapter12LockFreeQueueReport ch12 =
108         compsys::model_ch12_spsc_ring(ring_operations, ←
109     ↪ COMPSYS_RING_CAPACITY);
110     const compsys::Chapter12bReclamationReport ch12b =
111         compsys::model_ch12b_epoch_reclamation(retired_nodes,
112                                                 protected_nodes);
113     const compsys::Chapter13ReduceScanPartitionReport ch13 =
114         compsys::model_ch13_reduce_scan_partition(scan_values,
115     ↪ COMPSYS_PARTITION_THRESHOLD);
116     const compsys::Chapter14WorkStealingReport ch14 =
117         compsys::model_ch14_work_stealing(worker_queues);
118     const compsys::Chapter15AsyncIoReport ch15 =
119         compsys::model_ch15_async_io_backpressure(
120             COMPSYS_IO_REQUESTS,
121             COMPSYS_IO_QUEUE_DEPTH,
122             COMPSYS_IO_COMPLETIONS_PER_TICK);
123     const compsys::Chapter16DistributedModelReport ch16 =
124         compsys::model_ch16_distributed_attempts(task_ids,
125                                                 generations,
126                                                 COMPSYS_CURRENT_GENERATION↪
127     ↪ );
128     const compsys::Chapter17ReplicationReport ch17 =
129         compsys::model_ch17_quorum_commit(COMPSYS_REPLICA_COUNT,
130                                         acknowledgements);
131     const compsys::Chapter18RecoveryReport ch18 =
132         compsys::model_ch18_checkpoint_recovery(
133             COMPSYS_INPUT_RECORDS,
134             COMPSYS_CHECKPOINT_INTERVAL,
135             COMPSYS_FAILED_AFTER_RECORD);
136
137     std::cout << "chapter_models," << COMPSYS_CHAPTER_MODEL_COUNT << '\n';
138     std::cout << "ch01_pipeline," << ch01.records_total << ','
139     << ch01.accepted << ',' << ch01.rejected << '\n';
140     std::cout << "ch02_branch," << ch02.actual_taken << ','
141     << ch02.mispredictions << '\n';
142     std::cout << "ch03_cache_tlb," << ch03.unique_cache_lines << ','
143     << ch03.unique_pages << ',' << ch03.tlb_misses << '\n';
144     std::cout << "ch04_numa," << ch04.local_accesses << ','
145     << ch04.remote_accesses << ',' << ch04.coherence_transfers <<↪
146     ↪ '\n';
147     std::cout << "ch05_roofline," << ch05.arithmetic_intensity << ','
148     << ch05.attainable_gflops << ',' << (ch05.memory_bound ? 1 : ↪
149     ↪ 0)
150     << '\n';
151     std::cout << "ch05b_pmu_simd," << ch05b.retired_instructions << ','

```

```

148         << ch05b.vector_fraction << ','
149         << (ch05b.evidence_complete ? 1 : 0) << '\n';
150     std::cout << "ch06_layout," << ch06.aos_bytes_touched << ','
151     << ch06.soa_bytes_touched << ',' << ch06.saved_bytes << '\n';
152     std::cout << "ch07_simd," << ch07.vector_iterations << ','
153     << ch07.scalar_tail << '\n';
154     std::cout << "ch08_kernel," << ch08.filtered_count << ','
155     << ch08.reduced_sum << '\n';
156     std::cout << "ch09_scheduling," << ch09.task_count << ','
157     << ch09.imbalance << '\n';
158     std::cout << "ch09b_wait," << ch09b.spurious_wakeups << ','
159     << (ch09b.completed ? 1 : 0) << '\n';
160     std::cout << "ch10_backpressure," << ch10.produced << ','
161     << ch10.backpressure_events << ',' << ch10.max_depth << '\n';
162     std::cout << "ch11_atomic," << ch11.stale_reads << ','
163     << (ch11.monotonic_observations ? 1 : 0) << '\n';
164     std::cout << "ch12_ring," << ch12.pushed << ',' << ch12.popped << ','
165     << ch12.rejected_full << ',' << ch12.empty_pops << '\n';
166     std::cout << "ch12b_reclamation," << ch12b.reclaimed_nodes << ','
167     << ch12b.deferred_nodes << '\n';
168     std::cout << "ch13_reduce_scan_partition," << ch13.reduce_sum << ','
169     << ch13.last_prefix << ',' << ch13.partition_true_count << '\n';
170     std::cout << "ch14_work_stealing," << ch14.stolen_tasks << ','
171     << ch14.completed_tasks << '\n';
172     std::cout << "ch15_async_io," << ch15.submitted << ','
173     << ch15.completed << ',' << ch15.backpressure_events << '\n';
174     std::cout << "ch16_distributed," << ch16.logical_tasks << ','
175     << ch16.duplicate_attempts << ','
176     << ch16.stale_generation_attempts << '\n';
177     std::cout << "ch17_quorum," << ch17.quorum_size << ','
178     << (ch17.committed ? 1 : 0) << '\n';
179     std::cout << "ch18_recovery," << ch18.checkpoints << ','
180     << ch18.replay_from_record << ',' << ch18.replay_records << '\n';
181     return EXIT_SUCCESS;
182 } catch (const std::exception& error) {
183     std::cerr << "compsys chapter models probe failure: " << error.what() <<
184     << '\n';
185     return EXIT_FAILURE;
186 }

```

Listing 2.27 src/wait_channels_probe.cpp

```

1 #include <compsys/wait_channels.hpp>
2
3 #include <cstdlib>
4 #include <iomanip>
5 #include <iostream>
6
7 int main() {

```

```

8     try {
9         const compsys::EventFdEpollReport report = compsys::↵
↵ run_eventfd_epoll_contract();
10        std::cout << std::fixed << std::setprecision(0);
11        std::cout << "contract,"
12                    << report.wake_write_count << ','
13                    << report.first_counter_read << ','
14                    << report.second_counter_read << ','
15                    << report.epoll_ready_observations << ','
16                    << report.epoll_empty_observations << ','
17                    << report.queue_drain_count << ','
18                    << report.semaphore_read_count << ','
19                    << (report.queue_fifo_ok ? 1 : 0) << ','
20                    << (report.semaphore_empty_after_reads ? 1 : 0) << '\n';
21        return EXIT_SUCCESS;
22    } catch (const std::exception& error) {
23        std::cerr << "compsys wait channel probe failure: " << error.what() << ↵
↵ '\n';
24        return EXIT_FAILURE;
25    }
26 }

```

Listing 2.28 src/task_runtime_probe.cpp

```

1  #include <compsys/task_runtime.hpp>
2
3  #include <cstdlib>
4  #include <exception>
5  #include <iostream>
6
7  namespace {
8
9  constexpr std::int32_t COMPSYS_TASK_PROBE_WORKERS = 3;
10 constexpr std::int32_t COMPSYS_TASK_PROBE_ROOTS = 5;
11
12 } // namespace
13
14 int main() {
15     try {
16         const compsys::SamePoolFutureWaitReport same_pool =
17         compsys::run_same_pool_future_wait_probe(COMPSYS_TASK_PROBE_WORKERS,↵
↵ );
18         const compsys::ContinuationRuntimeReport continuation =
19         compsys::run_continuation_runtime_probe(COMPSYS_TASK_PROBE_WORKERS,
20                                                 COMPSYS_TASK_PROBE_ROOTS);
21
22         std::cout << "task_runtime,"
23                    << same_pool.worker_count << ','
24                    << same_pool.blocked_parent_count << ','
25                    << same_pool.child_submitted_count_at_stall << ','
26                    << same_pool.child_started_before_release << ','
27                    << (same_pool.starved_without_extra_worker ? 1 : 0) << ','

```

```

28         << continuation.completed_roots << ','
29         << continuation.completed_continuations << ','
30         << continuation.final_sum << ','
31         << continuation.max_queue_depth << ','
32         << (continuation.wait_idle_completed ? 1 : 0) << '\n';
33     return EXIT_SUCCESS;
34 } catch (const std::exception& error) {
35     std::cerr << "compsys task runtime probe failure: " << error.what() << "\n";
36     return EXIT_FAILURE;
37 }
38 }

```

Listing 2.29 src/futex_lab_probe.cpp

```

1 #include <compsys/futex_lab.hpp>
2
3 #include <cstdlib>
4 #include <exception>
5 #include <iostream>
6 #include <stdexcept>
7
8 namespace {
9
10 constexpr std::int32_t COMPSYS_FUTEX_MUTEX_WORKERS = 4;
11 constexpr std::int32_t COMPSYS_FUTEX_MUTEX_INCREMENTS = 50;
12
13 void require(bool condition, const char* message) {
14     if (!condition) {
15         throw std::runtime_error(message);
16     }
17 }
18
19 } // namespace
20
21 int main() {
22     try {
23         const compsys::FutexMutexProbeReport mutex_report =
24             compsys::run_futex_mutex_probe(COMPSYS_FUTEX_MUTEX_WORKERS,
25                                             COMPSYS_FUTEX_MUTEX_INCREMENTS);
26         const compsys::FutexContractProbeReport contract_report =
27             compsys::run_futex_contract_probe();
28         const std::int32_t expected_final_value =
29             COMPSYS_FUTEX_MUTEX_WORKERS * COMPSYS_FUTEX_MUTEX_INCREMENTS;
30
31         require(mutex_report.final_value == expected_final_value,
32                "futex mutex final value mismatch");
33         require(mutex_report.contended_count > 0,
34                "futex mutex did not observe contention");
35         require(mutex_report.wait_count > 0,
36                "futex mutex did not enter wait path");
37         require(mutex_report.wake_count > 0,

```



```

38         "futex mutex did not wake a waiter");
39     require(contract_report.waiter_count == 1,
40         "futex contract waiter count mismatch");
41     require(contract_report.wake_count > 0,
42         "futex contract wake count mismatch");
43     require(contract_report.eagain_count == 1,
44         "futex contract EAGAIN count mismatch");
45     require(contract_report.timeout_count == 1,
46         "futex contract timeout count mismatch");
47     require(contract_report.eintr_count == 1,
48         "futex contract EINTR count mismatch");
49     require(contract_report.observed_ready_count == 1,
50         "futex contract ready observation mismatch");
51     require(contract_report.wait_released_on_ready,
52         "futex contract ready release mismatch");
53
54     std::cout << "futex_mutex,"
55         << mutex_report.worker_count << ','
56         << mutex_report.increments_per_worker << ','
57         << mutex_report.final_value << ','
58         << mutex_report.contended_count << ','
59         << mutex_report.wait_count << ','
60         << mutex_report.wake_count << '\n';
61     std::cout << "futex_contract,"
62         << contract_report.waiter_count << ','
63         << contract_report.wake_count << ','
64         << contract_report.eagain_count << ','
65         << contract_report.timeout_count << ','
66         << contract_report.eintr_count << ','
67         << contract_report.observed_ready_count << ','
68         << (contract_report.wait_released_on_ready ? 1 : 0) << '\n';
69     return EXIT_SUCCESS;
70 } catch (const std::exception& error) {
71     std::cerr << "compsys futex lab probe failure: " << error.what() << '\n'
72     ↪ ;
73     return EXIT_FAILURE;
74 }

```

四、测试和 Benchmark

这一组文件是进入正文结论的门槛。测试固定 reference pipeline 和 compute systems 的语义边界；benchmark 只提供性能入口，不替代正确性测试。新增优化前，先跑测试，再跑 benchmark smoke，最后才写性能报告。

Listing 2.30 reference pipeline tests/reference_pipeline_tests.cpp

```

1 #include <cse/reference_pipeline.hpp>
2
3 #include <gmock/gmock.h>
4 #include <gtest/gtest.h>
5

```



```

6 #include <stdexcept>
7 #include <string>
8
9 namespace {
10
11 using ::testing::HasSubstr;
12
13 TEST(CseReferencePipelineTest, NormalInputCountsAndChecksumAreStable)
14 {
15     const cse::InputCase input{
16         "normal_5_lines",
17         1,
18         "view,1\nclick,1\nview,2\nlogin,1\nlogout,1\n",
19     };
20
21     const cse::ReferenceResult first = cse::run_reference(input);
22     const cse::ReferenceResult second = cse::run_reference(input);
23
24     EXPECT_EQ(first.records_total, 5U);
25     EXPECT_EQ(first.accepted, 5U);
26     EXPECT_TRUE(first.diagnostics.empty());
27     EXPECT_EQ(first.counts.at("view"), 2U);
28     EXPECT_EQ(first.counts.at("click"), 1U);
29     EXPECT_NE(first.checksum, 0U);
30     EXPECT_EQ(first.checksum, second.checksum);
31 }
32
33 TEST(CseReferencePipelineTest, BadLinesReturnStructuredDiagnostics)
34 {
35     const cse::InputCase input{
36         "bad_lines",
37         7,
38         "view,1\nmissing\nclick,1,extra\n,2\nlogin,abc",
39     };
40
41     const cse::ReferenceResult result = cse::run_reference(input);
42
43     EXPECT_EQ(result.records_total, 5U);
44     EXPECT_EQ(result.accepted, 1U);
45     ASSERT_EQ(result.diagnostics.size(), 4U);
46     EXPECT_EQ(result.diagnostics[0].error_kind, "missing_field");
47     EXPECT_EQ(result.diagnostics[0].record.line_number, 2U);
48     EXPECT_EQ(result.diagnostics[1].error_kind, "extra_field");
49     EXPECT_EQ(result.diagnostics[2].error_kind, "empty_event");
50     EXPECT_EQ(result.diagnostics[3].error_kind, "bad_value");
51     EXPECT_EQ(result.diagnostics[3].record.byte_offset, 32U);
52 }
53
54 TEST(CseReferencePipelineTest, ReportAndManifestAreDeterministic)
55 {
56     const cse::InputCase input{
57         "mixed",

```

```

58     3,
59     "view,1\nview,2\nbad\n",
60 };
61
62 const cse::ReferenceResult result = cse::run_reference(input);
63 const std::string report = cse::format_report(result);
64 const cse::ManifestRow row = cse::make_manifest_row(result);
65 const std::string manifest = cse::format_manifest_row(row);
66
67 EXPECT_EQ(row.input_name, "mixed");
68 EXPECT_EQ(row.records_total, 3U);
69 EXPECT_EQ(row.accepted, 2U);
70 EXPECT_EQ(row.rejected, 1U);
71 EXPECT_THAT(report, HasSubstr("count.view=2"));
72 EXPECT_THAT(report, HasSubstr("error.missing_field=1"));
73 EXPECT_THAT(manifest, testing::StartsWith("mixed,3,3,2,1,"));
74 }
75
76 TEST(CseReferencePipelineTest, EmptyInputHasZeroRecordsButStableChecksum)
77 {
78     const cse::InputCase input{"empty", 1, ""};
79     const cse::ReferenceResult result = cse::run_reference(input);
80
81     EXPECT_EQ(result.records_total, 0U);
82     EXPECT_EQ(result.accepted, 0U);
83     EXPECT_TRUE(result.diagnostics.empty());
84     EXPECT_TRUE(result.counts.empty());
85     EXPECT_EQ(result.checksum, cse::run_reference(input).checksum);
86 }
87
88 TEST(CseReferencePipelineTest, DiagnosticDetailNamesTheContract)
89 {
90     const cse::ReferenceResult result =
91         cse::run_reference(cse::InputCase{"bad", 1, "too,few,nope\n"});
92
93     ASSERT_EQ(result.diagnostics.size(), 1U);
94     EXPECT_EQ(result.diagnostics[0].error_kind, "extra_field");
95     EXPECT_THAT(result.diagnostics[0].detail, HasSubstr("exactly two fields"));
96 }
97
98 TEST(CseReferencePipelineTest, ↵
99     ↵ SplitInputMatchesWholeReferenceWhenBoundaryFallsInsideLine)
100 {
101     const cse::InputCase input{
102         "split_middle",
103         9,
104         "view,1\nclick,1\nview,2\nmissing\nlogout,1\n",
105     };
106
107     const cse::ReferenceResult whole = cse::run_reference(input);
108     const cse::ReferenceResult split = cse::run_reference_with_splits(input, ↵
109         ↵ 10);

```

```

108     EXPECT_EQ(split.records_total, whole.records_total);
109     EXPECT_EQ(split.accepted, whole.accepted);
110     EXPECT_EQ(split.counts, whole.counts);
111     ASSERT_EQ(split.diagnostics.size(), whole.diagnostics.size());
112     EXPECT_EQ(split.diagnostics[0].record.line_number, 4U);
113     EXPECT_EQ(split.diagnostics[0].record.byte_offset, 22U);
114     EXPECT_EQ(split.checksum, whole.checksum);
115 }
116
117
118 TEST(CseReferencePipelineTest, SplitInputKeepsLongLineWhole)
119 {
120     const cse::InputCase input{
121         "long_line",
122         2,
123         "very_long_event_name,123456789\nclick,1\n",
124     };
125
126     const std::vector<cse::InputSplit> splits = cse::split_input_case(input, 4)↵
↵ ;
127
128     ASSERT_EQ(splits.size(), 2U);
129     EXPECT_EQ(splits[0].byte_begin, 0U);
130     EXPECT_EQ(splits[0].first_line_number, 1U);
131     EXPECT_EQ(splits[0].text, "very_long_event_name,123456789\n");
132     EXPECT_EQ(splits[1].byte_begin, 31U);
133     EXPECT_EQ(splits[1].first_line_number, 2U);
134
135     const cse::ReferenceResult whole = cse::run_reference(input);
136     const cse::ReferenceResult split = cse::run_reference_with_splits(input, 4)↵
↵ ;
137     EXPECT_EQ(split.records_total, whole.records_total);
138     EXPECT_EQ(split.accepted, whole.accepted);
139     EXPECT_EQ(split.counts, whole.counts);
140     EXPECT_EQ(split.checksum, whole.checksum);
141 }
142
143 TEST(CseReferencePipelineTest, SplitInputRejectsZeroTargetBytes)
144 {
145     const cse::InputCase input{"bad_split", 1, "view,1\n"};
146     EXPECT_THROW((void)cse::split_input_case(input, 0), std::invalid_argument);
147 }
148
149 } // namespace

```

Listing 2.31 bench/reference_pipeline_bench.cpp

```

1 #include <benchmark/benchmark.h>
2
3 #include <cse/reference_pipeline.hpp>
4
5 #include <string>

```

```

6
7 namespace {
8
9 [[nodiscard]] std::string make_input(int records)
10 {
11     std::string text;
12     text.reserve(static_cast<std::size_t>(records) * 12);
13     for (int index = 0; index < records; ++index) {
14         text.append(index % 3 == 0 ? "view,1\n" : "click,1\n");
15     }
16     return text;
17 }
18
19 void bm_run_reference(benchmark::State& state)
20 {
21     const cse::InputCase input{
22         "bench",
23         1,
24         make_input(static_cast<int>(state.range(0))),
25     };
26
27     for (auto _ : state) {
28         benchmark::DoNotOptimize(cse::run_reference(input));
29     }
30     state.SetItemsProcessed(state.iterations() * state.range(0));
31 }
32
33 BENCHMARK(bm_run_reference)->Arg(128)->Arg(4096)->Arg(65536);
34
35 } // namespace
36
37 BENCHMARK_MAIN();

```

Listing 2.32 compute systems tests/compsys_tests.cpp

```

1 #include <compsys/chapter_models.hpp>
2 #include <compsys/futex_lab.hpp>
3 #include <compsys/parallel_reduce.hpp>
4 #include <compsys/sync_primitives.hpp>
5 #include <compsys/task_runtime.hpp>
6 #include <compsys/wait_channels.hpp>
7
8 #include <array>
9 #include <cstdint>
10 #include <cstdlib>
11 #include <cmath>
12 #include <iostream>
13 #include <mutex>
14 #include <optional>
15 #include <sstream>
16 #include <stdexcept>
17 #include <thread>

```

```

18 #include <vector>
19
20 namespace {
21
22 constexpr std::int32_t COMPSYS_TEST_SIZE = 1000;
23 constexpr std::int32_t COMPSYS_TEST_WORKERS = 8;
24 constexpr std::int32_t COMPSYS_TEST_COUNTER_THREADS = 4;
25 constexpr std::int32_t COMPSYS_TEST_COUNTER_STEPS = 250;
26 constexpr std::int32_t COMPSYS_TEST_QUEUE_CAPACITY = 3;
27 constexpr std::int32_t COMPSYS_TEST_WAIT_SPIN_LIMIT = 100000;
28 constexpr std::int32_t COMPSYS_TEST_FUTEX_MUTEX_WORKERS = 4;
29 constexpr std::int32_t COMPSYS_TEST_FUTEX_MUTEX_INCREMENTS = 50;
30 constexpr std::int32_t COMPSYS_TEST_RUNTIME_WORKERS = 3;
31 constexpr std::int32_t COMPSYS_TEST_CONTINUATION_ROOTS = 5;
32 constexpr std::int64_t COMPSYS_TEST_CONTINUATION_SUM = 15;
33 constexpr double COMPSYS_TEST_DOUBLE_TOLERANCE = 0.000001;
34
35 void require_near(double actual, double expected, const char* message) {
36     if (std::fabs(actual - expected) > COMPSYS_TEST_DOUBLE_TOLERANCE) {
37         throw std::runtime_error(message);
38     }
39 }
40
41 void require(bool condition, const char* message) {
42     if (!condition) {
43         throw std::runtime_error(message);
44     }
45 }
46
47 template <typename Predicate>
48 bool wait_until(Predicate predicate) {
49     for (std::int32_t i = 0; i < COMPSYS_TEST_WAIT_SPIN_LIMIT; ++i) {
50         if (predicate()) {
51             return true;
52         }
53         std::this_thread::yield();
54     }
55     return false;
56 }
57
58 std::vector<std::int32_t> make_values() {
59     std::vector<std::int32_t> values(static_cast<std::size_t>(COMPSYS_TEST_SIZE↵
60     ↵ ));
61     for (std::int32_t i = 0; i < COMPSYS_TEST_SIZE; ++i) {
62         values[static_cast<std::size_t>(i)] = (i % 11) - 5;
63     }
64     return values;
65 }
66
67 void test_parallel_sum() {
68     const std::vector<std::int32_t> values = make_values();
69     const std::int64_t sequential = compsys::sequential_sum(values);

```

```

69     const std::int64_t parallel = compsys::parallel_sum(values, ←
    ↪ COMPSYS_TEST_WORKERS);
70     require(sequential == parallel, "parallel sum mismatch");
71 }
72
73 void test_counters() {
74     compsys::MutexCounter mutex_counter;
75     compsys::AtomicCounter atomic_counter;
76     std::vector<std::thread> threads;
77     threads.reserve(static_cast<std::size_t>(COMPSYS_TEST_COUNTER_THREADS));
78
79     for (std::int32_t worker = 0; worker < COMPSYS_TEST_COUNTER_THREADS; ++worker) {
80         threads.emplace_back([&mutex_counter, &atomic_counter]() {
81             for (std::int32_t i = 0; i < COMPSYS_TEST_COUNTER_STEPS; ++i) {
82                 mutex_counter.add(1);
83                 atomic_counter.add(1);
84             }
85         });
86     }
87
88     for (std::thread& thread : threads) {
89         thread.join();
90     }
91
92     const std::int64_t expected =
93         static_cast<std::int64_t>(COMPSYS_TEST_COUNTER_THREADS) *
94         static_cast<std::int64_t>(COMPSYS_TEST_COUNTER_STEPS);
95     require(mutex_counter.value() == expected, "mutex counter mismatch");
96     require(atomic_counter.value() == expected, "atomic counter mismatch");
97 }
98
99 void test_queue() {
100     compsys::BoundedMpmcQueue queue(COMPSYS_TEST_QUEUE_CAPACITY);
101     require(queue.push(7), "first push should succeed");
102     require(queue.push(9), "second push should succeed");
103     require(queue.size() == 2, "queue size mismatch after push");
104
105     const std::optional<std::int32_t> first = queue.try_pop();
106     const std::optional<std::int32_t> second = queue.try_pop();
107     const std::optional<std::int32_t> third = queue.try_pop();
108
109     require(first.has_value() && first.value() == 7, "first queue value ←
    ↪ mismatch");
110     require(second.has_value() && second.value() == 9, "second queue value ←
    ↪ mismatch");
111     require(!third.has_value(), "empty queue should return nullopt");
112
113     const compsys::BoundedMpmcQueue::Report report = queue.report();
114     require(report.push_success == 2, "queue push report mismatch");
115     require(report.pop_success == 2, "queue pop report mismatch");
116     require(report.max_size == 2, "queue max size report mismatch");

```

```

117 }
118
119 void test_queue_close_wakes_consumer() {
120     compsys::BoundedMpmcQueue queue(1);
121     std::optional<std::int32_t> result;
122
123     std::thread consumer([&queue, &result]() {
124         result = queue.pop();
125     });
126
127     const bool saw_wait = wait_until([&queue]() {
128         return queue.report().pop_wait_count == 1;
129     });
130     queue.close();
131     consumer.join();
132
133     require(saw_wait, "consumer should enter pop wait before close");
134     require(!result.has_value(), "closed empty queue should end pop");
135     const compsys::BoundedMpmcQueue::Report report = queue.report();
136     require(report.close_count == 1, "close count mismatch");
137     require(report.pop_closed == 1, "closed pop report mismatch");
138     require(report.pop_wait_count == 1, "closed pop wait count mismatch");
139     require(report.pop_wait_ns > 0, "closed pop wait ns should be recorded");
140     require(queue.closed(), "queue should be closed");
141 }
142
143 void test_queue_close_wakes_blocked_producer() {
144     compsys::BoundedMpmcQueue queue(1);
145     require(queue.push(11), "initial push should succeed");
146
147     bool second_push_result = true;
148     std::thread producer([&queue, &second_push_result]() {
149         second_push_result = queue.push(22);
150     });
151
152     const bool saw_wait = wait_until([&queue]() {
153         return queue.report().push_wait_count == 1;
154     });
155     queue.close();
156     producer.join();
157
158     require(saw_wait, "producer should enter push wait before close");
159     require(!second_push_result, "push waiting on full queue should stop after ←
    ↪ close");
160     const compsys::BoundedMpmcQueue::Report report = queue.report();
161     require(report.push_success == 1, "push success report mismatch after close ←
    ↪ ");
162     require(report.push_closed == 1, "push closed report mismatch");
163     require(report.push_wait_count == 1, "push wait count mismatch");
164     require(report.push_wait_ns > 0, "push wait ns should be recorded");
165     require(report.close_count == 1, "close report mismatch for blocked ←
    ↪ producer");

```

```

166 }
167
168 void test_queue_multi_producer_consumer_drain() {
169     constexpr std::int32_t PRODUCERS = 3;
170     constexpr std::int32_t VALUES_PER_PRODUCER = 20;
171     constexpr std::int32_t CONSUMERS = 2;
172     constexpr std::int32_t EXPECTED_TOTAL = PRODUCERS * VALUES_PER_PRODUCER;
173
174     compsys::BoundedMpmcQueue queue(1);
175     std::vector<std::thread> producers;
176     std::vector<std::thread> consumers;
177     std::vector<std::int32_t> consumed;
178     std::mutex consumed_mutex;
179
180     producers.reserve(static_cast<std::size_t>(PRODUCERS));
181     consumers.reserve(static_cast<std::size_t>(CONSUMERS));
182
183     for (std::int32_t producer = 0; producer < PRODUCERS; ++producer) {
184         producers.emplace_back([producer, &queue]() {
185             for (std::int32_t i = 0; i < VALUES_PER_PRODUCER; ++i) {
186                 const std::int32_t value = producer * VALUES_PER_PRODUCER + i;
187                 if (!queue.push(value)) {
188                     return;
189                 }
190             }
191         });
192     }
193
194     for (std::int32_t consumer = 0; consumer < CONSUMERS; ++consumer) {
195         consumers.emplace_back([&queue, &consumed, &consumed_mutex]() {
196             while (true) {
197                 std::optional<std::int32_t> value = queue.pop();
198                 if (!value.has_value()) {
199                     return;
200                 }
201                 std::lock_guard<std::mutex> lock(consumed_mutex);
202                 consumed.push_back(value.value());
203             }
204         });
205     }
206
207     for (std::thread& producer : producers) {
208         producer.join();
209     }
210     queue.close();
211     for (std::thread& consumer : consumers) {
212         consumer.join();
213     }
214
215     require(static_cast<std::int32_t>(consumed.size()) == EXPECTED_TOTAL,
216             "drain queue consumed count mismatch");
217

```



```

218     std::vector<bool> seen(static_cast<std::size_t>(EXPECTED_TOTAL), false);
219     for (const std::int32_t value : consumed) {
220         require(value >= 0 && value < EXPECTED_TOTAL, "drain queue value out of ↵
↵ range");
221         const std::size_t index = static_cast<std::size_t>(value);
222         require(!seen[index], "drain queue duplicate value");
223         seen[index] = true;
224     }
225
226     const compsys::BoundedMpmcQueue::Report report = queue.report();
227     require(report.push_success == EXPECTED_TOTAL, "drain queue push report ↵
↵ mismatch");
228     require(report.pop_success == EXPECTED_TOTAL, "drain queue pop report ↵
↵ mismatch");
229     require(report.max_size == 1, "capacity one queue should report max size ↵
↵ one");
230 }
231
232 void test_invalid_queue_capacity() {
233     bool threw = false;
234     try {
235         compsys::BoundedMpmcQueue queue(0);
236     } catch (const std::runtime_error&) {
237         threw = true;
238     }
239     require(threw, "invalid queue capacity should throw");
240 }
241
242 void test_permit_limiter() {
243     constexpr std::int32_t TASK_COUNT = 16;
244     constexpr std::int32_t PERMIT_COUNT = 3;
245
246     const compsys::PermitLimiterReport report =
247         compsys::run_permit_limiter(TASK_COUNT, PERMIT_COUNT);
248
249     require(report.task_count == TASK_COUNT, "permit limiter task count ↵
↵ mismatch");
250     require(report.permit_count == PERMIT_COUNT, "permit limiter permit count ↵
↵ mismatch");
251     require(report.entered_count == TASK_COUNT, "permit limiter entered count ↵
↵ mismatch");
252     require(report.max_in_flight <= PERMIT_COUNT,
253             "permit limiter exceeded permit count");
254     require(report.max_in_flight == PERMIT_COUNT,
255             "permit limiter should observe the full permit window");
256 }
257
258 void test_phase_barrier() {
259     constexpr std::int32_t WORKER_COUNT = 4;
260     constexpr std::int32_t PHASE_COUNT = 5;
261
262     const compsys::PhaseBarrierReport report =

```

```

263     compsys::run_phase_barrier(WORKER_COUNT, PHASE_COUNT);
264
265     require(report.worker_count == WORKER_COUNT, "barrier worker count mismatch"↵
↵ );
266     require(report.phase_count == PHASE_COUNT, "barrier phase count mismatch");
267     require(report.completed_phases == PHASE_COUNT, "barrier completion count ↵
↵ mismatch");
268     require(report.arrival_count == WORKER_COUNT * PHASE_COUNT,
269             "barrier arrival count mismatch");
270 }
271
272 void test_future_result_paths() {
273     const compsys::FutureResultReport report = compsys::run_future_result_paths↵
↵ ();
274
275     require(report.value_ready, "future value path failed");
276     require(report.value == 42, "future value mismatch");
277     require(report.exception_ready, "future exception path failed");
278     require(report.exception_message == "future path failed",
279             "future exception message mismatch");
280     require(report.broken_promise_seen, "broken promise path failed");
281 }
282
283 void test_atomic_wait_version_counter() {
284     constexpr std::int32_t PUBLISH_COUNT = 6;
285     const compsys::AtomicWaitReport report =
286         compsys::run_atomic_wait_version_counter(PUBLISH_COUNT);
287
288     require(report.final_version == PUBLISH_COUNT, "atomic wait final version ↵
↵ mismatch");
289     require(report.wake_count >= 1, "atomic wait did not observe any wake");
290     require(report.observed_sum >= PUBLISH_COUNT,
291             "atomic wait observed sum should include final version");
292 }
293
294 void test_eventfd_epoll_contract() {
295     const compsys::EventFdEpollReport report = compsys::↵
↵ run_eventfd_epoll_contract();
296
297     require(report.wake_write_count == 2, "eventfd wake write count mismatch");
298     require(report.first_counter_read == 1, "eventfd first counter read ↵
↵ mismatch");
299     require(report.second_counter_read == 1, "eventfd second counter read ↵
↵ mismatch");
300     require(report.epoll_ready_observations == 2,
301             "eventfd epoll ready observation mismatch");
302     require(report.epoll_empty_observations == 2,
303             "eventfd epoll empty observation mismatch");
304     require(report.queue_drain_count == 3, "eventfd queue drain count mismatch"↵
↵ );
305     require(report.semaphore_read_count == 3, "eventfd semaphore read count ↵
↵ mismatch");

```

```

306     require(report.queue_fifo_ok, "eventfd queue FIFO contract failed");
307     require(report.semaphore_empty_after_reads, "eventfd semaphore empty ↵
↵ contract failed");
308 }
309
310 void test_futex_lab_probe() {
311     const compsys::FutexMutexProbeReport mutex_report =
312         compsys::run_futex_mutex_probe(COMPSYS_TEST_FUTEX_MUTEX_WORKERS,
313                                         COMPSYS_TEST_FUTEX_MUTEX_INCREMENTS);
314     const std::int32_t expected_final_value =
315         COMPSYS_TEST_FUTEX_MUTEX_WORKERS * COMPSYS_TEST_FUTEX_MUTEX_INCREMENTS;
316
317     require(mutex_report.worker_count == COMPSYS_TEST_FUTEX_MUTEX_WORKERS,
318             "futex mutex worker count mismatch");
319     require(mutex_report.increments_per_worker ==
320             COMPSYS_TEST_FUTEX_MUTEX_INCREMENTS,
321             "futex mutex increment count mismatch");
322     require(mutex_report.final_value == expected_final_value,
323             "futex mutex final value mismatch");
324     require(mutex_report.contended_count > 0,
325             "futex mutex should observe contention");
326     require(mutex_report.wait_count > 0,
327             "futex mutex should enter the wait slow path");
328     require(mutex_report.wake_count > 0,
329             "futex mutex should wake a waiting thread");
330
331     const compsys::FutexContractProbeReport contract_report =
332         compsys::run_futex_contract_probe();
333
334     require(contract_report.waiter_count == 1,
335             "futex contract waiter count mismatch");
336     require(contract_report.wake_count > 0,
337             "futex contract should wake a waiter");
338     require(contract_report.eagain_count == 1,
339             "futex contract EAGAIN path mismatch");
340     require(contract_report.timeout_count == 1,
341             "futex contract timeout path mismatch");
342     require(contract_report.eintr_count == 1,
343             "futex contract signal interruption path mismatch");
344     require(contract_report.observed_ready_count == 1,
345             "futex contract ready observation mismatch");
346     require(contract_report.wait_released_on_ready,
347             "futex contract ready release mismatch");
348 }
349
350 void test_same_pool_future_wait_probe() {
351     const compsys::SamePoolFutureWaitReport report =
352         compsys::run_same_pool_future_wait_probe(COMPSYS_TEST_RUNTIME_WORKERS);
353
354     require(report.worker_count == COMPSYS_TEST_RUNTIME_WORKERS,
355             "same-pool probe worker count mismatch");
356     require(report.parent_count == COMPSYS_TEST_RUNTIME_WORKERS,

```

```

357         "same-pool probe parent count mismatch");
358     require(report.blocked_parent_count == COMPSYS_TEST_RUNTIME_WORKERS,
359         "same-pool probe should block every parent");
360     require(report.child_submitted_count_at_stall == ↵
↵ COMPSYS_TEST_RUNTIME_WORKERS,
361         "same-pool probe should submit every child at stall");
362     require(report.queued_child_count_at_stall >= 0,
363         "same-pool queued child count should be observable");
364     require(report.child_started_before_release == 0,
365         "same-pool child should not run while every worker is blocked");
366     require(report.child_completed_count == COMPSYS_TEST_RUNTIME_WORKERS,
367         "same-pool child completion mismatch after release");
368     require(report.reached_stall, "same-pool probe did not reach stall");
369     require(report.starved_without_extra_worker,
370         "same-pool probe should expose starvation risk");
371 }
372
373 void test_continuation_runtime_probe() {
374     const compsys::ContinuationRuntimeReport report =
375         compsys::run_continuation_runtime_probe(COMPSYS_TEST_RUNTIME_WORKERS,
376             COMPSYS_TEST_CONTINUATION_ROOTS↵
↵ );
377
378     require(report.worker_count == COMPSYS_TEST_RUNTIME_WORKERS,
379         "continuation probe worker count mismatch");
380     require(report.root_task_count == COMPSYS_TEST_CONTINUATION_ROOTS,
381         "continuation probe root count mismatch");
382     require(report.completed_roots == COMPSYS_TEST_CONTINUATION_ROOTS,
383         "continuation probe root completion mismatch");
384     require(report.completed_continuations == COMPSYS_TEST_CONTINUATION_ROOTS,
385         "continuation probe continuation completion mismatch");
386     require(report.rejected_continuations == 0,
387         "continuation probe should not reject continuations");
388     require(report.final_sum == COMPSYS_TEST_CONTINUATION_SUM,
389         "continuation probe final sum mismatch");
390     require(report.max_queue_depth > 0,
391         "continuation probe should observe queued work");
392     require(report.wait_idle_completed,
393         "continuation probe wait_idle should finish with no unfinished work↵
↵ ");
394 }
395
396 void test_task_runtime_report_csv() {
397     compsys::TaskRuntime runtime(COMPSYS_TEST_RUNTIME_WORKERS);
398     const bool accepted = runtime.submit([]() {});
399     require(accepted, "task runtime should accept first task");
400     runtime.wait_idle();
401     runtime.shutdown();
402
403     const compsys::TaskRuntime::Report report = runtime.report();
404     std::ostringstream output;
405     compsys::write_task_runtime_report_csv(output, report);

```

```

406
407     const std::string csv = output.str();
408     require(csv.find("worker_count,active_workers,queued_tasks,max_queue_depth"↵
↵ ) == 0,
409         "task runtime CSV header mismatch");
410     require(csv.find(",1,") != std::string::npos,
411         "task runtime CSV should include a completed count");
412 }
413
414 void test_chapter_models_pipeline_branch_and_memory() {
415     const std::vector<std::int32_t> records{4, -1, 7};
416     const compsys::Chapter01PipelineReport pipeline =
417         compsys::model_ch01_input_pipeline(records);
418     require(pipeline.records_total == 3, "chapter 1 total mismatch");
419     require(pipeline.accepted == 2, "chapter 1 accepted mismatch");
420     require(pipeline.rejected == 1, "chapter 1 rejected mismatch");
421
422     const std::vector<std::int32_t> branch_values{1, 9, 2, 10};
423     const compsys::Chapter02BranchReport branch =
424         compsys::model_ch02_branch_predictor(branch_values, 5);
425     require(branch.iterations == 4, "chapter 2 iteration mismatch");
426     require(branch.actual_taken == 2, "chapter 2 actual taken mismatch");
427     require(branch.mispredictions > 0, "chapter 2 should expose branch misses"↵
↵ );
428
429     const std::vector<std::uint64_t> offsets{0, 64, 4096, 8192, 0};
430     const compsys::Chapter03MemoryAccessReport memory =
431         compsys::model_ch03_cache_tlb(offsets, 64, 4096, 2);
432     require(memory.access_count == 5, "chapter 3 access count mismatch");
433     require(memory.unique_cache_lines == 4, "chapter 3 cache line mismatch");
434     require(memory.unique_pages == 3, "chapter 3 unique page mismatch");
435     require(memory.tlb_misses == 4, "chapter 3 TLB miss mismatch");
436 }
437
438 void test_chapter_models_numa_roofline_and_layout() {
439     const std::vector<std::int32_t> worker_nodes{0, 1, 1};
440     const std::vector<std::int32_t> memory_nodes{0, 0, 1};
441     const compsys::Chapter04NumaReport numa =
442         compsys::model_ch04_numa_coherence(worker_nodes, memory_nodes);
443     require(numa.local_accesses == 2, "chapter 4 local access mismatch");
444     require(numa.remote_accesses == 1, "chapter 4 remote access mismatch");
445     require(numa.coherence_transfers == 1, "chapter 4 transfer mismatch");
446
447     const compsys::Chapter05RooflineReport roofline =
448         compsys::model_ch05_roofline(200.0, 100.0, 10.0, 20.0);
449     require_near(roofline.arithmetic_intensity, 2.0,
450         "chapter 5 arithmetic intensity mismatch");
451     require_near(roofline.memory_bound_gflops, 40.0,
452         "chapter 5 memory roof mismatch");
453     require_near(roofline.attainable_gflops, 10.0,
454         "chapter 5 attainable mismatch");
455     require(!roofline.memory_bound, "chapter 5 should be compute-bound");

```

```

456
457 const compsys::Chapter05bPmuSimdReport simd_evidence =
458     compsys::model_ch05b_pmu_simd_evidence(80, 20, true);
459 require(simd_evidence.retired_instructions == 100,
460     "chapter 5b retired instruction mismatch");
461 require_near(simd_evidence.vector_fraction, 0.2,
462     "chapter 5b vector fraction mismatch");
463 require(simd_evidence.evidence_complete,
464     "chapter 5b evidence should be complete");
465
466 const compsys::Chapter06LayoutReport layout =
467     compsys::model_ch06_layout_locality(10, 4, 1, 8);
468 require(layout.aos_bytes_touched == 320, "chapter 6 AoS bytes mismatch");
469 require(layout.soa_bytes_touched == 80, "chapter 6 SoA bytes mismatch");
470 require(layout.saved_bytes == 240, "chapter 6 saved bytes mismatch");
471 }
472
473 void test_chapter_models_kernels_and_scheduling() {
474     const compsys::Chapter07SimdReport simd =
475         compsys::model_ch07_simd_ilp(10, 4);
476     require(simd.vector_iterations == 2, "chapter 7 vector iteration mismatch")↵↵
477     ↵↵ ;
478     require(simd.scalar_tail == 2, "chapter 7 scalar tail mismatch");
479
480     const std::vector<std::int32_t> kernel_values{1, 2, 3, 4};
481     const compsys::Chapter08KernelReport kernel =
482         compsys::model_ch08_map_filter_reduce(kernel_values);
483     require(kernel.filtered_count == 2, "chapter 8 filtered count mismatch");
484     require(kernel.mapped_sum == 20, "chapter 8 mapped sum mismatch");
485     require(kernel.reduced_sum == 20, "chapter 8 reduced sum mismatch");
486
487     const std::vector<std::int32_t> task_costs{5, 1, 4, 2};
488     const compsys::Chapter09SchedulingReport scheduling =
489         compsys::model_ch09_thread_scheduling(task_costs, 2);
490     require(scheduling.task_count == 4, "chapter 9 task count mismatch");
491     require(scheduling.imbalance > 0, "chapter 9 should expose imbalance");
492
493     const std::array<bool, 3> ready_observations{false, false, true};
494     const compsys::Chapter09bLinuxWaitReport wait =
495         compsys::model_ch09b_linux_wait_wake(ready_observations);
496     require(wait.spurious_wakeups == 2, "chapter 9b spurious wake mismatch");
497     require(wait.completed, "chapter 9b should complete");
498
499 void test_chapter_models_sync_atomic_and_reclamation() {
500     const compsys::Chapter10BackpressureReport backpressure =
501         compsys::model_ch10_sync_backpressure(3, 4, 1, 5);
502     require(backpressure.backpressure_events > 0,
503         "chapter 10 should observe backpressure");
504     require(backpressure.max_depth <= backpressure.capacity,
505         "chapter 10 max depth exceeded capacity");
506

```



```

507     const std::vector<std::int32_t> observed_versions{1, 2, 2, 3};
508     const compsys::Chapter11AtomicPublicationReport atomic =
509         compsys::model_ch11_atomic_publication(observed_versions, 3);
510     require(atomic.stale_reads == 3, "chapter 11 stale read mismatch");
511     require(atomic.monotonic_observations,
512         "chapter 11 observations should be monotonic");
513
514     const std::vector<std::int32_t> ring_operations{1, 1, 1, -1, -1, -1};
515     const compsys::Chapter12LockFreeQueueReport ring =
516         compsys::model_ch12_spsc_ring(ring_operations, 2);
517     require(ring.pushed == 2, "chapter 12 pushed mismatch");
518     require(ring.rejected_full == 1, "chapter 12 rejected mismatch");
519     require(ring.popped == 2, "chapter 12 popped mismatch");
520     require(ring.empty_pops == 1, "chapter 12 empty pop mismatch");
521
522     const std::vector<std::int32_t> retired_nodes{1, 2, 3};
523     const std::vector<std::int32_t> protected_nodes{2};
524     const compsys::Chapter12bReclamationReport reclamation =
525         compsys::model_ch12b_epoch_reclamation(retired_nodes, protected_nodes);
526     require(reclamation.reclaimed_nodes == 2, "chapter 12b reclaimed mismatch")↵
527     ↵ ;
528     require(reclamation.deferred_nodes == 1, "chapter 12b deferred mismatch");
529 }
530
531 void test_chapter_models_runtime_io_and_distributed() {
532     const std::vector<std::int32_t> scan_values{1, 2, 3};
533     const compsys::Chapter13ReduceScanPartitionReport scan =
534         compsys::model_ch13_reduce_scan_partition(scan_values, 2);
535     require(scan.reduce_sum == 6, "chapter 13 reduce sum mismatch");
536     require(scan.last_prefix == 6, "chapter 13 last prefix mismatch");
537     require(scan.partition_true_count == 2, "chapter 13 partition true mismatch")↵
538     ↵ );
539
540     const std::vector<std::int32_t> worker_queues{6, 0, 0};
541     const compsys::Chapter14WorkStealingReport stealing =
542         compsys::model_ch14_work_stealing(worker_queues);
543     require(stealing.stolen_tasks > 0, "chapter 14 should steal tasks");
544     require(stealing.completed_tasks == 6, "chapter 14 completed mismatch");
545
546     const compsys::Chapter15AsyncIoReport io =
547         compsys::model_ch15_async_io_backpressure(5, 2, 1);
548     require(io.submitted == 5, "chapter 15 submitted mismatch");
549     require(io.completed == 5, "chapter 15 completed mismatch");
550     require(io.max_in_flight == 2, "chapter 15 max in-flight mismatch");
551     require(io.backpressure_events > 0,
552         "chapter 15 should observe backpressure");
553
554     const std::vector<std::int32_t> task_ids{1, 1, 2, 3};
555     const std::vector<std::int32_t> generations{2, 2, 1, 2};
556     const compsys::Chapter16DistributedModelReport distributed =
557         compsys::model_ch16_distributed_attempts(task_ids, generations, 2);
558     require(distributed.logical_tasks == 2, "chapter 16 logical task mismatch")↵

```

```

↵ ;
557 require(distributed.duplicate_attempts == 1,
558         "chapter 16 duplicate attempt mismatch");
559 require(distributed.stale_generation_attempts == 1,
560         "chapter 16 stale generation mismatch");
561
562 const std::array<bool, 3> acknowledgements{true, true, false};
563 const compsys::Chapter17ReplicationReport replication =
564     compsys::model_ch17_quorum_commit(3, acknowledgements);
565 require(replication.quorum_size == 2, "chapter 17 quorum mismatch");
566 require(replication.committed, "chapter 17 should commit");
567
568 const compsys::Chapter18RecoveryReport recovery =
569     compsys::model_ch18_checkpoint_recovery(100, 25, 63);
570 require(recovery.checkpoints == 4, "chapter 18 checkpoint mismatch");
571 require(recovery.replay_from_record == 50,
572         "chapter 18 replay start mismatch");
573 require(recovery.replay_records == 50, "chapter 18 replay count mismatch");
574 }
575
576 void test_chapter_model_error_paths() {
577     bool saw_invalid_ring = false;
578     try {
579         const std::vector<std::int32_t> invalid_operations{1, 0};
580         static_cast<void>(compsys::model_ch12_spesc_ring(invalid_operations, 2))↵
581     } catch (const std::runtime_error&) {
582         saw_invalid_ring = true;
583     }
584     require(saw_invalid_ring, "chapter 12 should reject unknown operations");
585
586     bool saw_mismatched_spans = false;
587     try {
588         const std::vector<std::int32_t> task_ids{1};
589         const std::vector<std::int32_t> generations{1, 1};
590         static_cast<void>(
591             compsys::model_ch16_distributed_attempts(task_ids, generations, 1))↵
592     } catch (const std::runtime_error&) {
593         saw_mismatched_spans = true;
594     }
595     require(saw_mismatched_spans, "chapter 16 should reject mismatched spans");
596 }
597
598 } // namespace
599
600 int main() {
601     try {
602         test_parallel_sum();
603         test_counters();
604         test_queue();
605         test_queue_close_wakes_consumer();

```



```
606     test_queue_close_wakes_blocked_producer();
607     test_queue_multi_producer_consumer_drain();
608     test_invalid_queue_capacity();
609     test_permit_limiter();
610     test_phase_barrier();
611     test_future_result_paths();
612     test_atomic_wait_version_counter();
613     test_eventfd_epoll_contract();
614     test_futex_lab_probe();
615     test_same_pool_future_wait_probe();
616     test_continuation_runtime_probe();
617     test_task_runtime_report_csv();
618     test_chapter_models_pipeline_branch_and_memory();
619     test_chapter_models_numa_roofline_and_layout();
620     test_chapter_models_kernels_and_scheduling();
621     test_chapter_models_sync_atomic_and_reclamation();
622     test_chapter_models_runtime_io_and_distributed();
623     test_chapter_model_error_paths();
624     std::cout << "compsys tests passed\n";
625     return EXIT_SUCCESS;
626 } catch (const std::exception& error) {
627     std::cerr << "compsys test failure: " << error.what() << "\n";
628     return EXIT_FAILURE;
629 }
630 }
```

术语表

Compute Systems Engine 本卷贯穿项目，用于承接第二册原理卷中的硬件、并发、运行时、I/O 和分布式计算机制。它不是一个单一 demo，而是一组逐步扩展的工程合同、模型、测试和报告。

reference 参考实现，用清楚、确定、可复查的方式定义正确结果。reference 可以慢，但不能含糊；优化实现必须先与 reference 对齐，才有资格比较性能。

oracle 判定器，用于比较 reference 和待测实现的输出是否满足同一语义。oracle 可以比较精确值，也可以按误差合同比较浮点或统计结果。

contract 合同，代码边界的明确约定：输入、输出、错误、前置条件和不承诺行为。合同越早写成类型和测试，后续重构越不容易漂移。

RecordId 记录身份，用 input name、generation、byte offset 和 line number 定位一条输入。它解决的是“坏行从哪里来”和“同名输入的哪一代产生了这个结果”。

InputCase 一条输入用例，通常包含原始文本、来源和可重放身份。它比裸字符串多了上下文，因此诊断和报告能追溯。

ReferenceResult 参考实现产生的结构化输出，用来喂给 oracle 做正确性判定。它包含输入身份、总记录数、accepted、diagnostics、counts 和 checksum。

ParseDiagnostic 解析阶段产生的结构化错误事实，包含错误类别、位置和可读说明。它不是普通日志，因为测试会断言它的字段。

diagnostic 结构化诊断事实，说明哪条记录为什么被拒绝或哪条边界失败。好的 diagnostic 同时服务机器重放和人类定位。

checksum 校验和，把输出内容压成摘要，用于稳定性检查、截断检查和重放对比。checksum 不能替代完整结果，但能快速发现结果漂移。

manifest 机器可读清单，记录输入版本、输出位置、校验和、参数和提交事实。manifest 的价值在于发布后仍可复查，而不是只在屏幕上看一次。

checkpoint 检查点，保存可恢复状态，让失败后可以从已提交边界继续。checkpoint 要说明覆盖到哪条记录、哪个 epoch、哪些输出已经安全发布。

attempt 一次物理尝试。同一个逻辑任务可能因为失败重试而产生多个 attempt；如果报告不区分 attempt，就会把重复执行误当新任务。

epoch 版本或轮次编号，用来区分不同时刻的状态和提交边界。内存回收、配置发布和分布式提交都常用 epoch 避免旧状态覆盖新状态。

watermark 水位线，表示某个进度之前的数据已经被观察、处理或提交。它常用于流处理、恢复和状态清理，帮助系统判断哪些旧数据可以安全释放。

quorum 法定多数或提交多数。分布式系统里，达到 quorum 后才认为某个值有足够副本支持。第 17 章模型用它解释为什么三个副本通常需要两个确认。

backpressure 反压。当下游处理速度低于上游生产速度时，系统必须限制提交、阻塞生产者、丢弃低优先级任务或扩容。没有 backpressure，队列会无限增长，最终把内存和延迟一起拖垮。

idempotent 幂等，重复执行同一逻辑请求不会产生额外业务结果。重试、manifest、checkpoint 和 RPC 都依赖幂等边界。

split 输入切分后的逻辑片段，通常决定调度、重试和 attempt 身份。split 不能破坏记录边界，否则会产生丢行、重行或错行号。

batch 热路径里成批处理的一组记录，通常服务缓存局部性、SIMD 或线程分摊。split 偏调度身份，batch 偏执行效率，二者不要混用。

partial result 局部结果。每个 split 或 worker 先产出 partial result，再由 merge 合并。partial result 必须保留足够身份和 checksum，才能定位错误来源。

merge 合并步骤。它把多个 partial result 变成一个全局结果。正确 merge 不只是相加，还要处理

- 排序、诊断位置、重复 attempt 和 checksum 合同。
- reference pipeline** 本卷第一条代码主线，从 **event, value** 输入推到 parser、reference、report、manifest、split 和 merge。
- chapter model** 每章机制的小模型。它把原理卷里的概念变成输入字段、输出字段和测试断言，例如 TLB miss、lane utilization、quorum size 或 replay range。
- probe** 探针程序。它运行一个模型或机制并打印稳定输出，方便在书、测试和命令行之间对照。
- PMU: Performance Monitoring Unit** 性能监控单元。它提供硬件事件计数，是判断 SIMD、cache、branch 和 pipeline 行为的重要证据来源。
- SIMD: Single Instruction, Multiple Data** 单指令多数据。一条指令同时处理多个元素。实践代码会同时记录 vector iterations 和 scalar tail，避免只看是否“用了向量指令”。
- ILP: Instruction-Level Parallelism** 指令级并行。独立指令越多，CPU 越可能重叠执行；依赖链、分支和内存等待会降低 ILP。
- TLB: Translation Lookaside Buffer** 地址转换后备缓冲。它缓存虚拟页到物理页的映射；TLB miss 和 cache miss 位置不同、优化手段也不同。
- NUMA: Non-Uniform Memory Access** 非一致内存访问。本地内存和远端内存成本不同，线程绑核、内存分配和共享数据布局都会影响 NUMA 行为。
- roofline** 性能上限模型，用 FLOPs、字节量、内存带宽和峰值计算能力估计程序受计算限制还是受内存限制。它不是最终性能报告，只是定位瓶颈的框架。
- MPMC: Multiple Producer, Multiple Consumer** 多生产者多消费者队列。它必须定义容量、关闭、等待、唤醒和重复消费边界。
- SPSC: Single Producer, Single Consumer** 单生产者单消费者队列。常见于 ring buffer，模型重点是 full、empty、push 和 pop 的边界条件。
- CAS: Compare-And-Swap** 比较并交换。无锁结构常用它尝试发布新状态；CAS 失败表示状态已被别人修改，需要重新读取和决策。
- ABA Problem** ABA 问题。共享值从 A 变 B 又变回 A，观察者只看当前值会误判没有变化。解决方式包括版本号、hazard pointer、epoch 或禁止过早复用节点。
- RCU: Read-Copy-Update** 读-复制-更新。读路径尽量轻，写路径发布新版本并延迟回收旧版本。它帮助解释为什么 epoch reclamation 要知道哪些读者还在旧视图里。
- futex: Fast Userspace Mutex** Linux 快速用户态互斥机制。无竞争时在用户态完成，有竞争时通过内核 wait/wake 阻塞和唤醒。
- eventfd** Linux 事件计数文件描述符。它可被 **epoll** 监听，适合把跨线程完成事件接入统一事件循环。
- epoll** Linux I/O 多路复用接口。它报告文件描述符 ready，但 ready 不等于业务数据已经完全处理；代码仍要 drain、解析和提交。
- future/promise** C++ 异步结果通道。promise 负责设置结果，future 负责等待和读取结果。本卷把它用于解释任务运行时里的等待边界和同池等待风险。
- atomic wait** C++20 原子等待机制。线程可以等待原子值变化，而不是一直自旋。实践中要配合正确的 memory order 和 notify。
- work stealing** 工作窃取。空闲 worker 从其他 worker 的队列偷任务，改善负载不均。模型会记录 stolen tasks、completed tasks 和队列深度。
- CPU: Central Processing Unit** 中央处理器。本卷先把实现、测试和 benchmark 落在本地 CPU 环境。
- C++20** C++ 标准版本名。本卷默认使用的语言和标准库能力边界。
- CMake** C++ 工程构建系统，用于组织 target、测试、benchmark 和工具程序。
- GTest: GoogleTest** C++ 单元测试框架。它固定行为边界，让正文结论能被代码验证。
- GMock: GoogleMock** GoogleTest 的 mock 框架，用于控制依赖和断言交互。
- Google Benchmark** C++ benchmark 库。本卷用它做性能入口，但 benchmark 输出必须配合输入规模、构建类型和正确性校验解释。
- CLI: Command-Line Interface** 命令行接口，用参数、标准输入输出和退出码表达工具行为。

CI: Continuous Integration 持续集成，用自动构建、测试和检查保护每次提交。

JSON: JavaScript Object Notation 结构化文本格式，适合保存配置、manifest 和报告。

CSV: Comma-Separated Values 逗号分隔文本表格，适合保存实验结果和统计数据。

I/O: Input/Output 输入输出。异步 I/O 中 submit、completion 和 commit 不是同一件事。

EOF: End Of File 输入结束。读取代码要区分 EOF、短读和错误。

offset 输入中的字节偏移，用于机器重放和精确定位。

line number 输入文本中的行号，用于人阅读诊断和快速定位错误。

索引

checksum, 4

Compute Systems Engine, ii

contract, 2

diagnostic, 2

reference, 2